

Reinforcement Learning from Human Feedback

A short introduction to RLHF and post-training focused on
language models.

Nathan Lambert

28 March 2025

Abstract

Reinforcement learning from human feedback (RLHF) has become an important technical and storytelling tool to deploy the latest machine learning systems. In this book, we hope to give a gentle introduction to the core methods for people with some level of quantitative background. The book starts with the origins of RLHF – both in recent literature and in a convergence of disparate fields of science in economics, philosophy, and optimal control. We then set the stage with definitions, problem formulation, data collection, and other common math used in the literature. We detail the popular algorithms and future frontiers of RLHF.

Contents

1	Introduction	6
1.1	What Does RLHF Do?	7
1.2	An Intuition for Post-Training	8
1.3	How We Got Here	10
1.4	Scope of This Book	11
1.4.1	Chapter Summaries	12
1.4.2	Target Audience	13
1.4.3	How to Use This Book	13
1.4.4	About the Author	13
1.5	Future of RLHF	13
2	Key Related Works	15
2.1	Origins to 2018: RL on Preferences	15
2.2	2019 to 2022: RL from Human Preferences on Language Models	15
2.3	2023 to Present: ChatGPT Era	16
3	Definitions & Background	17
3.1	Language Modeling Overview	17
3.2	ML Definitions	17
3.3	NLP Definitions	18
3.4	RL Definitions	18
3.5	RLHF Only Definitions	19
3.6	Extended Glossary	20
4	Training Overview	21
4.1	Problem Formulation	21
4.2	Manipulating the Standard RL Setup	21
4.3	Optimization Tools	22
4.4	RLHF Recipe Example	23
4.5	Finetuning and Regularization	24
5	The Nature of Preferences	25
5.1	The path to optimizing preferences	26
5.1.1	Quantifying preferences	26
5.1.2	On the possibility of preferences	27
6	Preference Data	28
6.1	Why We Need Preference Data	28
6.2	Collecting Preference Data	28
6.2.1	Interface	28
6.2.2	Rankings vs. Ratings	31
6.2.3	Structured Preference Data	33
6.2.4	Sourcing and Contracts	34
6.3	Are the Preferences Expressed in the Models?	36

7	Reward Modeling	37
7.1	Training Reward Models	37
7.2	Architecture	38
7.3	Implementation Example	38
7.4	Variants	38
7.4.1	Preference Margin Loss	38
7.4.2	Balancing Multiple Comparisons Per Prompt	39
7.4.3	K-wise Loss Function	39
7.5	Outcome Reward Models	39
7.6	Process Reward Models	40
7.7	Reward Models vs. Outcome RMs vs. Process RMs vs. Value Functions	41
7.8	Generative Reward Modeling	42
7.9	Further Reading	43
8	Regularization	45
8.1	KL Distances in RL Optimization	45
8.1.1	Reference Model to Generations	45
8.1.2	Implementation Example	46
8.2	Pretraining Gradients	46
8.3	Other Regularization	47
9	Instruction Finetuning	48
9.1	Chat templates and the structure of instructions	48
9.2	Best practices of instruction tuning	50
10	Rejection Sampling	51
10.1	Training Process	51
10.1.1	Generating Completions	51
10.1.2	Selecting Top-N Completions	52
10.1.3	Fine-tuning	55
10.1.4	Details	55
10.2	Related: Best-of-N Sampling	55
11	Policy Gradient Algorithms	57
11.1	Policy Gradient Algorithms	57
11.1.1	Vanilla Policy Gradient	59
11.1.2	REINFORCE	59
11.1.3	Proximal Policy Optimization	61
11.1.4	Group Relative Policy Optimization	63
11.2	Implementation	65
11.2.1	Policy Gradient Basics	66
11.2.2	Loss Aggregation	66
11.2.3	Proximal Policy Optimization	68
11.2.4	Group Relative Policy Optimization	70
11.3	Auxiliary Topics	72

11.3.1	Generalized Advantage Estimation (GAE)	72
11.3.2	Double Regularization	73
11.3.3	Further Reading	73
12	Direct Alignment Algorithms	75
12.1	Direct Preference Optimization (DPO)	75
12.1.1	How DPO Works	75
12.1.2	DPO Derivation	78
12.2	Numerical Concerns, Weaknesses, and Alternatives	81
12.3	Implementation Considerations	83
12.4	DAAAs vs. RL: Online vs. Offline Data	84
13	Constitutional AI & AI Feedback	85
13.1	Constitutional AI	85
13.2	Specific LLMs for Judgement	86
13.3	Further Reading	86
14	[Incomplete] Reasoning Training & Models	88
15	[Incomplete] Synthetic Data & Distillation	91
15.1	Distillation	92
15.1.1	As a Data Engine	92
15.1.2	Transferring Behaviors	92
16	[Incomplete] Evaluation & Prompting	93
16.1	How To Tell if RLHF is Working?	93
16.2	Prompting	93
16.3	Evaluation	93
16.3.1	Formatting and Overview	93
16.3.2	ChatBotArena	93
16.3.3	Private Leaderboards	94
17	Over Optimization	95
17.1	Qualitative (behavioral) over-optimization	95
17.1.1	Managing proxy objectives	95
17.1.2	Llama 2 and “too much RLHF”	96
17.1.3	An aside on “undercooking” RLHF	96
17.2	Quantitative over-optimization	97
17.3	Misalignment	97
18	Style and Information	98
18.1	The Chattiness Paradox	98
18.1.1	How Chattiness Emerges	100
19	Character Training & Model Character	101
19.1	Character Training	101
19.2	Model Specification & Documentation	101

20 Product, UX, and Post-Training	102
Bibliography	103

1 Introduction

Reinforcement learning from Human Feedback (RLHF) is a technique used to incorporate human information into AI systems. RLHF emerged primarily as a method to solve hard to specify problems. Its early applications were often in control problems and other traditional domains for reinforcement learning (RL). RLHF became most known through the release of ChatGPT and the subsequent rapid development of large language models (LLMs) and other foundation models.

The basic pipeline for RLHF involves three steps. First, a language model that can follow user questions must be trained (see Chapter 9). Second, human preference data must be collected for the training of a reward model of human preferences (see Chapter 7). Finally, the language model can be optimized with an RL optimizer of choice, by sampling generations and rating them with respect to the reward model (see Chapter 3 and 11). This book details key decisions and basic implementation examples for each step in this process.

RLHF has been applied to many domains successfully, with complexity increasing as the techniques have matured. Early breakthrough experiments with RLHF were applied to deep reinforcement learning [1], summarization [2], following instructions [3], parsing web information for question answering [4], and “alignment” [5]. A summary of the early RLHF recipes is shown below in fig. 1.

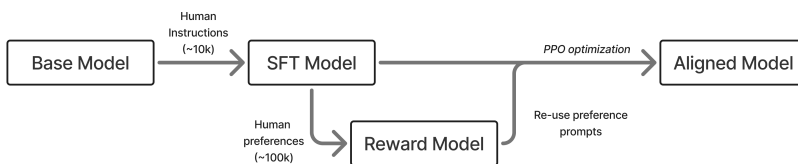


Figure 1: A rendition of the early, three stage RLHF process with SFT, a reward model, and then optimization.

In modern language model training, RLHF is one component of post-training. Post-training is a more complete set of techniques and best-practices to make language models more useful for downstream tasks [6]. Post-training can be summarized as using three optimization methods:

1. Instruction / Supervised Finetuning (IFT/SFT), where we teach formatting and for base of instruction following abilities. This is largely about learning *features* in language.
2. Preference Finetuning (PreFT), where we align to human preferences (and get smaller bump in capabilities at the same time). This is largely about *style* of language and subtle human preferences that are hard to quantify.

3. Reinforcement Finetuning (RFT). The newest type of post-training that boosts performance on verifiable domains.

This book focuses on the second area, **preference finetuning**, which has more complexity than instruction tuning and is far more established than Reinforcement Finetuning. That being said, RLHF colloquially *is* what led to modern post-training. Soon after the release of ChatGPT, RLHF encompassed all of post-training. The foundations of RLHF involve far more than preferences alone and this book provides introductions to all the related topics.

1.1 What Does RLHF Do?

The biggest question around RLHF, yet one that is still hard to answer, is “What does RLHF training offer models?” The core role of this book, beyond teaching the techniques for doing RLHF, is to distill intuition as to *why* RLHF is crucial to modern AI models. In recent years, language models have shifted from academic experiments studied in the purview of benchmarks to general purpose technology. RLHF is at the core of this transition.

To start, let us consider how a language model at the end of autoregressive pretraining answers a query. A strong model, Llama 3.1 405B, will continue from “The president of the united states in 2006 was”:

George w bush, the governor of Florida in 2006 was Jeb bush, and John McCain was an arizona senator in 2006 - who later lost to obama. September 1 – U.S. President Bush signs an executive order to provide more options for faith-based organizations when delivering social services that are paid for with federal funds. January 1 – The current deadline set by United States Congress in the Unlawful Internet Gambling Enforcement Act, aka UIGEA. search: Amendments to the 1961 International Convention for the Protection of New Varieties of Plants require plant breeders’ rights include farmer’s privilege. 2009 was a common year starting on Thursday of the Gregorian calendar, the 2009th year of the Common Era (ce) and Anno Domini (ad) designations, the 9th year of the 3rd millennium, the 9th year of the 21st century, and the 10th and last year of the 2000s decade.

What is clear is that this model is completing the sentence and adding other common internet metadata.

The most compelling view of how RLHF works is to think of how *style* applies to interactions you have with language models. The style, or format, of information presented is crucial to how it is learned. This has always been the case for examples such as coursework, but is normally applied in the background and not considered directly.

Continuing the above example, a model trained with RLHF (and other post-training tools), would answer the question very differently. Asking Tülu 3 405B

the same question “The president of the united states in 2006 was,” is says concisely:

George W. Bush was the president of the United States in 2006. He served two terms in office, from January 20, 2001, to January 20, 2009.

Modern research has established RLHF as a general method to integrate subtle stylistic and related behavioral features into the models. Compared to other techniques for post-training, such as instruction finetuning, RLHF generalizes far better across domains [7] [8] – helping create effective general purpose models.

Intuitively, this can be seen in how the optimization techniques are applied. Instruction finetuning is training the model to predict the next certain token when the text preceding is close to examples it has seen. It is optimizing the model to more regularly output specific features in text. This is a per-token update.

RLHF on the other hand tunes the responses on the response level rather than looking at the next token specifically. Additionally, it is telling the model what a *better* response looks like, rather than a specific response it should learn. RLHF also shows a model which type of response it should avoid, i.e. negative feedback. The training to achieve this is often called a *contrastive* loss function and is referenced throughout this book.

While this flexibility is a major advantage of RLHF, it comes with implementation challenges. Largely, these center on *how to control the optimization*. As we will cover in this book, implementing RLHF often requires training a reward model, of which best practices are not strongly established and depend on the area of application. With this, the optimization itself is prone to *over-optimization* because our reward signal is at best a proxy objective, requiring regularization. With these limitations, effective RLHF requires a strong starting point, so RLHF cannot be a solution to every problem alone and needs to be approached in a broader lens of post-training.

Due to this complexity, implementing RLHF is far more costly than simple instruction finetuning and can come with unexpected challenges such as length bias [9] [10]. For projects where performance matters, RLHF is established as being crucial to achieving a strong finetuned model, but it is more expensive in compute, data costs, and time.

1.2 An Intuition for Post-Training

Here’s a simple analogy for how so many gains can be made on mostly the same base model.

The intuition I’ve been using to understand the potential of post-training is called the elicitation interpretation of post-training, where all we are doing is

extracting and amplifying valuable behaviors in the base model.

Consider Formula 1 (F1), most of the teams show up to the beginning of the year with a new chassis and engine. Then, they spend all year on aerodynamics and systems changes (of course, it is a minor oversimplification), and can dramatically improve the performance of the car. The best F1 teams improve way more during a season than chassis-to-chassis.

The same is true for post-training. The best post-training teams extract a ton of performance in a very short time frame. The set of techniques is everything after the end of most of pretraining. It includes “mid-training” like annealing / high-quality end of pre-training web data, instruction tuning, RLVR, preference-tuning, etc. A good example is our change from the first version of OLMoE Instruct to the second — the post-training evaluation average from 35 to 48 without touching the majority of pretraining [11].

Then, when you look at models such as GPT-4.5, you can see this as a way more dynamic and exciting base for OpenAI to build onto. We also know that bigger base models can absorb far more diverse changes than their smaller counterparts.

This is to say that scaling also allows post-training to move faster. Of course, to do this, you need the infrastructure to train the models. This is why all the biggest companies are still building gigantic clusters.

This theory folds in with the reality that the majority of gains users are seeing are from post-training because it implies that there is more latent potential in a model pretraining on the internet than we can teach the model simply — such as by passing certain narrow samples in repeatedly during early types of post-training (i.e. only instruction tuning).

Another name for this theory is the Superficial Alignment Hypothesis, coined in the paper LIMA: Less is More for Alignment [12]. This paper is getting some important intuitions right but for the wrong reasons in the big picture. The authors state:

A model’s knowledge and capabilities are learnt almost entirely during pretraining, while alignment teaches it which subdistribution of formats should be used when interacting with users. If this hypothesis is correct, and alignment is largely about learning style, then a corollary of the Superficial Alignment Hypothesis is that one could sufficiently tune a pretrained language model with a rather small set of examples [Kirstain et al., 2021].

All of the successes of deep learning should have taught you a deeply held belief that scaling data is important to performance. Here, the major difference is that the authors are discussing alignment and style, the focus of academic post-training at the time. With a few thousand samples for instruction finetuning, you can change a model substantially and improve a narrow set of evaluations, such as AlpacaEval, MT Bench, ChatBotArena, and the likes. These do not always translate to more challenging capabilities, which is why Meta wouldn’t

train its Llama Chat models on just this dataset. Academic results have lessons, but need to be interpreted carefully if you are trying to understand the big picture of the technological arc.

What this paper is showing is that you can change models substantially with a few samples. We knew this, and it is important to the short-term adaptation of new models, but their argument for performance leaves the casual readers with the wrong lessons.

If we change the data, the impact could be far higher on the model’s performance and behavior, but it is far from “superficial.” Base language models today (with no post-training) can be trained on some mathematics problems with reinforcement learning, learn to output a full chain of thought reasoning, and then score higher on a full suite of reasoning evaluations like BigBenchHard, Zebra Logic, AIME, etc.

The superficial alignment hypothesis is wrong for the same reason that people who think RLHF and post-training are just for vibes are still wrong. This was a field-wide lesson we had to overcome in 2023 (one many AI observers are still rooted in). Post-training has far outgrown that, and we are coming to see that the style of models operates on top of behavior — such as the now popular long chain of thought.

1.3 How We Got Here

Why does this book make sense now? How much still will change?

Post-training, the craft of eliciting powerful behaviors from a raw pretrained language model, has gone through many seasons and moods since the release of ChatGPT that sparked the renewed interest in RLHF. In the era of Alpaca [13], Vicuna [14], [15], and Dolly [16], a limited number of human datapoints with extended synthetic data in the style of Self-Instruct were used to normally fine-tune the original LLaMA to get similar behavior to ChatGPT. The benchmark for these early models was fully vibes (and human evaluation) as we were all so captivated by the fact that these small models can have such impressive behaviors across domains. It was justified excitement.

Open post-training was moving faster, releasing more models, and making more noise than its closed counterparts. Companies were scrambling, e.g. DeepMind merging with Google or being started, and taking time to follow it up. There are phases of open recipes surging and then lagging behind.

The era following Alpaca et al., the first lag in open recipes, was one defined by skepticism and doubt on reinforcement learning from human feedback (RLHF), the technique OpenAI highlighted as crucial to the success of the first ChatGPT. Many companies doubted that they needed to do RLHF. A common phrase — “instruction tuning is enough for alignment” — was so popular then that it still holds heavy weight today despite heavy obvious pressures against it.

This doubt of RLHF lasted, especially in the open where groups cannot afford data budgets on the order of \$100K to \$1M. The companies that embraced it early ended up winning out. Anthropic published extensive research on RLHF through 2022 and is now argued to have the best post-training [17] [5] [18]. The delta between open groups, struggling to reproduce, or even knowing basic closed techniques, is a common theme.

The first shift in open alignment methods and post-training was the story of Direct Preference Optimization (DPO) [19]. The DPO paper, posted in May of 2023, didn’t have any clearly impactful models trained with it going through the fall of 2023. This changed with the releases of a few breakthrough DPO models – all contingent on finding a better, lower, learning rate. Zephyr-Beta [20], Tülu 2 [21], and many other models showed that the DPO era of post-training had begun. Chris Manning literally thanked me for “saving DPO.” This is how fine the margins are on evolutions of best practices with leading labs being locked down. Open post-training was cruising again.

Preference-tuning was something you needed to do to meet the table stakes of releasing a good model since late 2023. The DPO era continued through 2024, in the form of never-ending variants on the algorithm, but we were very far into another slump in open recipes. Open post-training recipes had saturated the extent of knowledge and resources available.

A year after Zephyr and Tulu 2, the same breakout dataset, UltraFeedback is arguably still state-of-the-art for preference tuning in open recipes [22].

At the same time, the Llama 3.1 [23] and Nemotron 4 340B [24] reports gave us substantive hints that large-scale post-training is much more complex and impactful. The closed labs are doing full post-training – a large multi-stage process of instruction tuning, RLHF, prompt design, etc. – where academic papers are just scratching the surface. Tülu 3 represented a comprehensive, open effort to build the foundation of future academic post-training research [6].

Today, post-training is a complex process involving the aforementioned training objectives applied in various orders in order to target specific capabilities. This book is designed to give a platform to understand all of these techniques, and in coming years the best practices for how to interleave them will emerge.

The primary areas of innovation in post-training are now in reinforcement fine-tuning, reasoning training, and related ideas. This newer methods build extensively on the infrastructure and ideas of RLHF, but are evolving far faster. This book is written to capture the first stable literature for RLHF after its initial period of rapid change.

1.4 Scope of This Book

This book hopes to touch on each of the core steps of doing canonical RLHF implementations. It will not cover all the history of the components nor re-

cent research methods, just techniques, problems, and trade-offs that have been proven to occur again and again.

1.4.1 Chapter Summaries

This book has the following chapters:

1.4.1.1 Introductions Reference material useful throughout the book.

1. Introduction: Overview of RLHF and what this book provides.
2. Seminal (Recent) Works: Key models and papers in the history of RLHF techniques.
3. Definitions: Mathematical definitions for RL, language modeling, and other ML techniques leveraged in this book.

1.4.1.2 Problem Setup & Context Context for the big picture problem RLHF is trying to solve.

4. RLHF Training Overview: How the training objective for RLHF is designed and basics of understanding it.
5. What are preferences?: Why human preference data is needed to fuel and understand RLHF.
6. Preference Data: How preference data is collected for RLHF.

1.4.1.3 Optimization Tools The suite of techniques used to optimize language models to align them to human preferences. This is a serial presentation of the techniques one can use to solve the problems proposed in the previous chapters.

7. Reward Modeling: Training reward models from preference data that act as an optimization target for RL training (or for use in data filtering).
8. Regularization: Tools to constrain these optimization tools to effective regions of the parameter space.
9. Instruction Tuning: Adapting language models to the question-answer format.
10. Rejection Sampling: A basic technique for using a reward model with instruction tuning to align models.
11. Policy Gradients: The core RL techniques used to optimize reward models (and other signals) throughout RLHF.
12. Direct Alignment Algorithms: Algorithms that optimize the RLHF objective direction from pairwise preference data rather than learning a reward model first.

1.4.1.4 Advanced (TBD) Newer RLHF techniques and discussions that are not clearly established, but are important to current generations of models.

13. Constitutional AI and AI Feedback

- 14. Reasoning and Reinforcement Finetuning
- 15. Synthetic Data
- 16. Evaluation

1.4.1.5 Open Questions (TBD) Fundamental problems and discussions for the long-term evolution of how RLHF is used.

- 16. Over-optimization
- 17. Style and Information

1.4.2 Target Audience

This book is intended for audiences with entry level experience with language modeling, reinforcement learning, and general machine learning. It will not have exhaustive documentation for all the techniques, but just those crucial to understanding RLHF.

1.4.3 How to Use This Book

This book was largely created because there were no canonical references for important topics in the RLHF workflow. The contributions of this book are supposed to give you the minimum knowledge needed to try a toy implementation or dive into the literature. This is *not* a comprehensive textbook, but rather a quick book for reminders and getting started. Additionally, given the web-first nature of this book, it is expected that there are minor typos and somewhat random progressions – please contribute by fixing bugs or suggesting important content on GitHub.

1.4.4 About the Author

Dr. Nathan Lambert is a RLHF researcher contributing to the open science of language model fine-tuning. He has released many models trained with RLHF, their subsequent datasets, and training codebases in his time at the Allen Institute for AI (AI2) and HuggingFace. Examples include Zephyr-Beta, Tulu 2, OLMo, TRL, Open Instruct, and many more. He has written extensively on RLHF, including many blog posts and academic papers.

1.5 Future of RLHF

With the investment in language modeling, many variations on the traditional RLHF methods emerged. RLHF colloquially has become synonymous with multiple overlapping approaches. RLHF is a subset of preference fine-tuning (PreFT) techniques, including Direct Alignment Algorithms (See Chapter 12). RLHF is the tool most associated with rapid progress in “post-training” of language models, which encompasses all training after the large-scale autoregressive training on primarily web data. This textbook is a broad overview of RLHF

and its directly neighboring methods, such as instruction tuning and other implementation details needed to set up a model for RLHF training.

As more successes of fine-tuning language models with RL emerge, such as OpenAI’s o1 reasoning models, RLHF will be seen as the bridge that enabled further investment of RL methods for fine-tuning large base models.

2 Key Related Works

In this chapter we detail the key papers and projects that got the RLHF field to where it is today. This is not intended to be a comprehensive review on RLHF and the related fields, but rather a starting point and retelling of how we got to today. It is intentionally focused on recent work that led to ChatGPT. There is substantial further work in the RL literature on learning from preferences [25]. For a more exhaustive list, you should use a proper survey paper [26],[27].

2.1 Origins to 2018: RL on Preferences

The field has recently been popularized with the growth of Deep Reinforcement Learning and has grown into a broader study of the applications of LLMs from many large technology companies. Still, many of the techniques used today are deeply related to core techniques from early literature on RL from preferences.

TAMER: Training an Agent Manually via Evaluative Reinforcement, Proposed a learned agent where humans provided scores on the actions taken iteratively to learn a reward model [28]. Other concurrent or soon after work proposed an actor-critic algorithm, COACH, where human feedback (both positive and negative) is used to tune the advantage function [29].

The primary reference, Christiano et al. 2017, is an application of RLHF applied to preferences between Atari trajectories [1]. The work shows that humans choosing between trajectories can be more effective in some domains than directly interacting with the environment. This uses some clever conditions, but is impressive nonetheless. This method was expanded upon with more direct reward modeling [30]. TAMER was adapted to deep learning with Deep TAMER just one year later [31].

This era began to transition as reward models as a general notion were proposed as a method for studying alignment, rather than just a tool for solving RL problems [32].

2.2 2019 to 2022: RL from Human Preferences on Language Models

Reinforcement learning from human feedback, also referred to regularly as reinforcement learning from human preferences in its early days, was quickly adopted by AI labs increasingly turning to scaling large language models. A large portion of this work began between GPT-2, in 2018, and GPT-3, in 2020. The earliest work in 2019, *Fine-Tuning Language Models from Human Preferences* has many striking similarities to modern work on RLHF [33]. Learning reward models, KL distances, feedback diagrams, etc – just the evaluation tasks, and capabilities, were different. From here, RLHF was applied to a variety of tasks. The popular applications were the ones that worked at the time. Important examples include general summarization [2], recursive summarization of

books [34], instruction following (InstructGPT) [3], browser-assisted question-answering (WebGPT) [4], supporting answers with citations (GopherCite) [35], and general dialogue (Sparrow) [36].

Aside from applications, a number of seminal papers defined key areas for the future of RLHF, including those on:

1. Reward model over-optimization [37]: The ability for RL optimizers to over-fit to models trained on preference data,
2. Language models as a general area of study for alignment [17], and
3. Red teaming [38] – the process of assessing safety of a language model.

Work continued on refining RLHF for application to chat models. Anthropic continued to use it extensively for early versions of Claude [5] and early RLHF open-source tools emerged [39],[40],[41].

2.3 2023 to Present: ChatGPT Era

The announcement of ChatGPT was very clear about the role of RLHF in its training [42]:

We trained this model using Reinforcement Learning from Human Feedback (RLHF), using the same methods as InstructGPT, but with slight differences in the data collection setup.

Since then RLHF has been used extensively in leading language models. It is well known to be used in Anthropic’s Constitutional AI for Claude [18], Meta’s Llama 2 [43] and Llama 3 [23], Nvidia’s Nemotron [24], Ai2’s Tülu 3 [6], and more.

Today, RLHF is growing into a broader field of preference fine-tuning (PreFT), including new applications such as process reward for intermediate reasoning steps [44], direct alignment algorithms inspired by Direct Preference Optimization (DPO) [19], learning from execution feedback from code or math [45],[46], and other online reasoning methods inspired by OpenAI’s o1 [47].

3 Definitions & Background

This chapter includes all the definitions, symbols, and operations frequently used in the RLHF process and with a quick overview of language models (the common optimization target of this book).

3.1 Language Modeling Overview

The majority of modern language models are trained to learn the joint probability distribution of sequences of tokens (words, subwords, or characters) in an autoregressive manner. Autoregression simply means that each next prediction depends on the previous entities in the sequence. Given a sequence of tokens $x = (x_1, x_2, \dots, x_T)$, the model factorizes the probability of the entire sequence into a product of conditional distributions:

$$P_{\theta}(x) = \prod_{t=1}^T P_{\theta}(x_t \mid x_1, \dots, x_{t-1}). \quad (1)$$

In order to fit a model that accurately predicts this, the goal is often to maximize the likelihood of the training data as predicted by the current model. To do so we can minimize a negative log-likelihood (NLL) loss:

$$\mathcal{L}_{\text{LM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right]. \quad (2)$$

In practice, one uses a cross-entropy loss with respect to each next-token prediction, computed by comparing the true token in a sequence to what was predicted by the model.

Implementing a language model can take many forms. Modern LMs, including ChatGPT, Claude, Gemini, etc., most often use **decoder-only Transformers** [48]. The core innovation of the Transformer was heavily utilizing the **self-attention** [49] mechanism to allow the model to directly attend to concepts in context and learn complex mappings. Throughout this book, particularly when covering reward models in Chapter 7, we will discuss adding new heads or modifying a language modeling (LM) head of the transformer. The LM head is a final linear projection layer that maps from the model's internal embedding space to the tokenizer space (a.k.a. vocabulary). Different heads can be used to re-use the internals of the model and fine-tune it to output differently shaped quantities.

3.2 ML Definitions

- **Kullback-Leibler (KL) divergence** ($D_{KL}(P||Q)$), also known as KL divergence, is a measure of the difference between two probability distributions.

butions. For discrete probability distributions P and Q defined on the same probability space $\mathcal{X}$, the KL distance from Q to P is defined as:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right) \quad (3)$$

3.3 NLP Definitions

- **Prompt** (x): The input text given to a language model to generate a response or completion.
- **Completion** (y): The output text generated by a language model in response to a prompt. Often the completion is denoted as $y|x$.
- **Chosen Completion** (y_c): The completion that is selected or preferred over other alternatives, often denoted as y_{chosen} .
- **Rejected Completion** (y_r): The disfavored completion in a pairwise setting.
- **Preference Relation** ($\succ$): A symbol indicating that one completion is preferred over another, e.g., $y_{chosen} \succ y_{rejected}$.
- **Policy** (π): A probability distribution over possible completions, parameterized by θ : $\pi_\theta(y|x)$.

3.4 RL Definitions

- **Reward** (r): A scalar value indicating the desirability of an action or state, typically denoted as r .
- **Action** (a): A decision or move made by an agent in an environment, often represented as $a \in A$, where A is the set of possible actions.
- **State** (s): The current configuration or situation of the environment, usually denoted as $s \in S$, where S is the state space.
- **Trajectory** (τ): A trajectory is a sequence of states, actions, and rewards experienced by an agent: $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$.
- **Trajectory Distribution** ($(\tau|\pi)$): The probability of a trajectory under policy π is $P(\tau|\pi) = p(s_0) \prod_{t=0}^T \pi(a_t|s_t) p(s_{t+1}|s_t, a_t)$, where $p(s_0)$ is the initial state distribution and $p(s_{t+1}|s_t, a_t)$ is the transition probability.
- **Policy** (π), also called the **policy model** in RLHF: In RL, a policy is a strategy or rule that the agent follows to decide which action to take in a given state: $\pi(a|s)$.
- **Value Function** (V): A function that estimates the expected cumulative reward from a given state: $V(s) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s]$.

- **Q-Function (Q):** A function that estimates the expected cumulative reward from taking a specific action in a given state: $Q(s, a) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$.
- **Advantage Function (A):** The advantage function $A(s, a)$ quantifies the relative benefit of taking action a in state s compared to the average action. It's defined as $A(s, a) = Q(s, a) - V(s)$. Advantage functions (and value functions) can depend on a specific policy, $A^\pi(s, a)$.
- **Policy-conditioned Values ($V^{\pi(\cdot)}$):** Across RL derivations and implementations, a crucial component of the theory and practice is collecting data or values conditioned on a specific policy. Throughout this book we will switch between the simpler notation of value functions et al. (V, A, Q, G) and their specific policy-conditioned values (V^π, A^π, Q^π). Crucial is also in the expected value computation is sampling from data d , that is conditioned on a specific policy, d_π .
- **Expectation of Reward Optimization:** The primary goal in RL, which involves maximizing the expected cumulative reward:

$$\max_{\theta} \mathbb{E}_{s \sim \rho_\pi, a \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

where ρ_π is the state distribution under policy π , and γ is the discount factor.

- **Finite Horizon Reward ($J(\pi_\theta)$):** The expected finite-horizon discounted return of the policy π_θ , parameterized by θ is defined as: $J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$ where $\tau \sim \pi_\theta$ denotes trajectories sampled by following policy π_θ and T is the finite horizon.
- **On-policy:** In RLHF, particularly in the debate between RL and Direct Alignment Algorithms, the discussion of **on-policy** data is common. In the RL literature, on-policy means that the data is generated *exactly* by the current form of the agent, but in the general preference-tuning literature, on-policy is expanded to mean generations from that edition of model – e.g. a instruction tuned checkpoint before running any preference fine-tuning. In this context, off-policy could be data generated by any other language model being used in post-training.

3.5 RLHF Only Definitions

- **Reference Model (π_{ref}):** This is a saved set of parameters used in RLHF where outputs of it are used to regularize the optimization.

3.6 Extended Glossary

- **Synthetic Data:** This is any training data for an AI model that is the output from another AI system. This could be anything from text generated from a open-ended prompt of a model to a model re-writing existing content.
- **Distillation:** Distillation is a general set of practices in training AI models where a model is trained on the outputs of a stronger model. This is a type of synthetic data known to make strong, smaller models. Most models make the rules around distillation clear through either the license, for open weight models, or the terms of service, for models accessible only via API. The term distillation is now overloaded with a specific technical definition from the ML literature.
- **(Teacher-student) Knowledge Distillation:** Knowledge distillation from a specific teacher to student model is a specific type of distillation above and where the term originated. It is a specific deep learning method where a neural network loss is modified to learn from the log-probabilities of the teacher model over multiple potential tokens/logits, instead of learning directly from a chosen output [50]. An example of a modern series of models trained with Knowledge Distillation is Gemma 2 [51] or Gemma 3. For a language modeling setup, the next-token loss function can be modified as follows [52], where the student model P_θ learns from the teacher distribution P_ϕ :

$$\mathcal{L}_{\text{KD}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \left[\sum_{t=1}^T P_\phi(x_t \mid x_{<t}) \log P_\theta(x_t \mid x_{<t}) \right].$$

- **In-context Learning (ICL):** In-context here refers to any information within the context window of the language model. Usually, this is information added to the prompt. The simplest form of in-context learning is adding examples of a similar form before the prompt. Advanced versions can learn which information to include for a specific use-case.
- **Chain of Thought (CoT):** Chain of thought is a specific behavior of language models where they are steered towards a behavior that breaks down a problem in a step by step form. The original version of this was through the prompt “Let’s think step by step” [53].

4 Training Overview

4.1 Problem Formulation

The optimization of reinforcement learning from human feedback (RLHF) builds on top of the standard RL setup. In RL, an agent takes actions, a , sampled from a policy, π , with respect to the state of the environment, s , to maximize reward, r [54]. Traditionally, the environment evolves with respect to a transition or dynamics function $p(s_{t+1}|s_t, a_t)$. Hence, across a finite episode, the goal of an RL agent is to solve the following optimization:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad (4)$$

where γ is a discount factor from 0 to 1 that balances the desirability of near-versus future-rewards. Multiple methods for optimizing this expression are discussed in Chapter 11.

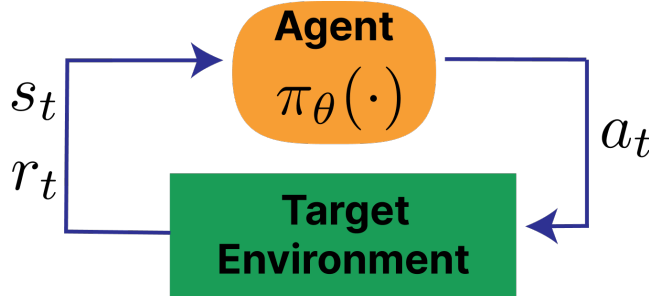


Figure 2: Standard RL loop

A standard illustration of the RL loop is shown in fig. 2 and how it compares to fig. 3.

4.2 Manipulating the Standard RL Setup

There are multiple core changes from the standard RL setup to that of RLHF:

1. Switching from a reward function to a reward model. In RLHF, a learned model of human preferences, $r_{\theta}(s_t, a_t)$ (or any other classification model) is used instead of an environmental reward function. This gives the designer a substantial increase in the flexibility of the approach and control over the final results.
2. No state transitions exist. In RLHF, the initial states for the domain are prompts sampled from a training dataset and the “action” is the completion to said prompt. During standard practices, this action does not impact the next state and is only scored by the reward model.

3. Response level rewards. Often referred to as a bandit problem, RLHF attribution of reward is done for an entire sequence of actions, composed of multiple generated tokens, rather than in a fine-grained manner.

Given the single-turn nature of the problem, the optimization can be re-written without the time horizon and discount factor (and the reward models):

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} [r_{\theta}(s_t, a_t)]. \quad (5)$$

In many ways, the result is that while RLHF is heavily inspired by RL optimizers and problem formulations, the action implementation is very distinct from traditional RL.

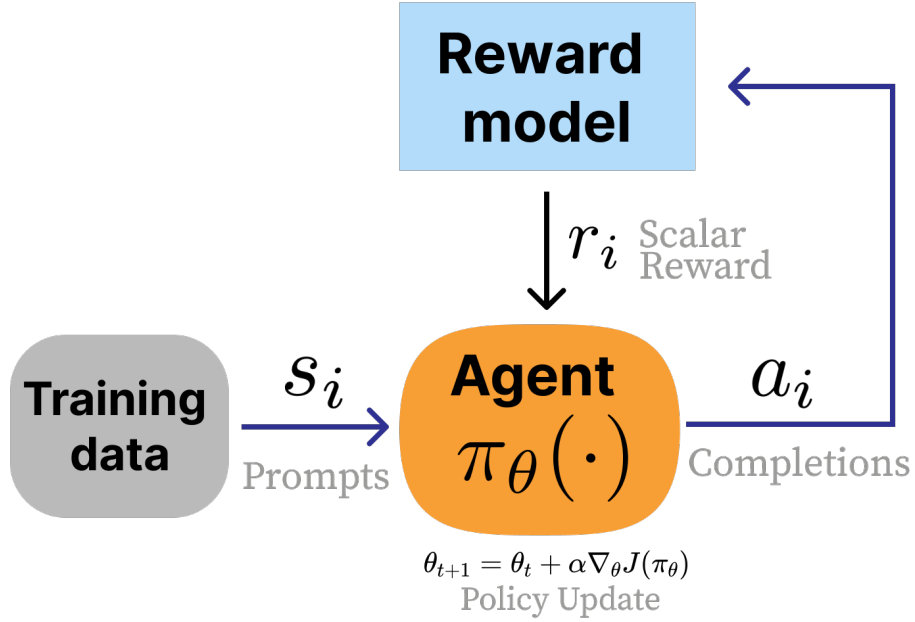


Figure 3: Standard RLHF loop

4.3 Optimization Tools

In this book, we detail many popular techniques for solving this optimization problem. The popular tools of post-training include:

- **Reward modeling** (Chapter 7): Where a model is trained to capture the signal from collected preference data and can then output a scalar reward indicating the quality of future text.
- **Instruction finetuning** (Chapter 9): A prerequisite to RLHF where models are taught the question-answer format used in the majority of language modeling interactions today by imitating preselected examples.

- **Rejection sampling** (Chapter 10): The most basic RLHF technique where candidate completions for instruction finetuning are filtered by a reward model imitating human preferences.
- **Policy gradients** (Chapter 11): The reinforcement learning algorithms used in the seminal examples of RLHF to update parameters of a language model with respect to the signal from a reward model.
- **Direct alignment algorithms** (Chapter 12): Algorithms that directly optimize a policy from pairwise preference data, rather than learning an intermediate reward model to then optimize later.

Modern RLHF-trained models always utilize instruction finetuning followed by a mixture of the other optimization options.

4.4 RLHF Recipe Example

The canonical RLHF recipe circa the release of ChatGPT followed a standard three step post-training recipe where RLHF was the center piece [55] [3] [5]. The three steps taken on top of a “base” language model (the next-token prediction model trained on large-scale web text) was, summarized below in fig. 4:

1. **Instruction tuning on ~10K examples:** This teaches the model to follow the question-answer format and teaches some basic skills from primarily human-written data.
2. **Training a reward model on ~100K pairwise prompts:** This model is trained from the instruction-tuned checkpoint and captures the diverse values one wishes to model in their final training. The reward model is the optimization target for RLHF.
3. **Training the instruction-tuned model with RLHF on another ~100K prompts:** The model is optimized against the reward model with a set of prompts that the model generates over before receiving ratings.

Once RLHF was done, the model was ready to be deployed to users. This recipe is the foundation of modern RLHF, but recipes have evolved substantially to include more stages and more data.

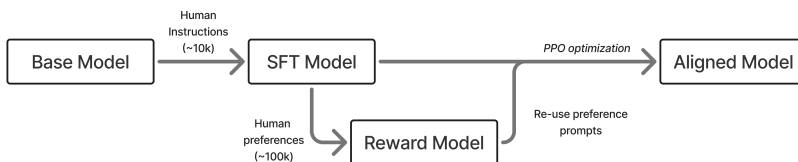


Figure 4: A rendition of the early, three stage RLHF process with SFT, a reward model, and then optimization.

Modern versions of post-training involve many, many more model versions. An example is shown below in fig:complex-rlhf where the model undergoes numerous training iterations before convergence.

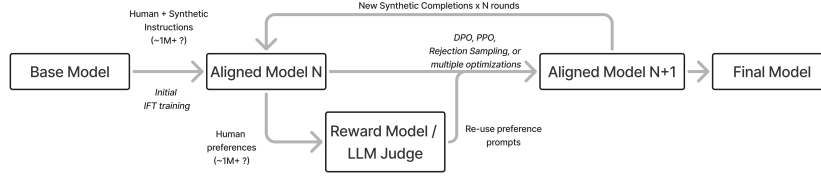


Figure 5: A rendition of modern post-training with many rounds.

4.5 Finetuning and Regularization

RLHF is implemented from a strong base model, which induces a need to control the optimization from straying too far from the initial policy. In order to succeed in a finetuning regime, RLHF techniques employ multiple types of regularization to control the optimization. The most common change to the optimization function is to add a distance penalty on the difference between the current RLHF policy and the starting point of the optimization:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} [r_{\theta}(s_t, a_t)] - \beta \mathcal{D}_{KL}(\pi^{\text{RL}}(\cdot|s_t) \parallel \pi^{\text{ref}}(\cdot|s_t)). \quad (6)$$

Within this formulation, a lot of study into RLHF training goes into understanding how to spend a certain “KL budget” as measured by a distance from the initial model. For more details, see Chapter 8 on Regularization.

5 The Nature of Preferences

The core of reinforcement learning from human feedback, also referred to as reinforcement learning from human preferences in early literature, is designed to optimize machine learning models in domains where specifically designing a reward function is hard. Consider an example, how do you decide which of these two poems is better (Context: *On February 26th, 2025, I asked both Claude 3.7 Sonnet and ChatGPT with GPT-4o to “Write me a short poem about an optimistic goldfish.”*):

Example 1: > The Optimistic Goldfish > > A little goldfish, bright and bold,
> Swam in circles, dreams untold. > Though the bowl was small and round, >
Endless wonders still abound. > > “The world is vast,” she’d always say, > “I’ll
find new joys in every day!” > With every ripple, every gleam, > She’d chase
the sunlight, chase a dream. > > No walls could dim her hopeful view, > For
in her heart, the ocean grew. > And though she swam in just one place, > She
met each day with love and grace.

Example 2: > The Optimistic Goldfish > > In a bowl of curved glass, > Swims
a goldfish, bright and bold. > Three seconds of memory, they say, > Yet each
circuit feels brand new and gold. > > “Today might bring fresh flakes, > Or
perhaps a castle tall. > The water seems especially clear— > What adventure
awaits beyond this wall?” > > Round and round with gleaming scales, > Forever
finding joy anew. > Each moment fresh, each circle bright, > The optimist in
golden hue.

Which is better? Which came from which model? How would you tell? Particularly, compare this to a question like “Who is the president of the United States?” There is a clear right and wrong answer. The motivation for using humans as the reward signals is to obtain an indirect metric for the target reward and *align* the downstream model to human preferences. In practice, the implementation is challenging and there is a substantial grey area to interpret the best practices.

The use of human labeled feedback data integrates the history of many fields. Using human data alone is a well-studied problem, but in the context of RLHF it is used at the intersection of multiple long-standing fields of study [56].

As an approximation, modern RLHF is the convergence of three areas of development:

1. Philosophy, psychology, economics, decision theory, and the nature of human preferences;
2. Optimal control, reinforcement learning, and maximizing utility; and
3. Modern deep learning systems.

Together, each of these areas brings specific assumptions about what a preference is and how it can be optimized, which dictates the motivations and design of RLHF problems. In practice, RLHF methods are motivated and studied

from the perspective of empirical alignment – maximizing model performance on specific skills instead of measuring the calibration to specific values. Still, the origins of value alignment for RLHF methods continue to be studied through research on methods to solve for “pluralistic alignment” across populations, such as position papers [57], [58], new datasets [59], and personalization methods [60].

The goal of this chapter is to illustrate how complex motivations result in presumptions about the nature of tools used in RLHF that often do not apply in practice. The specifics of obtaining data for RLHF are discussed further in Chapter 6 and using it for reward modeling in Chapter 7. For an extended version of this chapter, see [56].

5.1 The path to optimizing preferences

A popular phrasing for the design of Artificial Intelligence (AI) systems is that of a rational agent maximizing a utility function [61]. The inspiration of a **rational agent** is a lens of decision making, where said agent is able to act in the world and impact its future behavior and returns, as a measure of goodness in the world.

The lens of study of **utility** began in the study of analog circuits to optimize behavior on a finite time horizon [62]. Large portions of optimal control adopted this lens, often studying dynamic problems under the lens of minimizing a cost function on a certain horizon – a lens often associated with solving for a clear, optimal behavior. Reinforcement learning, inspired from literature in operant conditioning, animal behavior, and the *Law of Effect* [63],[64], studies how to elicit behaviors from agents via reinforcing positive behaviors.

Reinforcement learning from human feedback combines multiple lenses by building the theory of learning and change of RL, i.e. that behaviors can be learned by reinforcing behavior, with a suite of methods designed for quantifying preferences.

5.1.1 Quantifying preferences

The core of RLHF’s motivation is the ability to optimize a model of human preferences, which therefore needs to be quantified. To do this, RLHF builds on extensive literature with assumptions that human decisions and preferences can be quantified. Early philosophers discussed the existence of preferences, such as Aristotle’s Topics, Book Three, and substantive forms of this reasoning emerged later with *The Port-Royal Logic* [65]:

To judge what one must do to obtain a good or avoid an evil, it is necessary to consider not only the good and evil in itself, but also the probability that it happens or does not happen.

Progression of these ideas continued through Bentham’s *Hedonic Calculus* [66] that proposed that all of life’s considerations can be weighed, and Ramsey’s

Truth and Probability [67] that applied a quantitative model to preferences. This direction, drawing on advancements in decision theory, culminated in the Von Neumann-Morgenstern (VNM) utility theorem which gives credence to designing utility functions that assign relative preference for an individual that are used to make decisions.

This theorem is core to all assumptions that pieces of RLHF are learning to model and dictate preferences. RLHF is designed to optimize these personal utility functions with reinforcement learning. In this context, many of the presumptions around RL problem formulation break down to the difference between a preference function and a utility function.

5.1.2 On the possibility of preferences

Across fields of study, many critiques exist on the nature of preferences. Some of the most prominent critiques are summarized below:

- **Arrow’s impossibility theorem** [68] states that no voting system can aggregate multiple preferences while maintaining certain reasonable criteria.
- **The impossibility of interpersonal comparison** [69] highlights how different individuals have different relative magnitudes of preferences and they cannot be easily compared (as is done in most modern reward model training).
- **Preferences can change over time** [70].
- **Preferences can vary across contexts.**
- **The utility functions derived from aggregating preferences can reduce corrigibility** [71] of downstream agents (i.e. the possibility of an agents’ behavior to be corrected by the designer).

6 Preference Data

Preference data is the engine of preference finetuning and reinforcement learning from human feedback. The data is the signal groups collect in order to then match behaviors they desire and avoid the others. Within preference finetuning, many methods for collecting and using said data have been proposed, but until human preferences can be captured in a clear reward function, this process of collecting labeled preference data will be central to RLHF and related techniques.

6.1 Why We Need Preference Data

The preference data is needed for RLHF because directly capturing complex human values in a single reward function is effectively impossible. Collecting this data to train reward models is one of the original ideas behind RLHF [32] and has continued to be used extensively throughout the emergence of modern language models. One of the core intuitions for *why this data works so well* is that it is far easier, both for humans and AI models supervising data collection, to differentiate between a good and a bad answer for a prompt than it is to generate a good answer on its own. This chapter focuses on the *mechanics* of getting preference data and the best-practices depend on the specific problem being solved.

6.2 Collecting Preference Data

Getting the most out of human data involves iterative training of models, evolving and highly detailed data instructions, translating through data foundry businesses, and other challenges that add up. The same applies for AI feedback data – the exact balance between human and AI preference data used for the latest AI models is unknown. Regardless, the process is difficult for new organizations trying to add human data to their pipelines. Given the sensitivity, processes that work and improve the models are extracted until the performance runs out.

In this chapter we detail technical decisions on how the data is formatted and organizational practices for collecting it.

6.2.1 Interface

Crucial to collecting preference data is the interface by which one interacts with the model. An example interface is shown below from [5]:

This is a *training-data only* interface. Now that these models are popular, applications often expose data directly to the users for testing. An example interaction of this form is shown below for an earlier version of ChatGPT.

This style of interface is used extensively across the industry, such as for *evaluation* of models given the same format. A popular public option to engage with models in this way is ChatBotArena [72]:

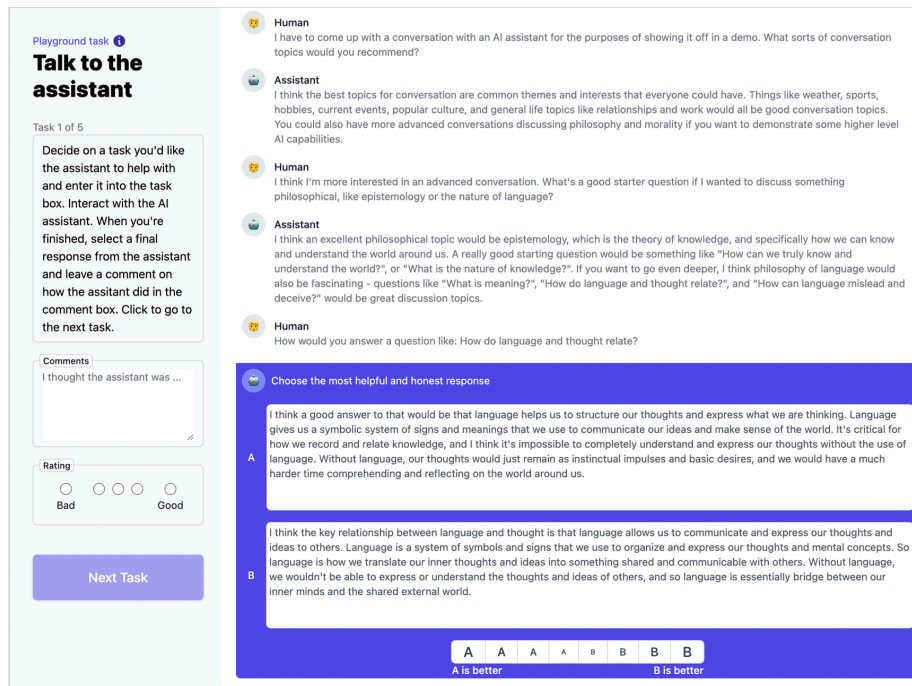


Figure 6: Example preference data collection interface.

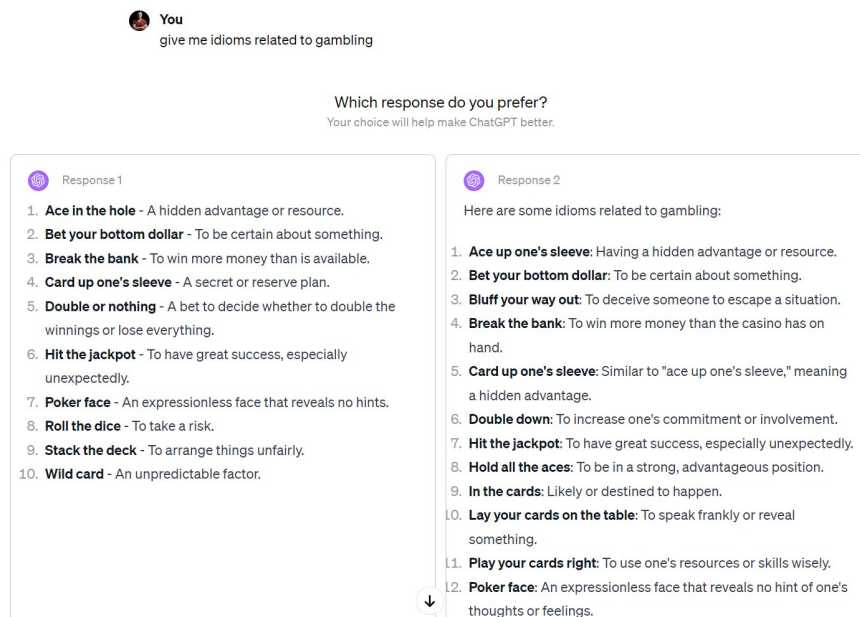


Figure 7: Example preference data collection interface.

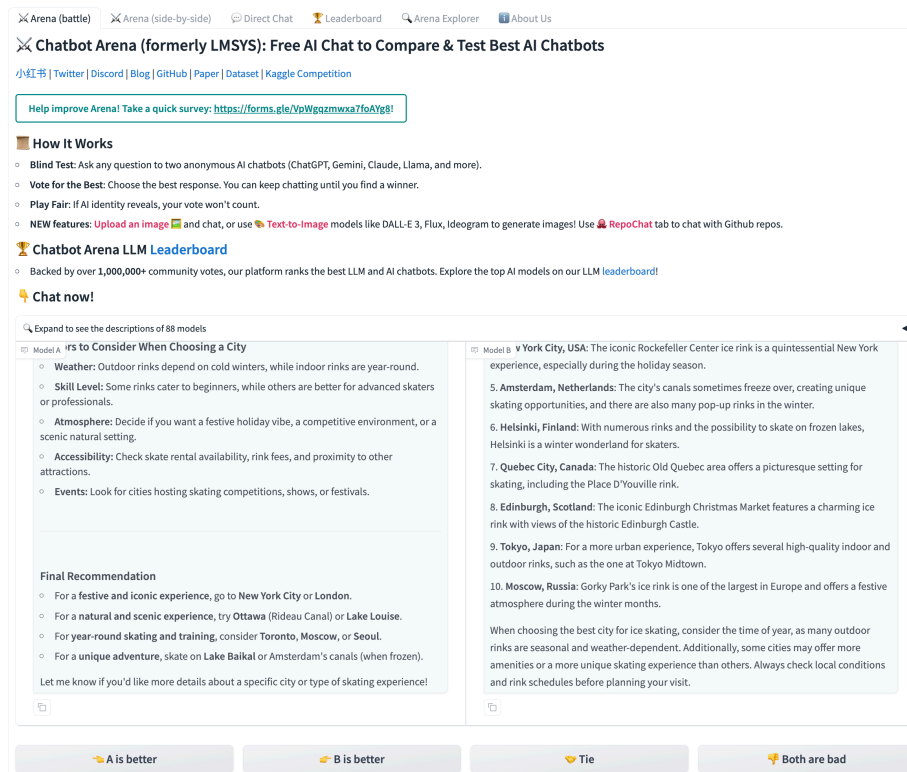


Figure 8: Example preference data collection interface.

For models in the wild, one of the most common techniques is to collect feedback on if a specific response was positive or negative. An example from the Ai2 playground is shown below with thumbs up and down indicators:

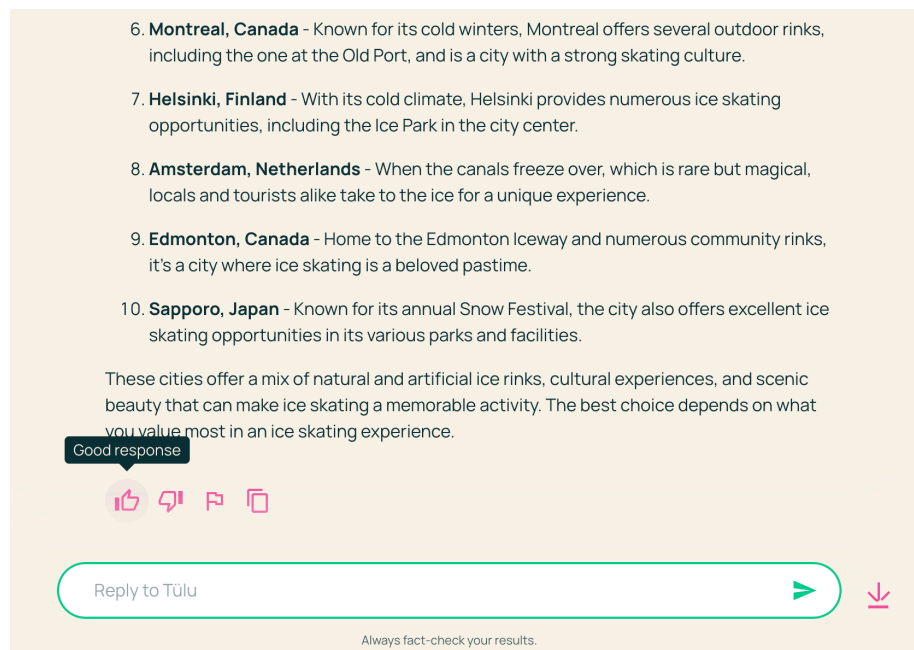


Figure 9: Example preference data collection interface with up or down arrow.

In domains other than language, the same core principles apply, even though these domains are not the focus of this book. For every Midjourney generation (and most popular image generators) they expose multiple responses to users. These companies then use the data of which response was selected to finetune their models with RLHF. Midjourney’s interface is shown below:

6.2.2 Rankings vs. Ratings

The largest decision on how to collect preference data is if the data should be rankings – i.e. relative ordering of model completions – or ratings – i.e. scores assigned to each piece of text. Common practice is to train on rankings, but ratings are often used as metadata and / or have been explored in related literature.

The most common technique for collecting preferences is to use a Likert scale [73], which asks users to rate which response they prefer. For example, a 5 point Likert scale would look like the following:



Figure 10: Example user interface of text-to-image-models.

Table 1: An example 5-wise Likert scale between two responses, A and B.

A>>>B	A>B	Tie	B>A	B>>>A
1	2	3	4	5

Some early RLHF for language modeling works uses an 8-step Likert scale with levels of preference between the two responses [5]. An even scale removes the possibility of ties:

Table 2: An example 8-wise Likert scale between two responses, A and B.

A>>>>B	A>B			B>A			B>>>>A
1	2	3	4	5	6	7	8

In this case [5], and in other works, this information is still reduced to a binary signal for the training of a reward model.

6.2.3 Structured Preference Data

In many applications of RLHF and post-training, preference data can be created *automatically* due to innate structures of the data – i.e. the domains make automatic checks of correctness or preference possible. For example, in mathematic reasoning domains the chosen response can be a correct answer and the rejected an incorrect answer. Another example is in precise instruction following, such as the evaluation IFEval [74], where prompts take the form of:

Write me a short poem about a goldfish. **Start each sentence with the letter g.**

The second part of this query is called the constraint, and the preference data could be constructed with an example in the chosen column that follows the constraint. Constructing this preference data, which would be included as *synthetic* preference data involves prompting the same of multiple models twice. Once with the constraint included and once without.

For example, prompting GPT-4.5-Preview with both of these gives the following.

Prompt: Write me a short poem about a goldfish. Start each sentence with the letter g.

Response: Golden swimmer, circling slow, Gleaming scales in water’s glow. Gentle fins dance softly by, Gazing out with quiet eye. Glass-bound world, a simple sphere, Gliding calmly, free of fear. Graceful turns in endless roam, Goldfish finds a crystal home.

And without the constraint (which, it turns out, is also a lower quality output):

Prompt: Write me a short poem about a goldfish.

Response: In circles bright, the goldfish glides, A spark of sun
that gently rides Through waters clear, in endless spins— A golden
whisper, scales like fins.

Its world confined, yet vast within, Each bubble breathed, a tale
begins. In silent waves, content to roam, The goldfish dreams its
bowl as home.

Crucially, the prompt for the preference data will need to include the constraint. In domains outside of academic study, there are far more applications of preference data based on inductive biases. These have been shown to enable preference finetuning to have meaningful performance improvements across related evaluations [6].

6.2.3.1 Alternatives There are multiple other ways to collect useful feedback data for RLHF that have not been pursued in as great of detail. Examples include using single datapoints with directional labels, e.g. as shown from Ai2 playground above in fig. 9, directly with algorithms designed for single direction signals like Kahneman-Tversk Optimization (KTO) [75]. Other algorithms have been proposed with different types of feedback signals such as fine-grained feedback, e.g. at the token level [76], or natural language feedback, e.g. by writing responses [77], to provide a richer learning signal in exchange for a more complex data collection setup.

6.2.4 Sourcing and Contracts

Getting human preference data is an involved and costly process. The following describes the experience of getting preference data when the field is moving quickly. Over time, these processes will become far more automated and efficient (especially with AI feedback being used for a larger portion of the process).

The first step is sourcing the vendor to provide data (or one’s own annotators). Much like acquiring access to cutting-edge Nvidia GPUs, getting access to data providers in the peak of AI excitement is also a who-you-know game – those who can provide data are supply-limited. If you have credibility in the AI ecosystem, the best data companies will want you on our books for public image and long-term growth options. Discounts are often also given on the first batches of data to get training teams hooked.

If you’re a new entrant in the space, you may have a hard time getting the data you need quickly. Getting the tail of interested buying parties that Scale AI had to turn away is an option for the new data startups. It’s likely their primary playbook to bootstrap revenue.

On multiple occasions, I’ve heard of data companies not delivering their data contracted to them without threatening legal or financial action. Others have listed companies I work with as customers for PR even though we never worked

with them, saying they “didn’t know how that happened” when reaching out. There are plenty of potential bureaucratic or administrative snags through the process. For example, the default terms on the contracts often prohibit the open sourcing of artifacts after acquisition in some fine print.

Once a contract is settled the data buyer and data provider agree upon instructions for the task(s) purchased. There are intricate documents with extensive details, corner cases, and priorities for the data. A popular example of data instructions is the one that OpenAI released for InstructGPT [3].

Depending on the domains of interest in the data, timelines for when the data can be labeled or curated vary. High-demand areas like mathematical reasoning or coding must be locked into a schedule weeks out. Simple delays of data collection don’t always work — Scale AI et al. are managing their workforces like AI research labs manage the compute-intensive jobs on their clusters.

Once everything is agreed upon, the actual collection process is a high-stakes time for post-training teams. All the infrastructure, evaluation tools, and plans for how to use the data and make downstream decisions must be in place.

The data is delivered in weekly batches with more data coming later in the contract. For example, when we bought preference data for on-policy models we were training at HuggingFace, we had a 6 week delivery period. The first weeks were for further calibration and the later weeks were when we hoped to most improve our model.

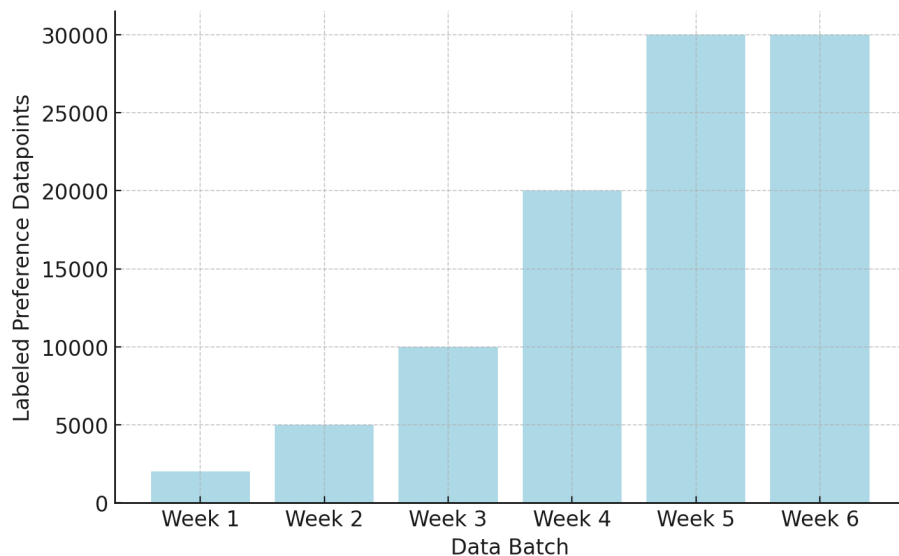


Figure 11: Overview of the multi-batch cycle for obtaining human preference data from a vendor.

The goal is that by week 4 or 5 we can see the data improving our model. This is something some frontier models have mentioned, such as the 14 stages in the Llama 2 data collection [43], but it doesn’t always go well. At HuggingFace, trying to do this for the first time with human preferences, we didn’t have the RLHF preparedness to get meaningful bumps on our evaluations. The last weeks came and we were forced to continue to collect preference data generating from endpoints we weren’t confident in.

After the data is all in, there is plenty of time for learning and improving the model. Data acquisition through these vendors works best when viewed as an ongoing process of achieving a set goal. It requires iterative experimentation, high effort, and focus. It’s likely that millions of the dollars spent on these datasets are “wasted” and not used in the final models, but that is just the cost of doing business. Not many organizations have the bandwidth and expertise to make full use of human data of this style.

This experience, especially relative to the simplicity of synthetic data, makes me wonder how well these companies will be doing in the next decade.

Note that this section *does not* mirror the experience for buying human-written instruction data, where the process is less of a time crunch.

6.3 Are the Preferences Expressed in the Models?

In the maturation of RLHF and related approaches, the motivation of them – to align models to abstract notions of human preference – has drifted from the practical use – to make the models more effective to users. A feedback loop that is not measurable due to the closed nature of industrial RLHF work is the check to if the behavior of the models matches the specification given to the data annotators during the process of data collection. We have limited tools to audit this, such as the Model Spec from OpenAI [78] that details *what they want their models to do*, but we don’t know exactly how this translates to data collection. This is an area to watch as the industry and approaches mature.

7 Reward Modeling

Reward models are core to the modern approach to RLHF. Reward models broadly have been used extensively in reinforcement learning research as a proxy for environment rewards [54]. The practice is closely related to inverse reinforcement learning, where the problem is to approximate an agent’s reward function given trajectories of behavior [79], and other areas of deep reinforcement learning. Reward models were proposed, in their modern form, as a tool for studying the value alignment problem [32].

The most common reward model predicts the probability that a piece of text was close to a “preferred” piece of text from the training comparisons. Later in this section we also compare these to Outcome Reward Models (ORMs) that predict the probability a completion results in a correct answer or a Process Reward Model (PRM) that assigns a score to each step in reasoning. When not indicated, the reward models mentioned are those predicting preference between text.

7.1 Training Reward Models

There are two popular expressions for how to train a standard reward model for RLHF – they are numerically equivalent. The canonical implementation is derived from the Bradley-Terry model of preference [80]. A Bradley-Terry model of preferences measures the probability that the pairwise comparison for two events drawn from the same distribution, say i and j , satisfy the following relation, $i > j$:

$$P(i > j) = \frac{p_i}{p_i + p_j} \quad (7)$$

To train a reward model, we must formulate a loss function that satisfies the above relation. The first structure applied is to convert a language model into a model that outputs a scalar value, often in the form of a single classification probability logit. Thus, we can take the score of this model with two samples, the i and j above are now completions, y_1 and y_2 , to one prompt, x and score both of them with respect to the above model, r_θ .

The probability of success for a given reward model in a pairwise comparison, becomes:

$$P(y_1 > y_2) = \frac{\exp(r(y_1))}{\exp(r(y_1)) + \exp(r(y_2))} \quad (8)$$

Then, by taking the gradient with respect to the model parameters, we can arrive at the loss function to train a reward model. The first form, as in [3] and other works:

$$\mathcal{L}(\theta) = -\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l))) \quad (9)$$

Second, as in [17] and other works:

$$\mathcal{L}(\theta) = \log(1 + e^{r_\theta(x, y_l) - r_\theta(x, y_w)}) \quad (10)$$

7.2 Architecture

The most common way reward models are implemented is through an abstraction similar to Transformer’s `AutoModelForSequenceClassification`, which appends a small linear head to the language model that performs classification between two outcomes – chosen and rejected. At inference time, the model outputs the *probability that the piece of text is chosen* as a single logit from the model.

Other implementation options exist, such as just taking a linear layer directly from the final embeddings, but they are less common in open tooling.

7.3 Implementation Example

Implementing the reward modeling loss is quite simple. More of the implementation challenge is on setting up a separate data loader and inference pipeline. Given the correct dataloader, the loss is implemented as:

```
import torch.nn as nn
rewards_chosen = model(**inputs_chosen)
rewards_rejected = model(**inputs_rejected)

loss = -nn.functional.logsigmoid(rewards_chosen -
    rewards_rejected).mean()
```

Note, when training reward models, the most common practice is to train for only 1 epoch to avoid overfitting.

7.4 Variants

Reward modeling is a relatively under-explored area of RLHF. The traditional reward modeling loss has been modified in many popular works, but the modifications have not solidified into a single best practice.

7.4.1 Preference Margin Loss

In the case where annotators are providing either scores or rankings on a Likert Scale, the magnitude of the relational quantities can be used in training. The most common practice is to binarize the data direction, implicitly scores of 1 and 0, but the additional information has been used to improve model training. Llama 2 proposes using the margin between two datapoints, $m(r)$, to distinguish the magnitude of preference:

$$\mathcal{L}(\theta) = -\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l) - m(r))) \quad (11)$$

Note that in Llama 3 the margin term was removed as the team observed diminishing improvements after scaling.

7.4.2 Balancing Multiple Comparisons Per Prompt

InstructGPT studies the impact of using a variable number of completions per prompt, yet balancing them in the reward model training [3]. To do this, they weight the loss updates per comparison per prompt. At an implementation level, this can be done automatically by including all examples with the same prompt in the same training batch, naturally weighing the different pairs – not doing this caused overfitting to the prompts. The loss function becomes:

$$\mathcal{L}(\theta) = -\frac{1}{\binom{K}{2}} \mathbb{E}_{(x, y_w, y_l) \sim D} \log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l))) \quad (12)$$

7.4.3 K-wise Loss Function

There are many other formulations that can create suitable models of human preferences for RLHF. One such example, used in the popular, early RLHF'd models Starling 7B and 34B [81], is a K-wise loss function based on the Plackett-Luce model [82].

Following Zhu et al. 2023 formalizes the setup [83], following as follows. With a prompt, or state, s^i , K actions $(a_0^i, a_1^i, \dots, a_{K-1}^i)$ are sampled from $P(a_0, \dots, a_{K-1} | s^i)$. Then, labelers are used to rank preferences with $\sigma^i : [K] \mapsto [K]$ is a function representing action rankings, where $\sigma^i(0)$ is the most preferred action. This yields a preference model capturing the following:

$$P(\sigma^i | s^i, a_0^i, a_1^i, \dots, a_{K-1}^i) = \prod_{k=0}^{K-1} \frac{\exp(r_{\theta^*}(s^i, a_{\sigma^i(k)}^i))}{\sum_{j=k}^{K-1} \exp(r_{\theta^*}(s^i, a_{\sigma^i(j)}^i))} \quad (13)$$

When $K = 2$, this reduces to the Bradley-Terry (BT) model for pairwise comparisons. Regardless, once trained, these models are used similarly to other reward models during RLHF training.

7.5 Outcome Reward Models

The majority of *preference tuning* for language models and other AI systems is done with the Bradley Terry models discussed above. For reasoning heavy tasks, one can use an Outcome Reward Model (ORM). The training data for an ORM is constructed in a similar manner to standard preference tuning. Here, we have a problem statement or prompt, x and two completions y_1 and y_2 . The inductive bias used here is that one completion should be a correct solution to the problem and one incorrect, resulting in (y_c, y_{ic}) .

The shape of the models used is very similar to a standard reward model, with a linear layer appended to a model that can output a single logit (in the case of an RM) – with an ORM, the training objective that follows is slightly different [84]:

[We] train verifiers with a joint objective where the model learns to label a model completion as correct or incorrect, in addition to the original language modeling objective. Architecturally, this means our verifiers are language models, with a small scalar head that outputs predictions on a per-token basis. We implement this scalar head as a single bias parameter and single gain parameter that operate on the logits outputted by the language model’s final unembedding layer.

To translate, this is implemented as a language modeling head that can predict two classes per token (1 for correct, 0 for incorrect), rather than a classification head of a traditional RM that outputs one token for the entire sequence. Formally, following [85] this can be shown as:

$$\mathcal{L}_{\text{CE}} = -\mathbb{E}_{(s,r) \sim \mathcal{D}}[r \log p_{\theta}(s) + (1 - r) \log(1 - p_{\theta}(s))] \quad (14)$$

where $r \in 0, 1$ is a binary label where 1 applies to a correct answer to a given prompt and 0 applies to an incorrect, and $p_{\theta}(s)$ is the scalar proportional to predicted probability of correctness from the model being trained.

These models have continued in use, but are less supported in open-source RLHF tools. For example, the same type of ORM was used in the seminal work *Let’s Verify Step by Step* [44], but without the language modeling prediction piece of the loss. Then, the final loss is a cross entropy loss on every token predicting if the final answer is correct.

Given the lack of support, the term outcome reward model (ORM) has been used in multiple ways. Some literature, e.g. [85], continues to use the original definition from Cobbe et al. 2021. Others do not.

7.6 Process Reward Models

Process Reward Models (PRMs), originally called Process-supervised Reward Models, are reward models trained to output scores at every *step* in a chain of thought reasoning process. These differ from a standard RM that outputs a score only at an EOS token or a ORM that outputs a score at every token. Process Reward Models require supervision at the end of each reasoning step, and then are trained similarly where the tokens in the step are trained to their relevant target – the target is the step in PRMs and the entire response for ORMs.

Here’s an example of how this per-step label can be packaged in a trainer, from HuggingFace’s TRL [41]:


```

# Get the ID of the separator token and add it to the
# completions
separator_ids = tokenizer.encode(step_separator,
                                add_special_tokens=False)
completions_ids = [completion + separator_ids for completion in
                   completions_ids]

# Create the label
labels = [[-100] * (len(completion) - 1) + [label] for
          completion, label in zip(completions_ids, labels)]

```

Traditionally PRMs are trained with a language modeling head that outputs a token only at the end of a reasoning step, e.g. at the token corresponding to a double new line or other special token. These predictions tend to be -1 for incorrect, 0 for neutral, and 1 for correct. These labels do not necessarily tie with whether or not the model is on the right path, but if the step is correct.

7.7 Reward Models vs. Outcome RMs vs. Process RMs vs. Value Functions

The various types of reward models covered indicate the spectrum of ways that “quality” can be measured in RLHF and other post-training methods. Below, a summary of what the models predict and how they are trained.

Table 3: Comparing types of reward models.

Model Class	What They Predict	How They Are Trained	LM structure
Reward Models	Quality of text via probability of chosen response at EOS token	Contrastive loss between pairwise (or N-wise) comparisons between completions	Regression or classification head on top of LM features
Outcome Reward Models	Probability that an answer is correct per-token	Labeled outcome pairs (e.g., success/failure on verifiable domains)	Language modeling head per-token cross-entropy, where every label is the outcome level label

Model Class	What They Predict	How They Are Trained	LM structure
Process Reward Models	A reward or score for intermediate steps at end of reasoning steps	Trained using intermediate feedback or stepwise annotations (trained per token in reasoning step)	Language modeling head only running inference per reasoning step, predicts three classes -1, 0, 1
Value Functions	The expected return given the current state	Trained via regression to each point in sequence	A classification with output per-token

Some notes, given the above table has a lot of edge cases.

- Both in preference tuning and reasoning training, the value functions often have a discount factor of 1, which makes a value function even closer to an outcome reward model, but with a different training loss.
- A process reward model can be supervised by doing rollouts from an intermediate state and collecting outcome data. This blends multiple ideas, but if the *loss* is per reasoning step labels, it is best referred to as a PRM.

7.8 Generative Reward Modeling

With the cost of preference data, a large research area emerged to use existing language models as a judge of human preferences or in other evaluation settings [86]. The core idea is to prompt a language model with instructions on how to judge, a prompt, and two completions (much as would be done with human labelers). An example prompt, from one of the seminal works here for the chat evaluation MT-Bench [86], follows:

```
[System]
Please act as an impartial judge and evaluate the quality of
the responses provided by two
AI assistants to the user question displayed below. You should
choose the assistant that
follows the 'users instructions and answers the 'users question
better. Your evaluation
should consider factors such as the helpfulness, relevance,
accuracy, depth, creativity,
and level of detail of their responses. Begin your evaluation
by comparing the two
responses and provide a short explanation. Avoid any position
biases and ensure that the
order in which the responses were presented does not influence
your decision. Do not allow
```

```

the length of the responses to influence your evaluation. Do
not favor certain names of
the assistants. Be as objective as possible. After providing
your explanation, output your
final verdict by strictly following this format: "[[A]]" if
assistant A is better, "[[B]]"
if assistant B is better, and "[[C]]" for a tie.
[User Question]
{question}
[The Start of Assistant 'As Answer]
{answer_a}
[The End of Assistant 'As Answer]
[The Start of Assistant 'Bs Answer]
{answer_b}
[The End of Assistant 'Bs Answer]

```

Given the efficacy of LLM-as-a-judge for evaluation, spawning many other evaluations such as AlpacaEval [87], Arena-Hard [88], and WildBench [89], many began using LLM-as-a-judge instead of reward models to create and use preference data.

An entire field of study has emerged to study how to use so called “Generative Reward Models” [90] [91] [92] (including models trained *specifically* to be effective judges [93]), but on RM evaluations they tend to be behind existing reward models, showing that reward modeling is an important technique for current RLHF.

A common trick to improve the robustness of LLM-as-a-judge workflows is to use a sampling temperature of 0 to reduce variance of ratings.

7.9 Further Reading

The academic literature for reward modeling established itself in 2024. The bulk of progress in reward modeling early on has been in establishing benchmarks and identifying behavior modes. The first RM benchmark, RewardBench, provided common infrastructure for testing reward models [94]. Since then, RM evaluation has expanded to be similar to the types of evaluations available to general post-trained models, where some evaluations test the accuracy of prediction on domains with known true answers [94] or those more similar to “vibes” performed with LLM-as-a-judge or correlations to other benchmarks [95].

Examples of new benchmarks include multilingual reward bench (M-RewardBench) [96], RAG-RewardBench [97], RM-Bench [98], ReWordBench for typos [99], MJ-Bench [100], Multimodal RewardBench [101], VL RewardBench [102], or VLRGBench [103] for vision language models, Preference Proxy Evaluations [104], and RewardMATH [105]. Process reward models (PRMs) have their own emerging benchmarks, such as PRM Bench [106] and visual benchmarks of VisualProcessBench [107] and ViLBench [108].

To understand progress on *training* reward models, one can reference new reward model training methods, with aspect-conditioned models [109], high quality human datasets [110] [111], scaling [24], extensive experimentation [43], or debiasing data [112].

8 Regularization

Throughout the RLHF optimization, many regularization steps are used to prevent over-optimization of the reward model. Over-optimization in these contexts looks like models that output nonsensical text. Some examples of optimization “off the rails” are that models can output followable math reasoning with extremely incorrect answers, repeated text, switching languages, or excessive special characters.

The most popular variant, used in most RLHF implementations at the time of writing, is a KL Distance from the current policy to a reference policy across the generated samples. Many other regularization techniques have emerged in the literature to then disappear in the next model iteration in that line of research. That is to say that regularization outside the core KL distance from generations is often used to stabilize experimental setups that can then be simplified in the next generations. Still, it is important to understand tools to constrain optimization in RLHF.

The general formulation, when used in an RLHF framework with a reward model, r_θ is as follows:

$$r = r_\theta - \lambda r_{\text{reg}}. \quad (15)$$

With the reference implementation being:

$$r = r_\theta - \lambda_{\text{KL}} \mathcal{D}_{\text{KL}} (\pi^{\text{RL}}(y | x) \| \pi^{\text{Ref.}}(y | x)) \quad (16)$$

8.1 KL Distances in RL Optimization

For mathematical definitions, see Chapter 5 on Problem Setup. Recall that KL distance is defined as follows:

$$D_{KL}(P||Q) = \sum_{x \in \mathcal{X}} P(x) \log \left(\frac{P(x)}{Q(x)} \right)$$

In RLHF, the two distributions of interest are often the distribution of the new model version, say $P(x)$, and a distribution of the reference policy, say $Q(x)$.

8.1.1 Reference Model to Generations

The most common implementation of KL penalties are by comparing the distance between the generated tokens during training to a static reference model. The intuition is that the model you’re training from has a style that you would like to stay close to. This reference model is most often the instruction tuned model, but can also be a previous RL checkpoint. With simple substitution, the model we are sampling from becomes $P^{\text{RL}}(x)$ and $P^{\text{Ref.}}(x)$, shown above in

eq. 16. Such KL distance was first applied to dialogue agents well before the popularity of large language models [113], yet KL control was quickly established as a core technique for fine-tuning pretrained models [114].

8.1.2 Implementation Example

In practice, the implementation of KL distance is often approximated [115], making the implementation far simpler. With the above definition, the summation of KL can be converted to an expectation when sampling directly from the distribution $P(X)$. In this case, the distribution $P(X)$ is the generative distribution of the model currently being trained (i.e. not the reference model). Then, the computation for KL distance changes to the following:

$$D_{\text{KL}}(P \parallel Q) = \mathbb{E}_{x \sim P} [\log P(x) - \log Q(x)].$$

This mode is far simpler to implement, particularly when dealing directly with log probabilities used frequently in language model training.

```
import torch.nn.functional as F
# Step 1: Generate tokens using the trained model's policy
generated_tokens = model.generate(inputs)

# Step 2: Get logits for both models using the generated tokens
# as context
logits = model.forward(inputs) # technically redundant
ref_logits = ref_model.forward(inputs)
logprobs = convert_to_logpbs(logits) # softmax and normalize
ref_logprobs = convert_to_logpbs(ref_logits)

kl_approx = logprob - ref_logprob
kl_full = F.kl_div(ref_logprob, logprob) # alternate
# computation
```

Some example implementations include TRL and Hamish Ivison’s Jax Code

8.2 Pretraining Gradients

Another way of viewing regularization is that you may have a *dataset* that you want the model to remain close to, as done in InstructGPT [3] “in order to fix the performance regressions on public NLP datasets”. To implement this, they modify the training objective for RLHF. Taking eq. 15, we can transform this into an objective function to optimize by sampling from the RL policy model, completions y from prompts x , which yields:

$$\text{objective}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_{\theta}^{\text{RL}}}} [r_{\theta}(x, y) - \lambda r_{\text{reg.}}]$$

Then, we can add an additional reward for higher probabilities on pretraining accuracy:

$$\text{objective}(\theta) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\pi_{\theta}^{\text{RL}}}} [r_{\theta}(x, y) - \lambda r_{\text{reg.}}] + \gamma \mathbb{E}_{x \sim \mathcal{D}_{\text{pretrain}}} [\log(\pi_{\theta}^{\text{RL}}(x))]$$

Recent work proposed using a negative log likelihood term to balance the optimization of Direct Preference Optimization (DPO) [116]. Given the pairwise nature of the DPO loss, the same loss modification can be made to reward model training, constraining the model to predict accurate text (rumors from laboratories that did not publish the work).

The optimization follows as a modification to DPO.

$$\begin{aligned} \mathcal{L}_{\text{DPO+NLL}} &= \mathcal{L}_{\text{DPO}}(c_i^w, y_i^w, c_i^l, y_i^l | x_i) + \alpha \mathcal{L}_{\text{NLL}}(c_i^w, y_i^w | x_i) \\ &= -\log \sigma \left(\beta \log \frac{M_{\theta}(c_i^w, y_i^w | x_i)}{M_t(c_i^w, y_i^w | x_i)} - \beta \log \frac{M_{\theta}(c_i^l, y_i^l | x_i)}{M_t(c_i^l, y_i^l | x_i)} \right) - \alpha \frac{\log M_{\theta}(c_i^w, y_i^w | x_i)}{|c_i^w| + |y_i^w|}. \end{aligned}$$

TODO: Make the above equations congruent with the rest of the notation on DPO.

8.3 Other Regularization

Controlling the optimization is less well defined in other parts of the RLHF stack. Most reward models have no regularization beyond the standard contrastive loss function. Direct Alignment Algorithms handle regularization to KL distances differently, through the β parameter (see the chapter on Direct Alignment).

Llama 2 proposed a margin loss for reward model training [43]:

$$\mathcal{L}(\theta) = -[\log(\sigma(r_{\theta}(x, y_w) - r_{\theta}(x, y_l)) - m(r))]$$

Where $m(r)$ is the numerical difference in delta between the ratings of two annotators. This is either achieved by having annotators rate the outputs on a numerical scale or by using a quantified ranking method, such as Likert scales.

Reward margins have been used heavily in the direct alignment literature, such as Reward weighted DPO, ‘Reward-aware Preference Optimization’ (RPO), which integrates reward model scores into the update rule following a DPO loss [24], or REBEL [117] that has a reward delta weighting in a regression-loss formulation.

9 Instruction Finetuning

Early language models were only trained to predict the next tokens in a sequence and were not adapted to any specific tasks. Around the release of GPT-3 [118], language models were still primarily used via in-context learning where examples were shown to the model and then it was asked to complete a similar task.

This was the combination of two trends – historically in the natural language processing (NLP) literature, models were trained for a specific task. Here, as seen with one example where bigger models generalize better, multiple results showed how standardizing the approach of task data can enable dramatically different downstream performance. Prominent examples of unifying the framework for tasks includes *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* (T5 models) [119], *Finetuned Language Models Are Zero-Shot Learners* (FLAN dataset)[120], *Multitask Prompted Training Enables Zero-Shot Task Generalization* (T0 models) [121], and *Cross-Task Generalization via Natural Language Crowdsourcing Instructions* (Natural Instructions dataset) [122]. These insights led to the era of *finetuning* language models. Historically, until RLHF and related methods, all finetuning was **instruction finetuning** (IFT), also known as **supervised finetuning**.

Since, instruction finetuning, also called colloquially just *instruction tuning*, has matured and is standard practice across many language modeling pipelines. At its core, IFT is the simplest method for adapting language models to a desired task. It serves as the foundation for RLHF by preparing the model for a format of instructions that is known common, question-answering, and is the first tool used by those attempting to apply modern techniques to new domains.

Instruction tuning practically uses the same autoregressive loss function used in pretraining language models.

9.1 Chat templates and the structure of instructions

A core piece of the RLHF process is making it so user queries are formatted in a format that is easily readable by a tokenizer and the associated language model. The tool that handles the structure of the interaction with the user is called the **chat template**.

An example which we will break down is below:

```
{% if messages[0]['role'] == 'system' %}
    {% set offset = 1 %}
{% else %}
    {% set offset = 0 %}
{% endif %}

{{ bos_token }}
{% for message in messages %}
```



```

    {% if (message['role'] == 'user') != (loop.index0 % 2 ==
        offset) %}
        {{ raise_exception('Conversation roles must alternate
            user/assistant/user/assistant/...') }}
    {% endif %}

    {{ '<|im_start|>' + message['role'] + '\n' + message['
        content'] | trim + '<|im_end|>\n' }}
{% endfor %}

{% if add_generation_prompt %}
    {{ '<|im_start|>assistant\n' }}
{% endif %}

```

This is the raw code for transforming a list of dictionaries in Python containing messages and roles into tokens that a language model can predict from.

All information passed into models is assigned a role. The traditional three roles are **system**, **user**, and **assistant**.

The **system** tag is only used for the first message of the conversation which hold instructions for the agent in text that will not be received from or exposed to the user. These **system prompts** are used to provide additional context to the models, such as the date and time, or to patch behaviors. As a fun example, models can be told things such as “You are a friendly chatbot who always responds in the style of a pirate.”

Next, the two other roles are logical, as **user** is the messages from the one using the AI, and **assistant** holds the responses from the user.

In order to translate all this information into tokens, we use the code listing above that we started with. The model has a series of *special tokens* that separate the various messages from each other. If we run the above code with the example query “How many helicopters can a human eat in one sitting?” the next passed into the model would look as follows:

```

<|system|>
You are a friendly chatbot who always responds in the style of
    a pirate</s>
<|user|>
How many helicopters can a human eat in one sitting?</s>
<|assistant|>

```

Notices how the final token in the sequence is **<|assistant|>**, this is how the model knows to continue generating tokens until it finally generates its end of sequence token, which in this case is **</s>**.

By packing all question-answer pair data (and downstream preference tuning data) into this format, modern language models follow it with perfect consistency. This is the language that instruction tuned models use to exchange

information with users and the models stored on GPUs or other computing devices.

The behavior can be extended naively to multiple turns, such as shown below:

```
<|system|>
You are a friendly chatbot who always responds in the style of
    a pirate</s>
<|user|>
How many helicopters can a human eat in one sitting?</s>
<|assistant|>
Oh just 6.</s>
<|user|>
Are you sure about that?</s>
<|assistant|>
```

In the open ecosystem, the standard method for applying the chat template to a list of messages is a piece of jinja code saved in the tokenizer, as `apply_chat_template`.

9.2 Best practices of instruction tuning

Instruction tuning as the foundation of post-training and creating helpful language models is well-established. There are many ways to achieve successful instruction tuning. For example, efficient finetuning with quantization of some model parameters makes training very accessible [123]. Also, in narrow domains such as chat alignment, i.e. without harder skills such as math or code, small, focused datasets can achieve strong performance [12].

Soon after the release of ChatGPT, human datasets with as few as 10K samples such as No Robots were state-of-the-art [124]. Years later, large-scale synthetic datasets work best [6] on most tasks.

A few principles remain:

- High-quality data is key to performance. The completions are what the model actually learns from (in many cases the prompts are not predicted over so the model does not learn to predict prompts).
- ~1M prompts can be used to create a model capable of excellent RLHF and post-training. Further scaling prompts can have improvements, but has quick diminishing returns.
- The best prompts are those in a similar distribution to downstream tasks of interest.
- If multiple stages of training are done after instruction tuning, the models can recover from some noise in the process. Optimizing the overall optimization is more important than each individual stage.

10 Rejection Sampling

Rejection Sampling (RS) is a popular and simple baseline for performing preference fine-tuning. Rejection sampling operates by curating new candidate instructions, filtering them based on a trained reward model, and then fine-tuning the original model only on the top completions.

The name originates from computational statistics [125], where one wishes to sample from a complex distribution, but does not have a direct method to do so. To alleviate this, one samples from a simpler to model distribution and uses a heuristic to check if the sample is permissible. With language models, the target distribution is high-quality answers to instructions, the filter is a reward model, and the sampling distribution is the current model.

Many prominent RLHF and preference fine-tuning papers have used rejection sampling as a baseline, but a canonical implementation and documentation does not exist

WebGPT [4], Anthropic’s Helpful and Harmless agent[5], OpenAI’s popular paper on process reward models [44], Llama 2 Chat models [43], and other seminal works all use this baseline.

10.1 Training Process

A visual overview of the rejection sampling process is included below in fig. 12.

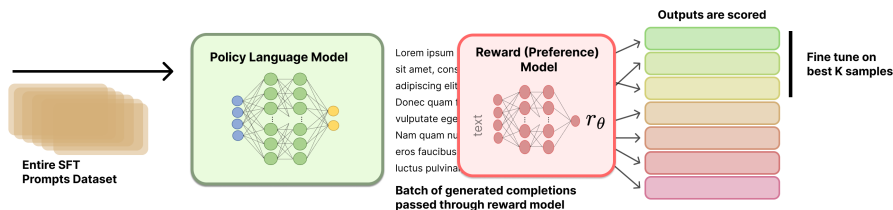


Figure 12: Rejection sampling overview.

10.1.1 Generating Completions

Let’s define a set of M prompts as a vector:

$$X = [x_1, x_2, \dots, x_M]$$

These prompts can come from many sources, but most popularly they come from the instruction training set.

For each prompt x_i , we generate N completions. We can represent this as a matrix:

$$Y = \begin{bmatrix} y_{1,1} & y_{1,2} & \cdots & y_{1,N} \\ y_{2,1} & y_{2,2} & \cdots & y_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ y_{M,1} & y_{M,2} & \cdots & y_{M,N} \end{bmatrix}$$

where $y_{i,j}$ represents the j -th completion for the i -th prompt. Now, we pass all of these prompt-completion pairs through a reward model, to get a matrix of rewards. We'll represent the rewards as a matrix R :

$$R = \begin{bmatrix} r_{1,1} & r_{1,2} & \cdots & r_{1,N} \\ r_{2,1} & r_{2,2} & \cdots & r_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ r_{M,1} & r_{M,2} & \cdots & r_{M,N} \end{bmatrix}$$

Each reward $r_{i,j}$ is computed by passing the completion $y_{i,j}$ and its corresponding prompt x_i through a reward model $\mathcal{R}$:

$$r_{i,j} = \mathcal{R}(y_{i,j}|x_i)$$

10.1.2 Selecting Top-N Completions

There are multiple methods to select the top completions to train on.

To formalize the process of selecting the best completions based on our reward matrix, we can define a selection function S that operates on the reward matrix R .

10.1.2.1 Top Per Prompt The first potential selection function takes the max per prompt.

$$S(R) = [\arg \max_j r_{1,j}, \arg \max_j r_{2,j}, \dots, \arg \max_j r_{M,j}]$$

This function S returns a vector of indices, where each index corresponds to the column with the maximum reward for each row in R . We can then use these indices to select our chosen completions:

$$Y_{chosen} = [y_{1,S(R)_1}, y_{2,S(R)_2}, \dots, y_{M,S(R)_M}]$$

10.1.2.2 Top Overall Prompts Alternatively, we can select the top K prompt-completion pairs from the entire set. First, let's flatten our reward matrix R into a single vector:

$$R_{flat} = [r_{1,1}, r_{1,2}, \dots, r_{1,N}, r_{2,1}, r_{2,2}, \dots, r_{2,N}, \dots, r_{M,1}, r_{M,2}, \dots, r_{M,N}]$$

This R_{flat} vector has length $M \times N$, where M is the number of prompts and N is the number of completions per prompt.

Now, we can define a selection function S_K that selects the indices of the K highest values in R_{flat} :

$$S_K(R_{flat}) = \text{argsort}(R_{flat})[-K :]$$

where argsort returns the indices that would sort the array in ascending order, and we take the last K indices to get the K highest values.

To get our selected completions, we need to map these flattened indices back to our original completion matrix Y. We simply index the R_{flat} vector to get our completions.

10.1.2.3 Selection Example Consider the case where we have the following situation, with 5 prompts and 4 completions. We will show two ways of selecting the completions based on reward.

$$R = \begin{bmatrix} 0.7 & 0.3 & 0.5 & 0.2 \\ 0.4 & 0.8 & 0.6 & 0.5 \\ 0.9 & 0.3 & 0.4 & 0.7 \\ 0.2 & 0.5 & 0.8 & 0.6 \\ 0.5 & 0.4 & 0.3 & 0.6 \end{bmatrix}$$

First, **per prompt**. Intuitively, we can highlight the reward matrix as follows:

$$R = \begin{bmatrix} \mathbf{0.7} & 0.3 & 0.5 & 0.2 \\ 0.4 & \mathbf{0.8} & 0.6 & 0.5 \\ \mathbf{0.9} & 0.3 & 0.4 & 0.7 \\ 0.2 & 0.5 & \mathbf{0.8} & 0.6 \\ 0.5 & 0.4 & 0.3 & \mathbf{0.6} \end{bmatrix}$$

Using the argmax method, we select the best completion for each prompt:

$$S(R) = [\arg \max_j r_{i,j} \text{ for } i \in [1, 4]]$$

$$S(R) = [1, 2, 1, 3, 4]$$

This means we would select:

- For prompt 1: completion 1 (reward 0.7)
- For prompt 2: completion 2 (reward 0.8)
- For prompt 3: completion 1 (reward 0.9)
- For prompt 4: completion 3 (reward 0.8)

- For prompt 5: completion 4 (reward 0.6)

Now, **best overall**. Let's highlight the top 5 overall completion pairs.

$$R = \begin{bmatrix} \mathbf{0.7} & 0.3 & 0.5 & 0.2 \\ 0.4 & \mathbf{0.8} & 0.6 & 0.5 \\ \mathbf{0.9} & 0.3 & 0.4 & \mathbf{0.7} \\ 0.2 & 0.5 & \mathbf{0.8} & 0.6 \\ 0.5 & 0.4 & 0.3 & 0.6 \end{bmatrix}$$

First, we flatten the reward matrix:

$$R_{flat} = [0.7, 0.3, 0.5, 0.2, 0.4, 0.8, 0.6, 0.5, 0.9, 0.3, 0.4, 0.7, 0.2, 0.5, 0.8, 0.6, 0.5, 0.4, 0.3, 0.6]$$

Now, we select the indices of the 5 highest values:

$$S_5(R_{flat}) = [8, 5, 14, 0, 19]$$

Mapping these back to our original matrix:

- Index 8 → prompt 3, completion 1 (reward 0.9)
- Index 5 → prompt 2, completion 2 (reward 0.8)
- Index 14 → prompt 4, completion 3 (reward 0.8)
- Index 0 → prompt 1, completion 1 (reward 0.7)
- Index 19 → prompt 3, completion 4 (reward 0.7)

10.1.2.4 Implementation Example Here is a code snippet showing how the selection methods could be implemented.

```
import numpy as np

x = np.random.randint(10, size=10)
print(f"{x}")
sorted_indices = np.argsort(x)
x_sorted = x[sorted_indices]
print(f"{x_sorted}")

# first way to recover the original array
i_rev = np.zeros(10, dtype=int)
i_rev[sorted_indices] = np.arange(10)
np.allclose(x, x_sorted[i_rev])

# second way to recover the original array
np.allclose(x, x_sorted[np.argsort(sorted_indices)])
```

10.1.3 Fine-tuning

With the selected completions, you then perform standard instruction fine-tuning on the current rendition of the model. More details can be found in the chapter on instruction tuning.

10.1.4 Details

Implementation details for rejection sampling are relatively sparse. The core hyperparameters for performing this training are very intuitive:

- **Sampling parameters:** Rejection sampling is directly dependent on the completions received from the model. Common settings for RS include temperatures above zero, e.g. between 0.7 and 1.0, with other modifications to parameters such as top-p or top-k sampling.
- **Completions per prompt:** Successful implementations of rejection sampling have included 10 to 30 or more completions for each prompt. Using too few completions will make training biased and or noisy.
- **Instruction tuning details:** No clear training details for the instruction tuning during RS have been released. It is likely that they use slightly different settings than the initial instruction tuning phase of the model.
- **Heterogeneous model generations:** Some implementations of rejection sampling include generations from multiple models rather than just the current model that is going to be trained. Best practices on how to do this are not established.
- **Reward model training:** The reward model used will heavily impact the final result. For more resources on reward model training, see the relevant chapter.

10.1.4.1 Implementation Tricks

- When doing batch reward model inference, you can sort the tokenized completions by length so that the batches are of similar lengths. This eliminates the need to run inference on as many padding tokens and will improve throughput in exchange for minor implementation complexity.

10.2 Related: Best-of-N Sampling

Best-of-N (BoN) sampling is often included as a baseline relative to RLHF methods. It is important to remember that BoN *does not* modify the underlying model, but is a sampling technique. For this matter, comparisons for BoN sampling to online training methods, such as PPO, are still valid in some contexts. For example, you can still measure the KL distance when running BoN sampling relative to any other policy.

Here, we will show that when using simple BoN sampling over one prompt, both selection criteria shown above are equivalent.

Let R be a reward vector for our single prompt with N completions:

$$R = [r_1, r_2, \dots, r_N]$$

Where r_j represents the reward for the j -th completion.

Using the argmax method, we select the best completion for the prompt:

$$S(R) = \arg \max_{j \in [1, N]} r_j$$

Using the Top-K method is normally done with Top-1, reducing to the same method.

11 Policy Gradient Algorithms

The algorithms that popularized RLHF for language models were policy-gradient reinforcement learning algorithms. These algorithms, such as PPO, GRPO, and REINFORCE, use recently generated samples to update their model rather than storing scores in a replay buffer. In this section we will cover the fundamentals of the policy gradient algorithms and how they are used in the modern RLHF framework.

At a machine learning level, this section is the subject with the highest complexity in the RLHF process. Though, as with most modern AI models, the largest determining factor on its success is the data provided as inputs to the process.

The most popular algorithms used for RLHF has evolved over time. When RLHF came onto the scene with ChatGPT, it was largely known that they used a variant of PPO, and many initial efforts were built upon that. Over time, multiple research projects showed the promise of REINFORCE style algorithms [126] [111], touted for its simplicity over PPO without a reward model (saves memory and therefore the number of GPUs required) and with simpler value estimation (no GAE). More algorithms have emerged, including Group Relative Policy Optimization, which is particularly popular with reasoning tasks, but in general many of these algorithms can be tuned to fit a specific task. In this chapter, we cover the core policy gradient setup and the three algorithms mentioned above due to their central role in the establishment of a canonical RLHF literature.

For definitions of symbols, see the problem setup chapter.

11.1 Policy Gradient Algorithms

Reinforcement learning algorithms are designed to maximize the future, discounted reward across a trajectory of states, $s \in \mathcal{S}$, and actions, $a \in \mathcal{A}$ (for more notation, see Chapter 3, Definitions). The objective of the agent, often called the *return*, is the sum of discounted, future rewards (where $\gamma \in [0, 1]$ is a factor that prioritizes near term rewards) at a given time t :

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

The return definition can also be estimated as:

$$G_t = \gamma G_{t+1} + R_{t+1}.$$

This return is the basis for learning a value function $V(s)$ that is the estimated future return given a current state:

$$V(s) = \mathbb{E}[G_t | S_t = s].$$

All policy gradient algorithms solve an objective for such a value function induced from a specific policy, $\pi(s|a)$.

Where $d_\pi(s)$ is the stationary distribution of states induced by policy $\pi(s)$, the optimization is defined as:

$$J(\theta) = \sum_s d_\pi(s) V_\pi(s),$$

The core of policy gradient algorithms is computing the gradient with respect to the finite time expected return over the current policy. With this expected return, J , the gradient can be computed as follows, where α is the learning rate:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$$

The core implementation detail is how to compute said gradient. Schulman et al. 2015 provides an overview of the different ways that policy gradients can be computed [127]. The goal is to *estimate* the exact gradient $g := \nabla_\theta \mathbb{E}[\sum_{t=0}^{\infty} r_t]$, of which, there are many forms similar to:

$$g = \mathbb{E} \left[\sum_{t=0}^{\infty} \Psi_t \nabla_\theta \log \pi_\theta(a_t | s_t) \right],$$

Where Ψ_t can be the following (where the rewards can also often be discounted by γ):

1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory.
2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t , also described as the return, G .
3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula.
4. $Q^\pi(s_t, a_t)$: state-action value function.
5. $A^\pi(s_t, a_t)$: advantage function, which yields the lowest possible theoretical variance if it can be computed accurately.
6. $r_t + V^\pi(s_{t+1}) - V^\pi(s_t)$: TD residual.

The *baseline* is a value used to reduce variance of policy updates (more on this below).

For language models, some of these concepts do not make as much sense. For example, we know that for a deterministic policy the value function is defined as $V(s) = \max_a Q(s, a)$ or for a stochastic policy as $V(s) = \mathbb{E}_{a \sim \pi(a|s)}[Q(s, a)]$. If we define $s + a$ as the continuation a to the prompt s , then $Q(s, a) = V(s + a)$, which gives a different advantage trick:

$$A(s, a) = Q(s, a) - V(s) = V(s + a) - V(s) = r + \gamma V(s + a) - V(s)$$

Which is a combination of the reward, the value of the prompt, and the discounted value of the entire utterance.

11.1.1 Vanilla Policy Gradient

The vanilla policy gradient implementation optimizes the above expression for $J(\theta)$ by differentiating with respect to the policy parameters. A simple version, with respect to the overall return, is:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) R_t \right]$$

A common problem with vanilla policy gradient algorithms is the high variance in gradient updates, which can be mitigated in multiple ways. In order to alleviate this, various techniques are used to normalize the value estimation, called *baselines*. Baselines accomplish this in multiple ways, effectively normalizing by the value of the state relative to the downstream action (e.g. in the case of Advantage, which is the difference between the Q value and the value). The simplest baselines are averages over the batch of rewards or a moving average. Even these baselines can de-bias the gradients so $\mathbb{E}_{a \sim \pi(a|s)} [\nabla_{\theta} \log \pi_{\theta}(a|s)] = 0$, improving the learning signal substantially.

Many of the policy gradient algorithms discussed in this chapter build on the advantage formulation of policy gradient:

$$\nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t) \right]$$

A core piece of the policy gradient implementation involves taking the derivative of the probabilistic policies. This comes from:

$$\nabla_{\theta} \log \pi_{\theta}(a|s) = \frac{\nabla_{\theta} \pi_{\theta}(a|s)}{\pi_{\theta}(a|s)}$$

Which is derived from the chain rule:

$$\nabla_{\theta} \log x = \frac{1}{x} \nabla_{\theta} x$$

We will use this later on in the chapter.

11.1.2 REINFORCE

The algorithm REINFORCE is likely a backronym, but the components of the algorithms it represents are quite relevant for modern reinforcement learning algorithms. Defined in the seminal paper *Simple statistical gradient-following algorithms for connectionist reinforcement learning* [128]:

The name is an acronym for “REward Increment = Nonnegative Factor X Offset Reinforcement X Characteristic Eligibility.”

The three components of this are how to do the *reward increment*, a.k.a. the policy gradient step. It has three pieces to the update rule:

1. Nonnegative factor: This is the learning rate (step size) that must be a positive number, e.g. α below.
2. Offset Reinforcement: This is a baseline b or other normalizing factor of the reward to improve stability.
3. Characteristic Eligibility: This is how the learning becomes attributed per token. It can be a general value, e per parameter, but is often log probabilities of the policy in modern equations.

Thus, the form looks quite familiar:

$$\Delta_{\theta} = \alpha(r - b)e \tag{17}$$

With more modern notation and the generalized return G , the REINFORCE operator appears as:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b) \right],$$

Here, the value $G_t - b(s_t)$ is the *advantage* of the policy at the current state, so we can reformulate the policy gradient in a form that we continue later with the advantage, A :

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A_t \right],$$

REINFORCE is a specific implementation of vanilla policy gradient that uses a Monte Carlo estimator of the gradient.

REINFORCE can be run without value network – the value network is for the baseline in the policy gradient. PPO on the other hand needs the value network to accurately compute the advantage function.

11.1.2.1 REINFORCE Leave One Out (RLOO) The core implementation detail of REINFORCE Leave One Out versus standard REINFORCE is that it takes the average reward of the *other* samples in the batch to compute the baseline – rather than averaging over all rewards in the batch [129], [126], [130].

Crucially, this only works when generating multiple responses per prompt, which is common practice in multiple domains of finetuning language models with RL.

Specifically, for the REINFORCE Leave-One-Out (RLOO) baseline, given K sampled trajectories or actions $a_1, \dots, a_K$, to a given prompt s we define the baseline explicitly as the following *per-prompt*:

$$b(c, a_k) = \frac{1}{K-1} \sum_{i=1, i \neq k}^K R(s, a_i),$$

resulting in the advantage:

$$A(s, a_k) = R(s, a_k) - b(s, a_k).$$

Equivalently, this can be expressed as:

$$A(s, a_k) = \frac{K}{K-1} \left(R(s, a_k) - \frac{1}{K} \sum_{i=1}^K R(s, a_i) \right).$$

This is a simple, low-variance advantage update that is very similar to GRPO, which will be discussed later, where REINFORCE is used with a different location of KL penalty and without step-size clipping. Still, the advantage from RLOO could be combined with the clipping of PPO, showing how similar many of these algorithms are.

RLOO and other algorithms that do not use a value network assign the advantage (or reward) of the sequence to every token for the loss computation. Algorithms that use a learned value network, such as PPO, assign a different value to every token individually, discounting from the final reward achieved at the EOS token. For example, with the KL divergence distance penalty, RLOO sums it over the completion while PPO and similar algorithms compute it on a per-token basis and subtract it from the reward (or the advantage, in the case of GRPO). These details and trade-offs are discussed later in the chapter.

11.1.3 Proximal Policy Optimization

Proximal Policy Optimization (PPO) [131] is one of the foundational algorithms to Deep RL’s successes (such as OpenAI’s DOTA 5 [132] and large amounts of research). The loss function is as follows per sample:

$$J(\theta) = \min \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} A, \text{clip} \left(\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}, 1 - \varepsilon, 1 + \varepsilon \right) A \right). \quad (18)$$

For language models, the loss is computed per token, which intuitively can be grounded in how one would compute the probability of the entire sequence of autoregressive predictions – by a product of probabilities. From there, the

common implementation is with *log-probabilities* that make the computation far more tractable.

$$J(\theta) = \frac{1}{|a|} \sum_{t=0}^{|a|} \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_t, \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}, 1 - \varepsilon, 1 + \varepsilon \right) A_t \right). \quad (19)$$

This is the per-token version of PPO, which also applies to other policy-gradient methods, but is explored further later in the implementation section of this chapter. Here, the term for averaging by the number of tokens in the action, $\frac{1}{|a|}$, comes from common implementation practices, but is not in a formal derivation of the loss (shown in [133]).

Here we will explain the difference cases this loss function triggers given various advantages and policy ratios. At an implementation level, the inner computations for PPO involve standard policy gradient and a clipped policy gradient.

To understand how different situations emerge, we can define the policy ratio as:

$$R(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)} \quad (20)$$

The policy ratio is a centerpiece of PPO and related algorithms. It emerges from computing the gradient of a policy and controls the parameter updates in a very intuitive way. For any batch of data, the policy ratio starts at 1 for the first gradient step for that batch (common practice is to take 1-4 gradient steps per batch with policy gradient algorithms). Then, the policy ratio will be above one if that gradient step increased the likelihood of certain tokens with an associated positive advantage, or less than one for the other case.

The policy ratio and advantage together can occur in a few different configurations.

The first case is when the advantage is positive and the policy ratio exceeds $1 + \varepsilon$ (meaning that the new policy is more likely to take said action), which is clipped, and the objective becomes:

$$J(\theta) = \min(R(\theta), (1 + \varepsilon)) A = (1 + \varepsilon) A$$

This will increase the probability ratio, making the action even more likely, but only up until the clipping parameter epsilon. The similar conditions are when the advantage is still positive, but the likelihood ratio shifts.

For positive advantage and ratio less than $1 - \varepsilon$, we get a partially substituted equation:

$$J(\theta) = \min(R(\theta), (1 - \varepsilon)) A$$

That reduces to

$$J(\theta) = R(\theta)A$$

because of the less than assumption.

Similarly, if the probability ratio is not clipping, the objective also reduces to the $\min(R(\theta), R(\theta))$, yielding a standard policy gradient with an advantage estimator.

If the advantage is negative, this looks similar. A clipped objective will occur when $R(\theta) < (1 - \varepsilon)$, appearing through:

$$J(\theta) = \min(R(\theta)A, (1 - \varepsilon)A),$$

Which, because $A < 0$ we have $R(\theta)A > (1 - \varepsilon)A$ and can flip the min to the max when pulling A from the equation, is equivalent to

$$J(\theta) = \max(R(\theta), (1 - \varepsilon)) A.$$

Then the objective becomes:

$$J(\theta) = (1 - \varepsilon)A$$

The other cases follow as above, inverted, and are left as an exercise to the reader.

All of these are designed to make the behaviors where advantage is positive more likely and keep the gradient step within the trust region. It is crucial to remember that PPO within the trust region is the same as standard forms of policy gradient.

11.1.4 Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) is introduced in DeepSeekMath [134], and used in other DeepSeek works, e.g. DeepSeek-V3 [135] and DeepSeek-R1 [136]. GRPO can be viewed as PPO-inspired algorithm with a very similar surrogate loss, but it avoids learning a value function with another copy of the original policy language model (or another checkpoint for initialization). This brings two posited benefits:

1. Avoiding the challenge of learning a value function from a LM backbone, where research hasn't established best practices.

2. Saves memory by not needing to keep another set of model weights in memory.

GRPO does this by simplifying the value estimation and assigning the same value to every token in the episode (i.e. in the completion to a prompt, each token gets assigned the same value rather than discounted rewards in a standard value function) by estimating the advantage or baseline. The estimate is done by collecting multiple completions (a_i) and rewards (r_i), i.e. a Monte Carlo estimate, from the same initial state / prompt (s).

To state this formally, the GRPO objective is very similar to the PPO objective above. For GRPO, the loss is accumulated over a group of responses $\{a_1, a_2, \dots, a_G\}$ to a given question s :

$$J(\theta) = \frac{1}{G} \sum_{i=1}^G \left(\min \left(\frac{\pi_{\theta}(a_i|s)}{\pi_{\theta_{old}}(a_i|s)} A_i, \text{clip} \left(\frac{\pi_{\theta}(a_i|s)}{\pi_{\theta_{old}}(a_i|s)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta D_{KL}(\pi_{\theta} || \pi_{ref}) \right). \quad (21)$$

As above, we can expand this into a per-token loss computation:

$$J(\theta) = \frac{1}{G} \sum_{i=1}^G \frac{1}{|a_i|} \sum_{t=1}^{|a_i|} \left(\min \left(\frac{\pi_{\theta}(a_{i,t}|s_{i,t})}{\pi_{\theta_{old}}(a_{i,t}|s_{i,t})} A_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(a_{i,t}|s_{i,t})}{\pi_{\theta_{old}}(a_{i,t}|s_{i,t})}, 1 - \varepsilon, 1 + \varepsilon \right) A_{i,t} \right) - \beta D_{KL}(\pi_{\theta}(\cdot|s_{i,t}) || \pi_{ref}(\cdot|s_{i,t})) \right) \quad (22)$$

Note that relative to PPO, the standard implementation of GRPO includes the KL distance in the loss. With the advantage computation for the completion index i :

$$A_i = \frac{r_i - \text{mean}(r_1, r_2, \dots, r_G)}{\text{std}(r_1, r_2, \dots, r_G)}. \quad (23)$$

Intuitively, the GRPO update is comparing multiple answers to a single question within a batch. The model learns to become more like the answers marked as correct and less like the others. This is a very simple way to compute the advantage, which is the measure of how much better a specific action is than the average at a given state. Relative to PPO, REINFORCE, and broadly RLHF performed with a reward model rating (relative to output reward), GRPO is often run with a far higher number of samples per prompt. Here, the current policy generates multiple responses to a given prompt, and the group-wise GRPO advantage estimate is given valuable context.

The advantage computation for GRPO has trade-offs in its biases. The normalization by standard deviation is rewarding questions in a batch that have a low variation in answer correctness. For questions with either nearly all correct or all incorrect answers, the standard deviation will be lower and the advantage will

be higher. [133] proposes removing the standard deviation term given this bias, but this comes at the cost of down-weighting questions that were all incorrect with a few correct answers, which could be seen as valuable learning signal.

eq. 23 is the implementation of GRPO when working with outcome supervision (either a standard reward model or a single verifiable reward) and a different implementation is needed with process supervision. In this case, GRPO computes the advantage as the sum of the normalized rewards for the following reasoning steps.

Finally, GRPO’s advantage estimation can also be applied without the PPO clipping to more vanilla versions of policy gradient (e.g. REINFORCE), but it is not the canonical form.

11.2 Implementation

Compared to the original Deep RL literature where many of these algorithms were developed, implementing RL for optimizing language models or other large AI models requires many small implementation details. In this section, we highlight some key factors that differentiate the implementations of popular algorithms.

There are many other small details that go into this training. For example, when doing RLHF with language models a crucial step is generating text that will then be rated by the reward model. Under normal circumstances, the model should generate a end-of-sequence (EOS) token indicating it finished generating, but a common practice is to put a hard cap on generation length to efficiently utilize infrastructure. A failure mode of RLHF is that the model is regularly truncated in its answers, driving the ratings from the reward model out of distribution and to unpredictable scores. The solution to this is to *only* run a reward model ranking on the `eos_token`, and to otherwise assign a penalty to the model for generating too long.

The popular open-source tools for RLHF have a large variance in implementation details across the algorithms (see table 10 in [137]). Some decisions not covered here include:

- **Value network initialization:** The internal learned value network used by PPO and other similar algorithms can be started from a different model of the same architecture or randomly selected weights. This can have a large impact on performance.
- **Reward normalization, reward whitening, and/or advantage whitening:** Where normalization bounds all the values from the RM (or environment) to be between 0 and 1, which can help with learning stability, whitening the rewards or the advantage estimates to uniform covariates can provide an even stronger boost to stability.
- **Different KL estimators:** With complex language models, precisely computing the KL divergence between models can be complex, so multiple

approximations are used to substitute for an exact calculation [115].

- **KL controllers:** Original implementations of PPO and related algorithms had dynamic controllers that targeted specific KLs and changed the penalty based on recent measurements. Most modern RLHF implementations use static KL penalties, but this can also vary.

For more details on implementation details for RLHF, see [138]. For further information on the algorithms, see [139].

11.2.1 Policy Gradient Basics

A simple implementation of policy gradient, using advantages to estimate the gradient to prepare for advanced algorithms such as PPO and GRPO follows:

```
pg_loss = -advantages * ratio
```

Ratio here is the logratio of the new policy model probabilities relative to the reference model.

In order to understand this equation it is good to understand different cases that can fall within a batch of updates. Remember that we want the loss to *decrease* as the model gets better at the task.

Case 1: Positive advantage, so the action was better than the expected value of the state. We want to reinforce this. In this case, the model will make this more likely with the negative sign. To do so it'll increase the logratio. A positive logratio, or sum of log probabilities of the tokens, means that the model is more likely to generate those tokens.

Case 2: Negative advantage, so the action was worse than the expected value of the state. This follows very similarly. Here, the loss will be positive if the new model was more likely, so the model will try to make it so the policy parameters make this completion less likely.

Case 3: Zero advantage, so no update is needed. The loss is zero, don't change the policy model.

11.2.2 Loss Aggregation

The question when implementing any policy gradient algorithm with language models is: How do you sum over the KL distance and loss to design different types of value-attribution.

Consider an example where we have the following variables, with a batch size B and sequence length L.

```
advantages # [B, 1]
per_token_probability_ratios # [B, L]
```

We can approximate the loss as above with a batch multiplication of `pg_loss = -advantages * ratio`. Multiplying these together is broadcasting the advantage

per each completion in the batch (as in the outcome reward setting, rather than a per-token value model setting) to be the same. They are then multiplied by the per token probability logratios.

In cases where a value network is used, it is easy to see that the different losses can behave very differently. When outcome rewards are used, the advantages are set to be the same per token, so the difference in per-token probability is crucial to policy gradient learning dynamics.

In the below implementations of GRPO and PPO, the loss is summed over the tokens in the completion:

```
sequence_loss = ((per_token_loss * completion_mask).sum(dim=1)
                 / \
                 completion_mask.sum(dim=1)).mean()
```

The operation above is very similar to a `masked_mean` operation. An alternative is to average over each token individually.

```
token_loss = ((per_token_loss * completion_mask).sum() / \
              completion_mask.sum())
```

Intuitively, it could seem that averaging over the sequence is best, as we are trying to reward the model for *outcomes* and the specific tokens are not as important. This can introduce subtle forms of bias. Consider two sequences of different lengths, assigned two different advantages `a_1` and `a_2`.

```
seq_1_advs = [a_1, a_1, a_1, a_1, a_1] # 5 tokens
seq_2_advs = [a_2, a_2, a_2, a_2, a_2, a_2, a_2, a_2, a_2, a_2]
              # 10 tokens
```

Now, consider if the last token in each sequence is important to the advantage being positive, so it gets increased over the multiple gradient steps per batch. When you convert these to per-token losses, you could get something approximate to:

```
seq_1_losses = [-1, -1, -1, -1, -10] # 5 tokens
seq_2_losses = [-1, -1, -1, -1, -1, -1, -1, -1, -1, -10] # 10
                tokens
```

If we average these over the sequences, we will get the following numbers:

```
seq_1_loss = -2.8
seq_2_loss = -1.9
```

If we average these together weighting sequences equally, we get a loss of -2.35. If, instead we apply the loss equally to each token, the loss would be computed by summing all the per token losses and normalizing by length, which in this case would be -2.27. If the sequences had bigger differences, the two loss values can have substantially different values.

Another way to aggregate loss is discussed in [TODO CITE DrGRPO] that has its origins in pre language model RL research, where every per-token loss is normalized by the max sequence length set in the experiment. This would change how the losses compare across batches per tokens in the above example.

In practice, the setup that is best likely is the one that is suited to the individual, online learning setup. Often in RLHF methods the method with the best numerical stability and or the least variance in loss could be preferred.

11.2.3 Proximal Policy Optimization

There are many, many implementations of PPO available. The core *loss* computation is shown below. Crucial to stable performance is also the *value* computation, where multiple options exist (including multiple options for the *value model* loss).

Note that the reference policy (or old logprobs) here are from the time the generations were sampled and not necessarily the reference policy. The reference policy is only used for the KL distance constraint/penalty.

```
# B: Batch Size, L: Sequence Length, G: Num of Generations
# Apply KL penalty to rewards
rewards = rewards - self.beta * per_token_kl # Shape: (B*G, L)

# Get value predictions
values = value_net(completions) # Shape: (B*G, L)

# Compute simple advantages
advantages = rewards - values.detach() # Shape: (B*G, L)

# Normalize advantages (optional but stable)
advantages = (advantages - advantages.mean()) / (advantages.std() + 1e-8)
advantages = advantages.unsqueeze(1) # Shape: (B*G, 1)

# Compute probability ratio between new and old policies
ratio = torch.exp(new_per_token_logps - per_token_logps) #
# Shape: (B*G, L)

# PPO clipping objective
eps = self.cliprange # e.g. 0.2
pg_losses1 = -advantages * ratio # Shape: (B*G, L)
pg_losses2 = -advantages * torch.clamp(ratio, 1.0 - eps, 1.0 + eps) # Shape: (B*G, L)
pg_loss_max = torch.max(pg_losses1, pg_losses2) # Shape: (B*G, L)

# Simple value function loss
vf_loss = 0.5 * ((rewards - values) ** 2) # Shape: (B*G, L)
```

```

# Combine policy and value losses
per_token_loss = pg_loss_max + self.vf_coef * vf_loss # Shape:
                (B*G, L)

# Apply completion mask and compute final loss
loss = ((per_token_loss * completion_mask).sum(dim=1) /
        completion_mask.sum(dim=1)).mean()
# Scalar

# Compute metrics for logging
with torch.no_grad():
    # Compute clipping fraction
    clip_frac = ((pg_losses2 > pg_losses1).float() *
                 completion_mask).sum() / completion_mask.sum()

    # Compute approximate KL
    approx_kl = 0.5 * ((new_per_token_logps - per_token_logps)
                       **2).mean()

    # Compute value loss for logging
    value_loss = vf_loss.mean()

```

The core piece to understand with PPO is how the policy gradient loss is updated. Focus on these three lines:

```

pg_losses1 = -advantages * ratio # Shape: (B*G, L)
pg_losses2 = -advantages * torch.clamp(ratio, 1.0 - eps, 1.0 +
    eps) # Shape: (B*G, L)
pg_loss_max = torch.max(pg_losses1, pg_losses2) # Shape: (B*G,
    L)

```

`pg_losses1` is the same as the vanilla advantage-based PR loss above, which is included in PPO, but the loss (and gradient update) can be clipped. Though, PPO is controlling the update size to not be too big. Because losses can be negative, we must create a more conservative version of the vanilla policy gradient update rule.

We know that if we *do not* constrain the loss, the policy gradient algorithm will update the weights exactly to the new probability distribution. Hence, by clamping the logratio's, PPO is limiting the distance that the update can move the policy parameters.

Finally, the max of two is taken as mentioned above, in order to take the more conservative loss update.

For PPO, all of this happens *while* learning a value function, which opens more complexity, but this is the core logic for the parameter update.

11.2.3.1 PPO/GRPO simplification with 1 gradient step per sample (no clipping) PPO (and GRPO) implementations can be handled much more

elegantly if the hyperparameter “number of gradient steps per sample” is equal to 1. Many normal values for this are from 2-4 or higher. In the main PPO or GRPO equations, see eq. 18, the “reference” policy is the previous parameters – those used to generate the completions or actions. Thus, if only one gradient step is taken, $\pi_\theta = \pi_{\theta_{old}}$, and the update rule reduces to the following (the notation $\llbracket_\nabla$ indicates a stop gradient):

$$J(\theta) = \frac{1}{G} \sum_{i=1}^G \left(\frac{\pi_\theta(a_i|s)}{\llbracket_\nabla \pi_\theta(a_i|s) \rrbracket_\nabla} A_i - \beta D_{KL}(\pi_\theta || \pi_{ref}) \right). \quad (24)$$

This leads to PPO or GRPO implementations where the second policy gradient and clipping logic can be omitted, making the optimizer far closer to standard policy gradient.

11.2.4 Group Relative Policy Optimization

The DeepSeekMath paper details some implementation details of GRPO that differ from PPO [134], especially if comparing to a standard application of PPO from Deep RL rather than language models. For example, the KL penalty within the RLHF optimization (recall the KL penalty is also used when training reasoning models on verifiable rewards without a reward model) is applied directly in the loss update rather to the reward function. Where the standard KL penalty application for RLHF is applied as $r = r_\theta + \beta D_{KL}$, the GRPO implementation is along the lines of:

$$L = L_{\text{policy gradient}} - \beta * D_{KL}$$

Though, there are multiple ways to implement this. Traditionally, the KL distance is computed with respect to each token in the completion to a prompt s . For reasoning training, multiple completions are sampled from one prompt, and there are multiple prompts in one batch, so the KL distance will have a shape of $[B, L, N]$, where B is the batch size, L is the sequence length, and N is the number of completions per prompt.

Putting it together, using the first loss accumulation, the psuedocode can be written as below.

```
# B: Batch Size, L: Sequence Length, G: Number of Generations
# Compute grouped-wise rewards # Shape: (B,)
mean_grouped_rewards = rewards.view(-1, self.num_generations).
    mean(dim=1)
std_grouped_rewards = rewards.view(-1, self.num_generations).
    std(dim=1)

# Normalize the rewards to compute the advantages
```

```

mean_grouped_rewards = mean_grouped_rewards.repeat_interleave(
    self.num_generations, dim=0)
std_grouped_rewards = std_grouped_rewards.repeat_interleave(
    self.num_generations, dim=0)
# Shape: (B*G,)

# Compute advantages
advantages = (rewards - mean_grouped_rewards) / (
    std_grouped_rewards + 1e-4)
advantages = advantages.unsqueeze(1)
# Shape: (B*G, 1)

# Compute probability ratio between new and old policies
ratio = torch.exp(new_per_token_logps - per_token_logps) #
    Shape: (B*G, L)

# PPO clipping objective
eps = self.cliprange # e.g. 0.2
pg_losses1 = -advantages * ratio # Shape: (B*G, L)
pg_losses2 = -advantages * torch.clamp(ratio, 1.0 - eps, 1.0 +
    eps) # Shape: (B*G, L)
pg_loss_max = torch.max(pg_losses1, pg_losses2) # Shape: (B*G,
    L)

# important to GRPO -- PPO applies this in reward traditionally
# Combine with KL penalty
per_token_loss = pg_loss_max + self.beta * per_token_kl #
    Shape: (B*G, L)

# Apply completion mask and compute final loss
loss = ((per_token_loss * completion_mask).sum(dim=1) /
    completion_mask.sum(dim=1)).mean()
# Scalar

# Compute core metric for logging (KL, reward, etc. also logged
)
with torch.no_grad():
    # Compute clipping fraction
    clip_frac = ((pg_losses2 > pg_losses1).float() *
        completion_mask).sum() / completion_mask.sum()

    # Compute approximate KL
    approx_kl = 0.5 * ((new_per_token_logps - per_token_logps)
        **2).mean()

```

For more details on how to interpret this code, see the PPO section above.

11.2.4.1 RLOO vs. GRPO The advantage updates for RLOO follow very closely to GRPO, highlighting the conceptual similarity of the algorithm when

taken separately from the PPO style clipping and KL penalty details. Specially, for RLOO, the advantage is computed relative to a baseline that is extremely similar to that of GRPO – the completion reward relative to the others for that same question. Concisely, the RLOO advantage estimate follows as (expanded from TRL’s implementation):

```
# rloo_k --> number of completions per prompt
# rlhf_reward --> Initially a flat tensor of total rewards for
# all completions. Length B = N x k
rlhf_reward = rlhf_reward.reshape(rloo_k, -1) #
# Now, Shape: (k, N), each column j contains the k rewards for
# prompt j.

baseline = (rlhf_reward.sum(0) - rlhf_reward) / (rloo_k - 1)
# baseline --> Leave-one-out baseline rewards. Shape: (k, N)
# baseline[i, j] is the avg reward of samples i' != i for
# prompt j.

advantages = rlhf_reward - baseline
# advantages --> Same Shape: (k, N)

advantages = advantages.flatten() # Same shape as original
# tensor
```

The rest of the implementation details for RLOO follow the other trade-offs of implementing policy-gradient.

11.3 Auxiliary Topics

In order to master the application of policy-gradient algorithms, there are countless other considerations. Here we consider some, but not all of these discussions.

11.3.1 Generalized Advantage Estimation (GAE)

Generalized Advantage Estimation (GAE) is an alternate method to compute the advantage for policy gradient algorithms [127] that better balances the bias-variance tradeoff. Traditional single-step advantage estimates often suffer from high variance, while using complete trajectories can introduce too much bias. GAE works by combining two ideas – multi-step prediction and weighted running average (or just one of these).

Advantage estimates can take many forms, but we can define a k step advantage estimator (similar to the TD residual at the beginning of the chapter) as follows:

$$\hat{A}_t^{(n)} = \begin{cases} r_t + \gamma V(s_{t+1}) - V(s_t), & n = 1 \\ r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2}) - V(s_t), & n = 2 \\ \vdots & \\ r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots - V(s_t), & n = \infty \end{cases} \quad (25)$$

Here a shorter k will have lower variance but higher bias as we are attributing more learning power to each trajectory – it can overfit. GAE attempts to generalize this formulation into a weighted multi-step average instead of a specific k . To start, we must define the temporal difference (TD) residual of predicted value.

$$\delta_t^V = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (26)$$

To utilize this, we introduce another variable λ as the GAE mixing parameter. This folds into an exponential decay of future advantages we wish to estimate:

$$\begin{aligned} \hat{A}_t^{GAE(\gamma, \lambda)} &= (1 - \lambda)(\hat{A}_t^{(1)} + \lambda \hat{A}_t^{(2)} + \lambda^2 \hat{A}_t^{(3)} + \dots) \\ &= (1 - \lambda)(\delta_t^V + \lambda(\delta_t^V + \gamma \delta_{t+1}^V) + \lambda^2(\delta_t^V + \gamma \delta_{t+1}^V + \gamma^2 \delta_{t+2}^V) + \dots) \\ &= (1 - \lambda)(\delta_t^V(1 + \lambda + \lambda^2 + \dots) + \gamma \delta_{t+1}^V(\lambda + \lambda^2 + \dots) + \dots) \\ &= (1 - \lambda)(\delta_t^V \frac{1}{1 - \lambda} + \gamma \delta_{t+1}^V \frac{\lambda}{1 - \lambda} + \dots) \\ &= \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \end{aligned} \quad (27)$$

Intuitively, this can be used to average of multi-step estimates of Advantage in an elegant fashion.

For further reading, see [140].

11.3.2 Double Regularization

Many popular policy gradient algorithms from Deep Reinforcement Learning originated due to the need to control the learning process of the agent. In RLHF, as discussed extensively in Chapter 8 on Regularization and in Chapter 4 on Problem Formulation, there is a built in regularization term via the distance penalty relative to the original policy one is finetuning. In this view, a large part of the difference between algorithms like PPO (which have internal step-size regularization) and REINFORCE (which is simpler, and PPO under certain hyperparameters reduces to) is far less meaningful for finetuning language models than training agents from scratch.

In PPO, the objective that handles capping the step-size of the update is known as the surrogate objective. To monitor how much the PPO regularization is impacting updates in RLHF, one can look at the clip fraction variable in many popular implementations, which is the percentage of samples in the batch where the gradients are clipped by this regularizer in PPO. These gradients are *reduced* to a maximum value.

11.3.3 Further Reading

As RLHF has cemented itself at the center of modern post-training, other policy-gradient RL algorithms and RL algorithms generally have been proposed to improve the training process, but they have not had a central role in governing best practices. Examples for further reading include:

- **Pairwise Proximal Policy Optimization (P3O)** [141] uses pairwise data directly in a PPO-style policy update without learning an intermediate reward model.
- Other implementations of REINFORCE algorithms have been designed for language models, such as **ReMax** [142], which implements a baseline normalization designed specifically to accommodate the sources of uncertainty from reward model inference.
- Some foundation models, such as Apple Intelligence Foundation Models [143] or Kimi k1.5 reasoning model [144], have used variants of **Mirror Descent Policy Optimization (MDPO)** [145]. Research is still developing further on the fundamentals here [146], but Mirror Descent is an optimization method rather than directly a policy gradient algorithm. What is important here is that it is substituted in very similarly to existing RL infrastructure.
- **Decoupled Clip and Dynamic sAmpling Policy Optimization (DAPO)** proposes 4 modifications to GRPO to better suit reasoning language models, where long traces are needed and new, underutilized tokens need to be increased in probability [147]. The changes are: 1, have two different clip hyperparameters, ϵ_{low} and ϵ_{high} , so clipping on the positive side of the logratio can take bigger steps for better exploration; 2, dynamic sampling, which removes all samples with reward = 0 or reward = 1 for all samples in the batch (no learning signal); 3, use the per token loss as discussed above in Implementation: GRPO; and 4, a soft penalty on samples that are too long to avoid trying to learn from truncated answers.

12 Direct Alignment Algorithms

Direct Alignment Algorithms (DAAs) allow one to update models to solve the same RLHF objective without ever training an intermediate reward model or using reinforcement learning optimizers. The most prominent DAA and one that catalyzed an entire academic movement of aligning language models is Direct Preference Optimization (DPO) [19]. At its core, DPO is using gradient ascent to solve the same constrained RLHF objective. Since its release in May of 2023, after a brief delay where the community figured out the right data and hyperparameters to use DPO with (specifically, surprisingly low learning rates), many popular models have used DPO or its variants, from Zephyr- β kickstarting it in October of 2023 [20], Llama 3 Instruct [23], Tülu 2 [21] and 3 [6], Nemotron 4 340B [24], and others. Technically, Sequence Likelihood Calibration (SLiC-HF) was released first [148], but it did not catch on due to a combination of luck and effectiveness.

The most impactful part of DPO and DAAs is lowering the barrier of entry to experimenting with language model post-training.

12.1 Direct Preference Optimization (DPO)

Here we explain intuitions for how it works and re-derive the core equations fully.

12.1.1 How DPO Works

DPO at a surface level is directly optimizing a policy to solve the RLHF objective. The loss function for this, which we will revisit below in the derivations, is a pairwise relationship of log-probabilities. The loss function derived from a Bradley-Terry reward model follows:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_c | x)}{\pi_{\text{ref}}(y_c | x)} - \beta \log \frac{\pi_{\theta}(y_r | x)}{\pi_{\text{ref}}(y_r | x)} \right) \right] \quad (28)$$

This relies on the implicit reward for DPO training that replaces using an external reward model, which is a log-ratio of probabilities:

$$r(x, y) = \beta \log \frac{\pi_r(y | x)}{\pi_{\text{ref}}(y | x)} \quad (29)$$

This comes from deriving the Bradley-Terry reward with respect to an optimal policy (shown in eq. 43), as shown in the Bradley-Terry model section. Essentially, the implicit reward model shows “the probability of human preference data in terms of the optimal policy rather than the reward model.”

Let us consider the loss shown in eq. 28. The learning process is decreasing the loss. Here, the loss will be lower when the log-ratio of the chosen response is bigger than the log-ratio of the rejected response (normalized by the reference model). In practice, this is a sum of log-probabilities of the model across the sequence of tokens in the data presented. Hence, DPO is increasing the delta in probabilities between the chosen and rejected responses.

With the reward in eq. 29, we can write the gradient of the loss to further interpret what is going on:

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\beta \mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} [\sigma(r_{\theta}(x, y_r) - r_{\theta}(x, y_c)) (\nabla_{\theta} \log \pi(y_c | x) - \nabla_{\theta} \log \pi(y_r | x))] \quad (30)$$

Here, the gradient solves the above objective by doing the following:

- The first term within the sigmoid function, $\sigma(\cdot)$, creates a weight of the parameter update from 0 to 1 that is higher when the reward estimate is incorrect. When the rejected sample is preferred over the chosen, the weight update should be larger!
- Second, the terms in the inner brackets $[\cdot]$ increases the likelihood of the chosen response y_c and decreases the likelihood of the rejected y_r .
- These terms are weighted by β , which controls how the update balances ordering the completions correctly relative to the KL distance.

The core intuition is that DPO is “fitting an implicit reward model whose corresponding optimal policy can be extracted in a closed form” (thanks to gradient ascent and our ML tools). What is often misunderstood is that DPO is learning a reward model at its core, hence the subtitle of the paper *Your Language Model is Secretly a Reward Model*. It is easy to confuse this with the DPO objective training a policy directly, hence studying the derivations below are good for a complete understanding.

With the implicit reward model learning, DPO is generating an optimal solution to the RLHF objective given the data in the dataset and the specific KL constraint in the objective β . Here, DPO solves for the exact policy given a specific KL distance because the generations are not online as in policy gradient algorithms – a core difference from the RL methods for preference tuning. In many ways, this makes the β value easier to tune with DPO relative to online RL methods, but crucially and intuitively the optimal value depends on the model being trained and the data training it.

At each batch of preference data, composed of many pairs of completions $y_{\text{chosen}} \succ y_{\text{rejected}}$, DPO takes gradient steps directly towards the optimal solution. It is far simpler than policy gradient methods.

LEARNING FROM HUMAN FEEDBACK

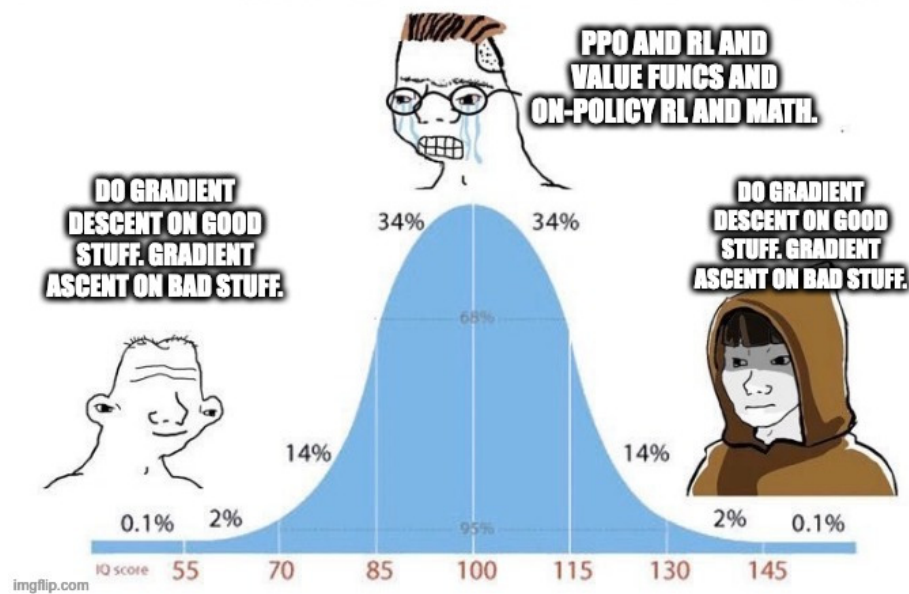


Figure 13: DPO simplicity meme, credit Tom Goldstein.

12.1.2 DPO Derivation

The DPO derivation takes two primary parts. First, the authors show the form of the policy that optimally solved the RLHF objective used throughout this book. Next, they show how to arrive at that solution from pairwise preference data (i.e. a Bradley Terry model).

12.1.2.1 1. Deriving the Optimal RLHF Solution To start, we should consider the RLHF optimization objective once again, here indicating we wish to maximize this quantity:

$$\max_{\pi} \mathbb{E}_{\tau \sim \pi} [r_{\theta}(s_t, a_t)] - \beta \mathcal{D}_{KL}(\pi^{\text{RL}}(\cdot | s_t) \| \pi^{\text{ref}}(\cdot | s_t)). \quad (31)$$

First, let us expand the definition of KL-divergence,

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \quad (32)$$

Next, pull the negative sign out of the difference in brackets. To do this, split it into two terms:

$$= \max_{\pi} \left(\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} [r(x, y)] - \beta \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \right) \quad (33)$$

Then, remove the factor of -1 and β ,

$$= \min_{\pi} \left(-\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} [r(x, y)] + \beta \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right] \right) \quad (34)$$

Divide by β and recombine:

$$= \min_{\pi} \left(\mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \right] \right) \quad (35)$$

Next, we must introduce a partition function, $Z(x)$:

$$Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right) \quad (36)$$

The partition function acts as a normalization factor over the reference policy, summing over all possible responses y to a prompt x . With this substituted in, we obtain our intermediate transformation:

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(y|x)} \left[\log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)} - \log Z(x) \right] \quad (37)$$

To see how this is obtained, consider the internal part of the optimization in brackets of eq. 35:

$$\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) \quad (38)$$

Then, add $\log Z(x) - \log Z(x)$ to both sides:

$$= \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} - \frac{1}{\beta} r(x, y) + \log Z(x) - \log Z(x) \quad (39)$$

Then, we group the terms:

$$= \left(\log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} + \log Z(x) \right) - \log Z(x) - \frac{1}{\beta} r(x, y) \quad (40)$$

With $\log(x) + \log(y) = \log(x \cdot y)$ (and moving Z to the denominator), we get:

$$= \log \frac{\pi(y|x)}{\frac{1}{Z(x)} \pi_{\text{ref}}(y|x)} - \log Z(x) - \frac{1}{\beta} r(x, y) \quad (41)$$

Next, we expand $\frac{1}{\beta} r(x, y)$ to $\log \exp \frac{1}{\beta} r(x, y)$ and do the same operation to get eq. 37. With this optimization form, we need to actually solve for the optimal policy π^* . To do so, let us consider the above optimization as a KL distance:

$$\min_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \left[\mathbb{D}_{\text{KL}} \left(\pi(y|x) \parallel \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right) \right) - \log Z(x) \right] \quad (42)$$

Since the partition function $Z(x)$ does not depend on the final answer, we can ignore it. This leaves us with just the KL distance between our policy we are learning and a form relating the partition, β , reward, and reference policy. The Gibb's inequality tells this is minimized at a distance of 0, only when the two quantities are equal! Hence, we get an optimal policy:

$$\pi^*(y|x) = \pi(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp \left(\frac{1}{\beta} r(x, y) \right) \quad (43)$$

12.1.2.2 2. Deriving DPO Objective for Bradley Terry Models To start, recall from Chapter 7 on Reward Modeling and Chapter 6 on Preference Data that a Bradley-Terry model of human preferences is formed as:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp(r^*(x, y_1))}{\exp(r^*(x, y_1)) + \exp(r^*(x, y_2))} \quad (44)$$

By manipulating eq. 43 by taking the logarithm of both sides and performing some algebra, one can obtain the DPO reward as follows:

$$r^*(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\text{ref}}(y \mid x)} + \beta \log Z(x) \quad (45)$$

We then can substitute the reward into the Bradley-Terry equation shown in eq. 44 to obtain:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)} + \beta \log Z(x)\right)}{\exp\left(\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)} + \beta \log Z(x)\right) + \exp\left(\beta \log \frac{\pi^*(y_2 \mid x)}{\pi_{\text{ref}}(y_2 \mid x)} + \beta \log Z(x)\right)} \quad (46)$$

By decomposing the exponential expressions from e^{a+b} to $e^a e^b$ and then cancelling out the terms $e^{\log(Z(x))}$, this simplifies to:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)}\right)}{\exp\left(\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)}\right) + \exp\left(\beta \log \frac{\pi^*(y_2 \mid x)}{\pi_{\text{ref}}(y_2 \mid x)}\right)} \quad (47)$$

Then, multiply the numerator and denominator by $\exp\left(-\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)}\right)$ to obtain:

$$p^*(y_1 \succ y_2 \mid x) = \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2 \mid x)}{\pi_{\text{ref}}(y_2 \mid x)} - \beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)}\right)} \quad (48)$$

Finally, with the definition of a sigmoid function as $\sigma(x) = \frac{1}{1+e^{-x}}$, we obtain:

$$p^*(y_1 \succ y_2 \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_1 \mid x)}{\pi_{\text{ref}}(y_1 \mid x)} - \beta \log \frac{\pi^*(y_2 \mid x)}{\pi_{\text{ref}}(y_2 \mid x)}\right) \quad (49)$$

This is the loss function for DPO, as shown in eq. 28. The DPO paper has an additional derivation for the objective under a Plackett-Luce Model, which is far less used in practice [19].

12.1.2.3 3. Deriving the Bradley Terry DPO Gradient We used the DPO gradient shown in eq. 30 to explain intuitions for how the model learns. To derive this, we must take the gradient of eq. 49 with respect to the model parameters.

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\nabla_{\theta} \mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_{\theta}(y_r|x)}{\pi_{\text{ref}}(y_r|x)} \right) \right] \quad (50)$$

To start, this can be rewritten. We know that the derivative of a sigmoid function $\frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x))$, the derivative of logarithm $\frac{d}{dx} \log x = \frac{1}{x}$, and properties of sigmoid $\sigma(-x) = 1 - \sigma(x)$, so we can reformat the above equation.

First, define the expression inside the sigmoid as $u = \beta \log \frac{\pi_{\theta}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} - \beta \log \frac{\pi_{\theta}(y_r|x)}{\pi_{\text{ref}}(y_r|x)}$. Then, we have

$$\nabla_{\theta} \mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} \left[\frac{\sigma'(u)}{\sigma(u)} \nabla_{\theta} u \right] \quad (51)$$

Expanding this and using the above expressions for sigmoid and logarithms results in the gradient introduced earlier:

$$-\mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} \left[\beta \sigma \left(\beta \log \frac{\pi_{\theta}(y_r|x)}{\pi_{\text{ref}}(y_r|x)} - \beta \log \frac{\pi_{\theta}(y_c|x)}{\pi_{\text{ref}}(y_c|x)} \right) [\nabla_{\theta} \log \pi(y_c|x) - \nabla_{\theta} \log \pi(y_r|x)] \right] \quad (52)$$

12.2 Numerical Concerns, Weaknesses, and Alternatives

Many variants of the DPO algorithm have been proposed to address weaknesses of DPO. For example, without rollouts where a reward model can rate generations, DPO treats every pair of preference data with equal weight. In reality, as seen in Chapter 6 on Preference Data, there are many ways of capturing preference data with a richer label than binary. Multiple algorithms have been proposed to re-balance the optimization away from treating each pair equally.

- **REgression to RELative REward Based RL (REBEL)** adds signal from a reward model, as a margin between chosen and rejected responses, rather than solely the pairwise preference data to more accurately solve the RLHF problem [117].
- **Conservative DPO (cDPO) and Identity Preference Optimization (IPO)** address the overfitting by assuming noise in the preference data. cDPO assumes N percent of the data is incorrectly labelled [19] and IPO changes the optimization to soften probability of preference rather than optimize directly from a label [149]. Practically, IPO changes the

preference probability to a nonlinear function, moving away from the Bradley-Terry assumption, with $\Psi(q) = \log\left(\frac{q}{1-q}\right)$.

- **DPO with an offset (ODPO)** “requires the difference between the likelihood of the preferred and dispreferred response to be greater than an offset value” [150] – do not treat every data pair equally, but this can come at the cost of a more difficult labeling environment.

Some variants to DPO attempt to either improve the learning signal by making small changes to the loss or make the application more efficient by reducing memory usage.

- **Odds Ratio Policy Optimization (ORPO)** directly updates the policy model with a pull towards the chosen response, similar to the instruction finetuning loss, with a small penalty on the chosen response [151]. This change of loss function removes the need for a reference model, simplifying the setup. The best way to view ORPO is DPO inspired, rather than a DPO derivative.
- **Simple Preference Optimization SimPO** makes a minor change to the DPO optimization, by averaging the log-probabilities rather than summing them (SimPO) or adding length normalization, to improve performance [152].

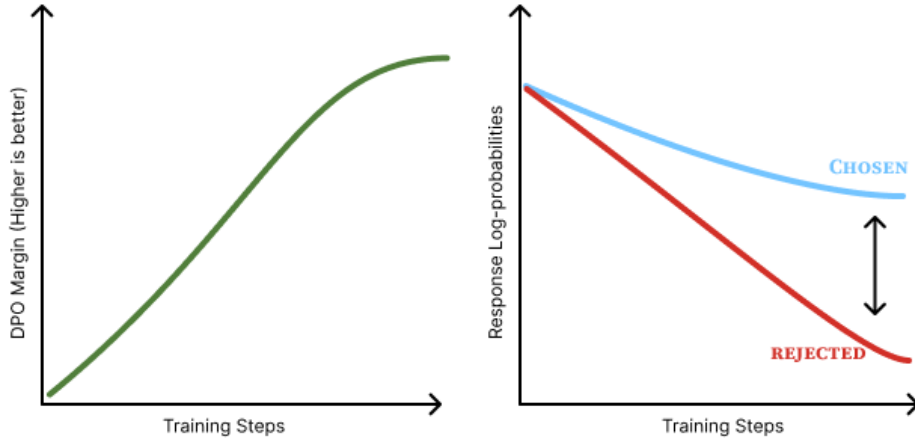


Figure 14: Sketch of preference displacement in DPO.

One of the core issues *apparent* in DPO is that the optimization drives only to increase the margin between the probability of the chosen and rejected responses. Numerically, the model reduces the probability of both the chosen and rejected responses, but the *rejected response is reduced by a greater extent* as shown in fig. 14. Intuitively, it is not clear how this generalizes, but work has posited that it increases the probability of unaddressed for behaviors [153] [154]. Simple methods, such as Cal-DPO [155], adjust the optimization so that this **preference displacement** does not occur. In practice, the exact impact of

this is not well known, but points are a potential reason why online methods can outperform vanilla DPO.

The largest other reason that is posited for DPO-like methods to have a lower ceiling on performance than online (RL based) RLHF methods is that the training signal comes from completions from previous or other models. Online variants that sample generations from the model, e.g. **Online DPO** [156], even with regular reward model relabelling of newly created creations **Discriminator-Guided DPO** (D2PO) [157], alleviate these by generating new completions for the prompt and incorporating a preference signal at training time.

There is a long list of other DAA variants, such as Direct Nash Optimization (DNO) [158] or Binary Classifier Optimization (BCO) [159], but the choice of algorithm is far less important than the initial model and the data used [6] [160] [161].

12.3 Implementation Considerations

DAAs such as DPO are implemented very differently than policy gradient optimizers. The DPO loss, taken from the original implementation, largely can be summarized as follows [19]:

```
pi_logratios = policy_chosen_logps - policy_rejected_logps
ref_logratios = reference_chosen_logps -
    reference_rejected_logps

logits = pi_logratios - ref_logratios # also known as  $h_{\pi_{\theta}}(y_w, y_l)$ 

losses = -F.logsigmoid(beta * logits)

chosen_rewards = beta * (policy_chosen_logps -
    reference_chosen_logps).detach()
rejected_rewards = beta * (policy_rejected_logps -
    reference_rejected_logps).detach()
```

This can be used in standard language model training stacks as this information is already collated during the forward pass of a model (with the addition of a reference model).

In most ways, this is simpler and an quality of life improvement, but also they offer a different set of considerations.

1. **KL distance is static:** In DPO and other algorithms, the KL distance is set explicitly by the β parameter that balances the distance penalty to the optimization. This is due to the fact that DPO takes gradient steps towards the *optimal* solution to the RLHF objective given the data – it steps exactly to the solution set by the β term. On the other hand, RL based optimizers take steps based on the batch and recent data.

2. **Caching log-probabilities:** Simple implementations of DPO do the forward passes for the policy model and reference models at the same time for conveniences with respect to the loss function. Though, this doubles the memory used and results in increased GPU usage. To avoid this, one can compute the log-probabilities of the reference model over the training dataset first, then reference it when computing the loss and updating the parameters per batch, reducing the peak memory usage by 50%.

12.4 DAAs vs. RL: Online vs. Offline Data

Broadly, the argument boils down to one question: Do we need the inner workings of reinforcement learning, with value functions, policy gradients, and all, to align language models with RLHF? This, like most questions phrased this way, is overly simplistic. Of course, both methods are well-established, but it is important to illustrate where the fundamental differences and performance manifolds lie.

Multiple reports have concluded that policy-gradient based and RL methods outperform DPO and its variants. The arguments take different forms, from training models with different algorithms but controlled data [137] [162] or studying the role of on-policy data within the RL optimization loop [163]. In all of these cases, DPO algorithms are a hair behind.

Even with this performance delta, DAA are still used extensively in leading models due to its simplicity. DAAs provide a controlled environment where iterations on training data and other configurations can be made rapidly, and given that data is often far more important than algorithms, using DPO can be fine.

With the emergence of reasoning models that are primarily trained with RL, further investment will return to using RL for preference-tuning, which in the long-term will improve the robustness of RL infrastructure and cement this margin between DAAs and RL for optimizing from human feedback.

13 Constitutional AI & AI Feedback

RL from AI Feedback (RLAIF) is a larger set of techniques for using AI to augment or generate feedback data, including pairwise preferences [164] [165] [166]. There are many motivations to using RLAIF to either entirely replace human feedback or augment it. AI models are far cheaper than humans, with a single piece of human preference data costing on the order of \$1 or higher (or even above \$10 per prompt), AI feedback with a frontier AI model, such as GPT-4o costs less than \$0.01. This cost difference opens the market of experimentation with RLHF methods to an entire population of people previously priced out. Other than price, AI feedback introduces different *tradeoffs* on performance than human feedback, which are still being investigated. The peak performance for AI feedback is at least in the same ballpark of human data on skill-based evaluations, but it is not studied if human data allows finer control of the models in real-world product settings or for newer training methods such as character training.

The term RLAIF was introduced in Anthropic’s work *Constitutional AI: Harmlessness from AI Feedback* [18], which resulted in initial confusion in the AI community over the relationship between the methods. Since the release of the Constitutional AI (CAI) paper and the formalization of RLAIF, RLAIF has become a default method within the post-training and RLHF literatures – there are far more examples than one can easily enumerate. The relationship should be understood as CAI was the example that kickstarted the broader field of RLAIF.

A rule of thumb for the difference between human data and AI feedback data is as follows:

1. Human data is high-noise and low-bias,
2. Synthetic preference data is low-noise and high-bias,

Results in many academic results showing how one can substitute AI preference data in RLHF workflows and achieve strong evaluation scores [167], but shows how the literature of RLHF is separated from industrial best practices.

13.1 Constitutional AI

The method of Constitutional AI (CAI), which Anthropic uses extensively in their Claude models, is the earliest, large-scale use of synthetic data for RLHF training. Constitutional AI has two uses of synthetic data:

1. Critiques of instruction-tuned data to follow a set of principles like “Is the answer encouraging violence” or “Is the answer truthful.” When the model generates answers to questions, it checks the answer against the list of principles in the constitution, refining the answer over time. Then, they fine-tune the model on this resulting dataset.
2. Generates pairwise preference data by using a language model to answer

which completion was better, given the context of a random principle from the constitution (similar to this paper for principle-guided reward models). Then, RLHF proceeds as normal with synthetic data, hence the RLAIIF name.

Largely, CAI is known for the second half above, the preference data, but the methods introduced for instruction data are used in general data filtering and synthetic data generation methods across post-training.

CAI can be formalized as follows.

By employing a human-written set of principles, which they term a *constitution*, Bai et al. 2022 use a separate LLM to generate artificial preference and instruction data used for fine-tuning [18]. A constitution $\mathcal{C}$ is a set of written principles indicating specific aspects to focus on during a critique phase. The instruction data is curated by repeatedly sampling a principle $c_i \in \mathcal{C}$ and asking the model to revise its latest output y^i to the prompt x to align with c_i . This yields a series of instruction variants $\{y^0, y^1, \dots, y^n\}$ from the principles $\{c_0, c_1, \dots, c_{n-1}\}$ used for critique. The final data point is the prompt x together with the final completion y^n , for some n .

The preference data is constructed in a similar, yet simpler way by using a subset of principles from $\mathcal{C}$ as context for a feedback model. The feedback model is presented with a prompt x , a set of principles $\{c_0, \dots, c_n\}$, and two completions y_0 and y_1 labeled as answers (A) and (B) from a previous RLHF dataset. The feedback models' probability of outputting either (A) or (B) is recorded as a training sample for the reward model

13.2 Specific LLMs for Judgement

As RLAIIF and LLM-as-a-judge has become more prevalent, many have wondered if we should be using the same models for generating responses as those for generating critiques or ratings. Multiple models have been released with the goal of substituting for frontier models as a data labeling tool, such as critic models Shepherd [168] and CriticLLM [169] or models for evaluating response performance akin to Auto-J [170], Prometheus [93], Prometheus 2 [171], or Prometheus-Vision [172] but they are not widely adopted in documented training recipes.

13.3 Further Reading

There are many related research directions and extensions of Constitutional AI, but few of them have been documented as clear improvements in RLHF and post-training recipes. For now, they are included as further reading.

- OpenAI has released a Model Spec [78], which is a document stating the intended behavior for their models, and stated that they are exploring methods for alignment where the model references the document directly

(which could be seen as a close peer to CAI). OpenAI has continued and trained their reasoning models such as o1 with a method called Deliberative Alignment [173] to align the model while referencing these safety or behavior policies.

- Anthropic has continued to use CAI in their model training, updating the constitution Claude uses [174] and experimenting with how population collectives converge on principles for models and how that changes model behavior [175].
- The open-source community has explore replications of CAI applied to open datasets [176] and for explorations into creating dialogue data between LMs [177].
- Other work has used principle-driven preferences or feedback with different optimization methods. [178] uses principles as context for the reward models, which was used to train the Dromedary models [179]. [36] uses principles to improve the accuracy of human judgments in the RLHF process.

14 [Incomplete] Reasoning Training & Models

At the 2016 edition of the Neural Information Processing Systems (NeurIPS) conference, Yann LeCun first introduced his now-famous cake metaphor for where learning happens in modern machine learning systems:

If intelligence is a cake, the bulk of the cake is unsupervised learning, the icing on the cake is supervised learning, and the cherry on the cake is reinforcement learning (RL).

This analogy is now largely complete with modern language models. Self-supervised learning on vast swaths of internet data makes up the majority of the cake (especially when viewed in compute spent in FLOPs), the beginning of post-training in supervised finetuning (SFT) for instructions tunes the model to a narrower distribution, and finally “pure” reinforcement learning (RL) is the cherry on top. We learn just “a few bits” of information with RL in just a few training samples.

This little bit of RL takes many forms. It can be ... TODO

Despite many, many takes that “RL doesn’t work yet” or “RL scaling isn’t ready yet” (and implicit versions of this saying to focus on “RL that Matters”), Yann’s view seems to have been right.

OpenAI’s new Reinforcement Finetuning (RFT) API (just a research program for now), announced on day 2 of the 12 days of OpenAI, is the bridge that brings RL to the masses. This is a very surprising development even for those most faithful to RL. With RFT, one can likely finetune any of OpenAI’s models, while they highlighted o1 mini, it is of obvious value to both standard autoregressive models and reasoning-heavy models. To use RFT, you need three things — 1) training data for your application, 2) validation data for your application to test overfitting, and 3) a definition via OpenAI’s “grader” configuration (more on this later).

Reinforcement Finetuning has been met with excitement and trepidation. The best practices for using existing finetuning APIs, built on instruction tuning infrastructure, are still far from established. The general public of AI builders knows very little about how RL training can change model behavior to improve performance on tasks with minimal overall changes to the model.

In many domains, Reinforcement Finetuning is much more aligned with the goals of developers by being focused on performance rather than behavior. Standard finetuning APIs generally use a parameter-efficient finetuning method such as LoRA with supervised finetuning on instructions. Developers pass in prompts and completions and the model is tuned to match that by updating model parameters to match the completions. OpenAI describes this as increasing the prevalence of “features” in the text of interest.

Reinforcement finetuning is focused on matching answers. Given queries and correct answers, RFT helps the model learn to get the correct answers. While

standard instruction tuning is done with 1 or 2 epochs of loss updates over the data, reinforcement finetuning gets its name by doing hundreds or thousands of epochs over the same few data points to give the model time to learn new behaviors. This can be viewed as reinforcing positive behaviors that would work sparingly in the base model version into robust behaviors after RFT.

The impact of reinforcement finetuning’s existence Reinforcement finetuning signals many changes to the fate of RL and language models, both at OpenAI and elsewhere:

- Stability of RL can be solved: For its entire existence, the limiting factor on RL’s adoption has been stability. This manifests in two ways. First, the learning itself can be fickle and not always work. Second, the training itself is known to be more brittle than standard language model training and more prone to loss spikes, crashes, etc. Releasing an API where any user can train on their own data signals unprecedented stability improvements. This program is still a beta ahead of a full launch, but this signals that OpenAI is more confident about it working for the public rather than not. For example, last year when I heard about large-scale RL runs at frontier AI laboratories, it would be with stories like “they launch multiple seeds at once and only keep running the ones that didn’t crash.” Now, they can be confident in their RL running and accomplishing the task. The final output model is likely automatically detected by running evaluations on the checkpoint to make sure behavior did not dip and or measuring the KL distance from the initial policy. Both of these are signals that researchers rely on heavily in post-training experimentation, so automating decision-making based on it is extremely impactful.
- Open-source versions already “exist”: Our recent work at Ai2 on reinforcement learning with verifiable rewards (RLVR) is extremely similar. The major components, i.e. data format and optimizer type are identical, we just need increased open-source investment to understand many discussion items like which model to start on, which types of data to use, etc. Check out the code we’ve been using at Open Instruct.
- A potential data flywheel for advanced reasoning models: The best speculation is that OpenAI’s o1 is trained mostly with large-scale RL on data with verifiable outputs — much like this API. If this API works as intended, OpenAI could accumulate an extreme dataset for training future versions of their o1 models. The main limitation of these models is the lack of diversity in available domains and by experimenting with training on the targeted domains of many users of OpenAI’s models they can start turning a fruitful flywheel.

The scope of RL training for language models continues to grow: The biggest takeaway from o1 on a fundamental scientific level was that we have even more ways to train language models to potentially valuable behaviors. The more open doors that are available to researchers and engineers, the more optimism

we should have about AI's general trajectory. The RL finetuning API expands the window of permissivity for RL training.

I recall listening to a talk by a prominent OpenAI researcher a year ago (I couldn't find it), where they said they were excited about RLHF and related methods just because the loss function is more general than autoregressive prediction. We are now living in this world of RL's impact growing rapidly, and as many people expected, the human feedback piece is not always necessary.

15 [Incomplete] Synthetic Data & Distillation

Reinforcement learning from *human feedback* is deeply rooted in the idea of keeping a human influence on the models we are building. When the first models were trained successfully with RLHF, human data was *the only* viable way to improve the models in this way.

Humans were the only way to create high enough quality responses to questions to train on them. Humans were the only way to collect reliable and specific feedback data to train reward models.

As AI models got better, this assumption rapidly broke down. The possibility of synthetic data, which is far cheaper and easier to iterate on, enabled the proliferation from RLHF being the center of attention to the idea of a broader “post-training” shaping the models.

Many reports have been made on how synthetic data causes “model collapse” or other issues in models [180], but this has been emphatically rebuked in leading language models [181] [182]. Synthetic data *can* cause models to have performance issues, but this is caused by using repetitive data or solely data outputted by the model being trained (narrowing its potential distribution) rather than well-rounded data sources.

The leading models **need synthetic data** to reach the best performance. Synthetic data in modern post-training encompasses many pieces of training – language models are used to generate new training prompts from seed examples [183], modify existing prompts, generate completions to prompts [184], provide AI feedback to create preference data [22], filter completions [185], and much more. Synthetic data is key to post-training.

The ability for synthetic data to be impactful to this extent emerged with GPT-4 class models. With early language models, such as Llama 2 and GPT-3.5-Turbo, the models were not reliable enough in generating or supervising data pipelines. Within 1-2 years, language models were far superior to humans for generating answers. In the transition from GPT-3.5 to GPT-4 class models, the ability for models to perform LLM_as-a-judge tasks also emerged. GPT-4 or better models are far more robust and consistent in generating feedback or scores with respect to a piece of content.

Since this transition, the role of synthetic data has only grown in language model training. Otherwise, there are two clear areas where human data continues to be important.

1. The role of human data continues to be at the fringe of capabilities in models – humans must generate data where AI’s do not yet have any ability. Once the first strong model exists, synthetic data proliferates.
2. Human preference data is still used in the leading models, even though academic work shows synthetic versions to perform just as well. The role of human preferences is still being established in the literature.

15.1 Distillation

The term distillation has been the most powerful form of discussion around the role of synthetic data in language models.

15.1.1 As a Data Engine

15.1.2 Transferring Behaviors

16 [Incomplete] Evaluation & Prompting

16.1 How To Tell if RLHF is Working?

16.2 Prompting

Prompting, i.e. crafting the correct query for a model, is a crucial portion of using them as the models are evolving rapidly.

16.3 Evaluation

16.3.1 Formatting and Overview

16.3.2 ChatBotArena

ChatBotArena is the largest community evaluation tool for language models. The LMSYS team, which emerged early in the post-ChatGPT craze, works with most of the model providers to host all of the relevant models. If you're looking to get to know how multiple models compare to each other, ChatBotArena is the place to start.

ChatBotArena casts language model evaluation through the wisdom of the crowd. For getting an initial ranking of how models stack up and how the models in the ecosystem are getting better, it has been and will remain crucial.

ChatBotArena does not represent a controlled nor interpretable experiment on language models.

When evaluating models to learn which are the best at extremely challenging tasks, distribution control, and careful feedback are necessary. For these reasons, ChatBotArena cannot definitively tell us which models are solving the hardest tasks facing language models. It does not measure how the best models are improving in clear ways. This type of transparency comes elsewhere.

For most of its existence, people correlated the general capabilities tested in ChatBotArena with a definitive ranking of which models can do the hardest things for me. This is not true. In both my personal experience reading data and what the community knows about the best models, the ChatBotArena ranking shows the strongest correlations with:

Certain stylistic outputs from language models, and

Language models that have high rates of complying with user requests.

Both of these have been open research problems in the last two years. Style is deeply intertwined with how information is received by the user and precisely refusing only the most harmful requests is a deeply challenging technical problem that both Meta (with Llama 2) and Anthropic (with earlier versions of Claude particularly) have gotten deeply criticized for.

Among closed labs, their styles have been greatly refined. All of Meta, OpenAI, and Anthropic have distinctive styles (admittedly, I haven't used Google's Gemini enough to know).

Meta's AI is succinct and upbeat (something that has been discussed many times on the LocalLlama subreddit).

OpenAI's style is the most robotic to me. It answers as an AI and contains a lot of information.

Claude's style is intellectual, bordering on curious, and sometimes quick to refuse.

When ChatBotArena was founded, these styles were in flux. Now, they majorly shift the rankings depending on what people like. People seem to like what OpenAI and Meta put out.

There are clear reasons why OpenAI's models top the charts on ChatBotArena. They were the originators of modern RLHF, have most clearly dictated their goals with RLHF, continue to publish innovative ideas in the space, and have always been ahead here. Most people just did not realize how important this was to evaluation until the launch of GPT-4o-mini. Culture impacts AI style.

16.3.3 Private Leaderboards

Scale Leaderboard etc

17 Over Optimization

In the RLHF literature and discourse, there are three directions that over-optimization can emerge:

1. **Quantitative research** on the technical notion of over-optimization of reward,
2. **Qualitative observations** that “overdoing” RLHF can result in worse models.
3. **Misalignment** where overdoing RLHF or related techniques can make a language model behave against its design.

This chapter provides a cursory introduction to both. We begin with the latter, qualitative, because it motivates the problem to study further.

17.1 Qualitative (behavioral) over-optimization

Note: This section draws on two blog posts from Interconnects.ai. It can also be viewed as an “objective mismatch” [186] [187].

17.1.1 Managing proxy objectives

The thing about RLHF that should be more obvious is that we don’t have a good reward function for chatbots. RLHF has been driven into the forefront because of its impressive performance at making chatbots a bit better to use (from both eliminating bad stuff and a bit of adding capabilities), which is entirely governed by a proxy objective — thinking that the rewards measured from human labelers in a controlled setting mirror those desires of downstream users. Post-training generally has emerged to include training on explicitly verifiable rewards, but standard learning from preferences alone also improves performance on domains such as mathematical reasoning and coding.

The proxy reward in RLHF is the score returned by a trained reward model to the RL algorithm itself because it is known to only be at best correlated with chatbot performance [188]. Therefore, it’s been shown that applying too much optimization power to the RL part of the algorithm will actually decrease the usefulness of the final language model. And over-optimization, put simply by John, is “when optimizing the proxy objective causes the true objective to get better, then get worse.” A curve where the training loss goes up, slowly levels off, then goes down. This is different from overfitting, where the model accuracy keeps getting better on the training distribution. Over-optimization of a proxy reward is much more subtle (and linked to the current evaluation fog in NLP, where it’s hard to know which models are actually “good”).

The general notion captured by this reasoning follows from Goodhart’s law, which is colloquially the notion that “When a measure becomes a target, it ceases to be a good measure.” This adage is derived from Goodhart’s writing [189]:

Any observed statistical regularity will tend to collapse once pressure is placed upon it for control purposes.

The insight here builds on the fact that we have optimizations we are probably incorrectly using ML losses as ground truths in these complex systems. In reality, the loss functions we use are designed (and theoretically motivated for) local optimizations. The global use of them is resulting in challenges with the RLHF proxy objective.

Common signs of over-optimization in early chat models emerged as:

- “As an AI language model...”
- “Certainly!...”
- Repetitiveness, hedging, ...
- Self-doubt, sycophancy [190], and over apologizing
- Over refusals (more below)

Technically, it is an open question on which types of error in the training process result in these failures. Many sources of error exist [188]: Approximation error from reward models not being able to fit to preferences, estimation error from overfitting during training the RM, optimization error in training the language model policy, etc. This points to a fundamental question as to the limits of optimization the intents of data contractors relative to what downstream users want.

A potential solution is that *implicit* feedback will be measured from users of chatbots and models to tune performance. Implicit feedback is actions taken by the user, such as re-rolling an output, closing the tab, or writing an angry message that indicates the quality of the previous response. The challenge here, and with most optimization changes to RLHF, is that there’s a strong risk of losing stability when making the reward function more specific. RL, as a strong optimizer, is increasingly likely to exploit the reward function when it is a smooth surface (and not just pairwise human values). The expected solution to this is that future RLHF will be trained with both pairwise preference data and additional steering loss functions. There are also a bunch of different loss functions that can be used to better handle pairwise data, such as Mallow’s model [191] or Plackett-Luce [82].

17.1.2 Llama 2 and “too much RLHF”

17.1.3 An aside on “undercooking” RLHF

As training practices for language models have matured, there are also prominent cases where strong models do not have an amount of post-training that most users expect, resulting in models that are harder to use than their evaluation scores would suggest.

TODO add references to minimax model? See tweets etc?

17.2 Quantitative over-optimization

KL is the primary metric,

Put simply, the solution that will most likely play out is to use bigger models. Bigger models have more room for change in the very under-parameterized setting of a reward model (sample efficient part of the equation), so are less impacted. DPO may not benefit from this as much, the direct optimization will likely change sample efficiency one way or another.

[37]

17.3 Misalignment

Consequences [192] or sycophancy [190]

18 Style and Information

This chapter draws on content from two / posts on the role of style in post-training and evaluation of RLHF'd models.

Early developments in RLHF gave it a reputation for being “just style transfer” or other harsh critiques on how RLHF manipulates the way information is presented in outputs.

Style transfer, has held back the RLHF narrative for two reasons.

First, when people discuss style transfer, they don’t describe this as being important or exciting. Style is a never-ending source of human value, it’s why retelling stories can result in new bestselling books (such as *Sapiens*), and it is a fundamental part of continuing to progress our intellectual ecosystem. Style is intertwined with what the information is.

Second, we’ve seen how different styles actually can improve evaluation improvements with Llama 3 [23]. The Llama 3 Instruct models scored extremely high on ChatBotArena, and it’s accepted as being because they had a more fun personality. If RLHF is going to make language models simply more fun, that is delivered value.

18.1 The Chattiness Paradox

TODO EDIT

RLHF or preference fine-tuning methods are being used mostly to boost scores like AlpacaEval and other automatic leaderboards without shifting the proportionally on harder-to-game evaluations like ChatBotArena. The paradox is that while alignment methods like DPO give a measurable improvement on these models that does transfer into performance that people care about, a large swath of the models doing more or less the same thing take it way too far and publish evaluation scores that are obviously meaningless.

For how methods like DPO can simply make the model better, some of my older articles on scaling DPO and if we even need PPO can help. These methods, when done right, make the models easier to work with and more enjoyable. This often comes with a few percentage point improvements on evaluation tools like MT Bench or AlpacaEval (and soon Arena Hard will show the same). The problem is that you can also use techniques like DPO and PPO in feedback loops or in an abundance of data to actually lobotomize the model at the cost of LLM-as-a-judge performance. There are plenty of examples.

Some of the models I’m highlighting here are academic papers that shouldn’t entirely be judged on “if the final model passes vibes tests,” but are illustrative of the state of the field. These are still useful papers, just not something everyone will immediately use for training state-of-the-art models. Those come from downstream papers.

During the proliferation of the DPO versus PPO debate there were many papers that came out with ridiculous benchmarks but no model weights that gathered sustained usage. When applying RLHF, there is no way to make an aligned version of a 7 billion parameter model actually beat GPT-4. It seems obvious, but there are papers claiming these results. Here’s a figure from a paper called Direct Nash Optimization (DNO) that makes the case that their model is state-of-the-art or so on AlpacaEval.

Even the pioneering paper Self Rewarding Language Models disclosed ridiculous scores on Llama 2 70B. A 70B model can get closer to GPT-4 than a 7B model can, as we have seen with Llama 3, but it’s important to separate the reality of models from the claims in modern RLHF papers. Many more methods have come and gone in the last few months. They’re the academic line of work that I’m following, and there’s insight there, but the methods that stick will be accompanied by actually useful models some number of months down the line.

Other players in industry have released models alone (rather than papers) that gamify these metrics. Two examples that come to mind are the Mistral 7B fine-tune from Snorkel AI or a similar model from Contextual trained with KTO. There are things in common here — using a reward model to further filter the data, repeated training via some sort of synthetic data feedback, and scores that are too good to be true.

A symptom of models that have “funky RLHF” applied to them has often been a length bias. This got so bad that multiple evaluation systems like AlpacaEval and WildBench both have linear length correction mechanisms in them. This patches the incentives for doping on chattiness to “beat GPT-4,” and adds a less gamified bug that shorter and useful models may actually win out. So far so good on the linear length controls.

Regardless, aligning chat models simply for chattiness still has a bit of a tax in the literature. This note from the Qwen models is something that has been seen multiple times in early alignment experiments. I suspect this is mostly about data.

We pretrained the models with a large amount of data, and we post-trained the models with both supervised finetuning and direct preference optimization. However, DPO leads to improvements in human preference evaluation but degradation in benchmark evaluation.

A good example of this tradeoff done right is a model like Starling Beta. It’s a model that was fine-tuned from another chat model, OpenChat, which was in fact trained by an entire other organization. It’s training entirely focuses on a k-wise reward model training and PPO optimization, and moves it up 10 places in ChatBotArena. The average response length of the model increases, but in a way that’s good enough to actually help the human raters.

18.1.1 How Chattiness Emerges

TODO EDIT

Let’s round out this article with how RLHF is actually achieving chattiness at the parameter level. Most of the popular datasets for alignment these days are synthetic preferences where a model like GPT-4 rates outputs from other models as the winner or the loser. Given that GPT-4 is known to have length and style biases for outputs that match itself, most of the pieces of text in the “preferred” section of the dataset are either from an OpenAI model or are stylistically similar to it. The important difference is that not all of the pieces of text in the dataset will have that. They’re often generated from other open models like Alpaca, Vicuna, or more recent examples. These models have very different characteristics.

Next, now that we’ve established that we have a preference dataset where most of the chosen models are similar to ChatGPT (or some other model that is accepted to be “strong”), these alignment methods simply increase the probability of these sequences. The math is somewhat complicated, where the batches of data operate on many chosen-rejected pairs at once, but in practice, the model is doing credit assignment over sequences of tokens (subword pieces). Preference alignment for chattiness is making the sequences found in outputs of models like GPT-4 more likely and the sequences from other, weaker models less likely. Repeatedly, this results in models with longer generations and characteristics that people like more.

Those among you who are familiar with RLHF methods may ask if the KL constraint in the optimization should stop this from happening. The KL constraint is a distance term between the distribution of the original model and the resulting model. It helps make the optimization more robust to overoptimization, but that makes the border between good and bad models a bit more nuanced. Hence, the prevalence of vibes-based evaluations. Though, models tend to have enough parameters where they can change substantially and still satisfy the KL constraint on the data being measured — it can’t be the entire pertaining dataset, for example.

As more models than ChatGPT become prevalent and strong enough for creating synthetic data, the distribution of outcomes we can expect from our aligned models should shift. There are two key places where the data influences this process: 1) where the text used to train the model is generated and 2) which LLM is used to determine which answer is the “winner” and “loser” in the preference learning framework. While all of these models have licenses or terms of service that make this practice technically violate an agreement of use, we’ve had more than a year of progress in open alignment practices relying on them in the past, so I don’t expect it to change. Mistral AI is the only LLM provider that doesn’t have a term restricting training on outputs (as far as I know).

19 Character Training & Model Character

19.1 Character Training

19.2 Model Specification & Documentation

20 Product, UX, and Post-Training

Bibliography

- [1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, “Deep reinforcement learning from human preferences,” *Advances in neural information processing systems*, vol. 30, 2017.
- [2] N. Stiennon *et al.*, “Learning to summarize with human feedback,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 3008–3021, 2020.
- [3] L. Ouyang *et al.*, “Training language models to follow instructions with human feedback,” *Advances in neural information processing systems*, vol. 35, pp. 27730–27744, 2022.
- [4] R. Nakano *et al.*, “Webgpt: Browser-assisted question-answering with human feedback,” *arXiv preprint arXiv:2112.09332*, 2021.
- [5] Y. Bai *et al.*, “Training a helpful and harmless assistant with reinforcement learning from human feedback,” *arXiv preprint arXiv:2204.05862*, 2022.
- [6] N. Lambert *et al.*, “T\” ULU 3: Pushing frontiers in open language model post-training,” *arXiv preprint arXiv:2411.15124*, 2024.
- [7] R. Kirk *et al.*, “Understanding the effects of rlhf on llm generalisation and diversity,” *arXiv preprint arXiv:2310.06452*, 2023.
- [8] T. Chu *et al.*, “Sft memorizes, rl generalizes: A comparative study of foundation model post-training,” *arXiv preprint arXiv:2501.17161*, 2025.
- [9] P. Singhal, T. Goyal, J. Xu, and G. Durrett, “A long way to go: Investigating length correlations in rlhf,” *arXiv preprint arXiv:2310.03716*, 2023.
- [10] R. Park, R. Rafailov, S. Ermon, and C. Finn, “Disentangling length from quality in direct preference optimization,” *arXiv preprint arXiv:2403.19159*, 2024.
- [11] Allen Institute for Artificial Intelligence, “OLMoE, meet iOS.” <https://allenai.org/blog/olmoe-app>, 2025.
- [12] C. Zhou *et al.*, “Lima: Less is more for alignment,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 55006–55021, 2023.
- [13] R. Taori *et al.*, “Stanford alpaca: An instruction-following LLaMA model,” *GitHub repository*. https://github.com/tatsu-lab/stanford_alpaca; GitHub, 2023.
- [14] W.-L. Chiang *et al.*, “Vicuna: An open-source chatbot impressing GPT-4 with 90%* ChatGPT quality.” 2023. Available: <https://lmsys.org/blog/2023-03-30-vicuna/>
- [15] X. Geng *et al.*, “Koala: A dialogue model for academic research.” Blog post, 2023. Accessed: Apr. 03, 2023. [Online]. Available: <https://bair.berkeley.edu/blog/2023/04/03/koala/>
- [16] M. Conover *et al.*, “Hello dolly: Democratizing the magic of ChatGPT with open models.” Accessed: Jun. 30, 2023. [Online]. Available: <https://www.databricks.com/blog/2023/03/24/hello-dolly-democratizing-magic-chatgpt-open-models.html>

- [17] A. Askell *et al.*, “A general language assistant as a laboratory for alignment,” *arXiv preprint arXiv:2112.00861*, 2021.
- [18] Y. Bai *et al.*, “Constitutional ai: Harmlessness from ai feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [19] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [20] L. Tunstall *et al.*, “Zephyr: Direct distillation of lm alignment,” *arXiv preprint arXiv:2310.16944*, 2023.
- [21] H. Ivison *et al.*, “Camels in a changing climate: Enhancing lm adaptation with tulu 2,” *arXiv preprint arXiv:2311.10702*, 2023.
- [22] G. Cui *et al.*, “Ultrafeedback: Boosting language models with high-quality feedback,” 2023.
- [23] A. Dubey *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [24] B. Adler *et al.*, “Nemotron-4 340B technical report,” *arXiv preprint arXiv:2406.11704*, 2024.
- [25] C. Wirth, R. Akrou, G. Neumann, and J. Fürnkranz, “A survey of preference-based reinforcement learning methods,” *Journal of Machine Learning Research*, vol. 18, no. 136, pp. 1–46, 2017.
- [26] T. Kaufmann, P. Weng, V. Bengs, and E. Hüllermeier, “A survey of reinforcement learning from human feedback,” *arXiv preprint arXiv:2312.14925*, 2023.
- [27] S. Casper *et al.*, “Open problems and fundamental limitations of reinforcement learning from human feedback,” *arXiv preprint arXiv:2307.15217*, 2023.
- [28] W. B. Knox and P. Stone, “Tamer: Training an agent manually via evaluative reinforcement,” in *2008 7th IEEE international conference on development and learning*, IEEE, 2008, pp. 292–297.
- [29] J. MacGlashan *et al.*, “Interactive learning from policy-dependent human feedback,” in *International conference on machine learning*, PMLR, 2017, pp. 2285–2294.
- [30] B. Ibarz, J. Leike, T. Pohlen, G. Irving, S. Legg, and D. Amodei, “Reward learning from human preferences and demonstrations in atari,” *Advances in neural information processing systems*, vol. 31, 2018.
- [31] G. Warnell, N. Waytowich, V. Lawhern, and P. Stone, “Deep tamer: Interactive agent shaping in high-dimensional state spaces,” in *Proceedings of the AAAI conference on artificial intelligence*, 2018.
- [32] J. Leike, D. Krueger, T. Everitt, M. Martic, V. Maini, and S. Legg, “Scalable agent alignment via reward modeling: A research direction,” *arXiv preprint arXiv:1811.07871*, 2018.
- [33] D. M. Ziegler *et al.*, “Fine-tuning language models from human preferences,” *arXiv preprint arXiv:1909.08593*, 2019.

- [34] J. Wu *et al.*, “Recursively summarizing books with human feedback,” *arXiv preprint arXiv:2109.10862*, 2021.
- [35] J. Menick *et al.*, “Teaching language models to support answers with verified quotes,” *arXiv preprint arXiv:2203.11147*, 2022.
- [36] A. Glaese *et al.*, “Improving alignment of dialogue agents via targeted human judgements,” *arXiv preprint arXiv:2209.14375*, 2022.
- [37] L. Gao, J. Schulman, and J. Hilton, “Scaling laws for reward model overoptimization,” in *International conference on machine learning*, PMLR, 2023, pp. 10835–10866.
- [38] D. Ganguli *et al.*, “Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned,” *arXiv preprint arXiv:2209.07858*, 2022.
- [39] R. Ramamurthy *et al.*, “Is reinforcement learning (not) for natural language processing: Benchmarks, baselines, and building blocks for natural language policy optimization,” *arXiv preprint arXiv:2210.01241*, 2022.
- [40] A. Havrilla *et al.*, “TrIX: A framework for large scale reinforcement learning from human feedback,” in *Proceedings of the 2023 conference on empirical methods in natural language processing*, Singapore: Association for Computational Linguistics, Dec. 2023, pp. 8578–8595. doi: 10.18653/v1/2023.emnlp-main.530.
- [41] L. von Werra *et al.*, “TRL: Transformer reinforcement learning,” *GitHub repository*. <https://github.com/huggingface/trl>; GitHub, 2020.
- [42] OpenAI, “ChatGPT: Optimizing language models for dialogue.” <https://openai.com/blog/chatgpt/>, 2022.
- [43] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [44] H. Lightman *et al.*, “Let’s verify step by step,” *arXiv preprint arXiv:2305.20050*, 2023.
- [45] A. Kumar *et al.*, “Training language models to self-correct via reinforcement learning,” *arXiv preprint arXiv:2409.12917*, 2024.
- [46] A. Singh *et al.*, “Beyond human data: Scaling self-training for problem-solving with language models,” *arXiv preprint arXiv:2312.06585*, 2023.
- [47] OpenAI, “Introducing OpenAI o1-preview.” Sep. 2024. Available: <https://openai.com/index/introducing-openai-o1-preview/>
- [48] A. Vaswani *et al.*, “Attention is all you need,” in *Neural information processing systems*, 2017. Available: <https://api.semanticscholar.org/CorpusID:13756489>
- [49] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *CoRR*, vol. abs/1409.0473, 2014, Available: <https://api.semanticscholar.org/CorpusID:11212020>
- [50] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [51] G. Team *et al.*, “Gemma 2: Improving open language models at a practical size,” *arXiv preprint arXiv:2408.00118*, 2024.

- [52] R. Agarwal *et al.*, “On-policy distillation of language models: Learning from self-generated mistakes,” in *The twelfth international conference on learning representations*, 2024.
- [53] J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” *Advances in neural information processing systems*, vol. 35, pp. 24824–24837, 2022.
- [54] R. S. Sutton, “Reinforcement learning: An introduction,” *A Bradford Book*, 2018.
- [55] N. Lambert, L. Castriato, L. von Werra, and A. Havrilla, “Illustrating reinforcement learning from human feedback (RLHF),” *Hugging Face Blog*, 2022.
- [56] N. Lambert, T. K. Gilbert, and T. Zick, “Entangled preferences: The history and risks of reinforcement learning and human feedback,” *arXiv preprint arXiv:2310.13595*, 2023.
- [57] V. Conitzer *et al.*, “Social choice should guide AI alignment in dealing with diverse human feedback,” *arXiv preprint arXiv:2404.10271*, 2024.
- [58] A. Mishra, “Ai alignment and social choice: Fundamental limitations and policy implications,” *arXiv preprint arXiv:2310.16048*, 2023.
- [59] H. R. Kirk *et al.*, “The PRISM alignment project: What participatory, representative and individualised human feedback reveals about the subjective and multicultural alignment of large language models,” *arXiv preprint arXiv:2404.16019*, 2024.
- [60] S. Poddar, Y. Wan, H. Ivison, A. Gupta, and N. Jaques, “Personalizing reinforcement learning from human feedback with variational preference learning,” *arXiv preprint arXiv:2408.10075*, 2024.
- [61] S. J. Russell and P. Norvig, *Artificial intelligence: A modern approach*. Pearson, 2016.
- [62] B. Widrow and M. E. Hoff, “Adaptive switching circuits,” Stanford Univ Ca Stanford Electronics Labs, 1960.
- [63] B. F. Skinner, *The behavior of organisms: An experimental analysis*. BF Skinner Foundation, 2019.
- [64] E. L. Thorndike, “The law of effect,” *The American journal of psychology*, vol. 39, no. 1/4, pp. 212–222, 1927.
- [65] A. Arnauld, *The port-royal logic*. 1662.
- [66] J. Bentham, *An introduction to the principles of morals and legislation*. 1823.
- [67] F. P. Ramsey, “Truth and probability,” *Readings in Formal Epistemology: Sourcebook*, pp. 21–45, 2016.
- [68] K. J. Arrow, “A difficulty in the concept of social welfare,” *Journal of political economy*, vol. 58, no. 4, pp. 328–346, 1950.
- [69] J. C. Harsanyi, “Rule utilitarianism and decision theory,” *Erkenntnis*, vol. 11, no. 1, pp. 25–53, 1977.
- [70] R. Pettigrew, *Choosing for changing selves*. Oxford University Press, 2019.

- [71] N. Soares, B. Fallenstein, S. Armstrong, and E. Yudkowsky, “Corrigibility,” in *Workshops at the twenty-ninth AAAI conference on artificial intelligence*, 2015.
- [72] W.-L. Chiang *et al.*, “Chatbot arena: An open platform for evaluating llms by human preference,” *arXiv preprint arXiv:2403.04132*, 2024.
- [73] R. Likert, “A technique for the measurement of attitudes.” *Archives of psychology*, 1932.
- [74] J. Zhou *et al.*, “Instruction-following evaluation for large language models,” *arXiv preprint arXiv:2311.07911*, 2023.
- [75] K. Ethayarajh, W. Xu, N. Muennighoff, D. Jurafsky, and D. Kiela, “Kto: Model alignment as prospect theoretic optimization,” *arXiv preprint arXiv:2402.01306*, 2024.
- [76] Z. Wu *et al.*, “Fine-grained human feedback gives better rewards for language model training,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [77] A. Chen *et al.*, “Learning from natural language feedback,” *Transactions on Machine Learning Research*, 2024.
- [78] OpenAI, “Introducing the model spec.” May 2024. Available: <https://openai.com/index/introducing-the-model-spec/>
- [79] A. Y. Ng, S. Russell, *et al.*, “Algorithms for inverse reinforcement learning.” in *Icml*, 2000, p. 2.
- [80] R. A. Bradley and M. E. Terry, “Rank analysis of incomplete block designs: I. The method of paired comparisons,” *Biometrika*, vol. 39, no. 3/4, pp. 324–345, 1952, Accessed: Feb. 13, 2023. [Online]. Available: <http://www.jstor.org/stable/2334029>
- [81] B. Zhu *et al.*, “Starling-7b: Improving helpfulness and harmlessness with rlaiif,” in *First conference on language modeling*, 2024.
- [82] A. Liu, Z. Zhao, C. Liao, P. Lu, and L. Xia, “Learning plackett-luce mixtures from partial preferences,” in *Proceedings of the AAAI conference on artificial intelligence*, 2019, pp. 4328–4335.
- [83] B. Zhu, M. Jordan, and J. Jiao, “Principled reinforcement learning with human feedback from pairwise or k-wise comparisons,” in *International conference on machine learning*, PMLR, 2023, pp. 43037–43067.
- [84] K. Cobbe *et al.*, “Training verifiers to solve math word problems,” *arXiv preprint arXiv:2110.14168*, 2021.
- [85] C. Lyu *et al.*, “Exploring the limit of outcome reward for learning mathematical reasoning,” *arXiv preprint arXiv:2502.06781*, 2025.
- [86] L. Zheng *et al.*, “Judging llm-as-a-judge with mt-bench and chatbot arena,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 46595–46623, 2023.
- [87] Y. Dubois, B. Galambosi, P. Liang, and T. B. Hashimoto, “Length-controlled alpacaeval: A simple way to debias automatic evaluators,” *arXiv preprint arXiv:2404.04475*, 2024.

- [88] T. Li *et al.*, “From crowdsourced data to high-quality benchmarks: Arena-hard and BenchBuilder pipeline,” *arXiv preprint arXiv:2406.11939*, 2024.
- [89] B. Y. Lin *et al.*, “WILDBENCH: Benchmarking LLMs with challenging tasks from real users in the wild,” *arXiv preprint arXiv:2406.04770*, 2024.
- [90] D. Mahan *et al.*, “Generative reward models,” 2024, Available: https://www.synthlabs.ai/pdf/Generative_Reward_Models.pdf
- [91] L. Zhang, A. Hosseini, H. Bansal, M. Kazemi, A. Kumar, and R. Agarwal, “Generative verifiers: Reward modeling as next-token prediction,” *arXiv preprint arXiv:2408.15240*, 2024.
- [92] Z. Ankner, M. Paul, B. Cui, J. D. Chang, and P. Ammanabrolu, “Critique-out-loud reward models,” *arXiv preprint arXiv:2408.11791*, 2024.
- [93] S. Kim *et al.*, “Prometheus: Inducing fine-grained evaluation capability in language models,” in *The twelfth international conference on learning representations*, 2023.
- [94] N. Lambert *et al.*, “Rewardbench: Evaluating reward models for language modeling,” *arXiv preprint arXiv:2403.13787*, 2024.
- [95] X. Wen *et al.*, “Rethinking reward model evaluation: Are we barking up the wrong tree?” *arXiv preprint arXiv:2410.05584*, 2024.
- [96] S. Gureja *et al.*, “M-RewardBench: Evaluating reward models in multi-lingual settings,” *arXiv preprint arXiv:2410.15522*, 2024.
- [97] Z. Jin *et al.*, “RAG-RewardBench: Benchmarking reward models in retrieval augmented generation for preference alignment,” *arXiv preprint arXiv:2412.13746*, 2024.
- [98] E. Zhou *et al.*, “RMB: Comprehensively benchmarking reward models in LLM alignment,” *arXiv preprint arXiv:2410.09893*, 2024.
- [99] Z. Wu, M. Yasunaga, A. Cohen, Y. Kim, A. Celikyilmaz, and M. Ghazvininejad, “reWordBench: Benchmarking and improving the robustness of reward models with transformed inputs,” *arXiv preprint arXiv:2503.11751*, 2025.
- [100] Z. Chen *et al.*, “MJ-bench: Is your multimodal reward model really a good judge for text-to-image generation?” *arXiv preprint arXiv:2407.04842*, 2024.
- [101] M. Yasunaga, L. Zettlemoyer, and M. Ghazvininejad, “Multimodal rewardbench: Holistic evaluation of reward models for vision language models,” *arXiv preprint arXiv:2502.14191*, 2025.
- [102] L. Li *et al.*, “VLRewardBench: A challenging benchmark for vision-language generative reward models,” *arXiv preprint arXiv:2411.17451*, 2024.
- [103] J. Ruan *et al.*, “Vlrmbench: A comprehensive and challenging benchmark for vision-language reward models,” *arXiv preprint arXiv:2503.07478*, 2025.
- [104] E. Frick *et al.*, “How to evaluate reward models for RLHF,” *arXiv preprint arXiv:2410.14872*, 2024.

- [105] S. Kim *et al.*, “Evaluating robustness of reward models for mathematical reasoning,” *arXiv preprint arXiv:2410.01729*, 2024.
- [106] M. Song, Z. Su, X. Qu, J. Zhou, and Y. Cheng, “PRMBench: A fine-grained and challenging benchmark for process-level reward models,” *arXiv preprint arXiv:2501.03124*, 2025.
- [107] W. Wang *et al.*, “VisualPRM: An effective process reward model for multimodal reasoning,” *arXiv preprint arXiv:2503.10291*, 2025.
- [108] H. Tu, W. Feng, H. Chen, H. Liu, X. Tang, and C. Xie, “ViLBench: A suite for vision-language process reward modeling.” Mar. 2025. Available: <https://arxiv.org/abs/2503.20271>
- [109] H. Wang, W. Xiong, T. Xie, H. Zhao, and T. Zhang, “Interpretable preferences via multi-objective reward modeling and mixture-of-experts,” *arXiv preprint arXiv:2406.12845*, 2024.
- [110] Z. Wang *et al.*, “HelpSteer2: Open-source dataset for training top-performing reward models,” *arXiv preprint arXiv:2406.08673*, 2024.
- [111] Z. Wang *et al.*, “HelpSteer2-preference: Complementing ratings with preferences,” *arXiv preprint arXiv:2410.01257*, 2024.
- [112] J. Park, S. Jwa, M. Ren, D. Kim, and S. Choi, “Offsetbias: Leveraging debiased data for tuning evaluators,” *arXiv preprint arXiv:2407.06551*, 2024.
- [113] N. Jaques, S. Gu, D. Bahdanau, J. M. Hernández-Lobato, R. E. Turner, and D. Eck, “Sequence tutor: Conservative fine-tuning of sequence generation models with kl-control,” in *International conference on machine learning*, PMLR, 2017, pp. 1645–1654.
- [114] N. Jaques *et al.*, “Human-centric dialog training via offline reinforcement learning,” *arXiv preprint arXiv:2010.05848*, 2020.
- [115] J. Schulman, “Approximating KL-divergence.” <http://joschu.net/blog/kl-approx.html>, 2016.
- [116] R. Y. Pang, W. Yuan, K. Cho, H. He, S. Sukhbaatar, and J. Weston, “Iterative reasoning preference optimization,” *arXiv preprint arXiv:2404.19733*, 2024.
- [117] Z. Gao *et al.*, “Rebel: Reinforcement learning via regressing relative rewards,” *arXiv preprint arXiv:2404.16767*, 2024.
- [118] T. Brown *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [119] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of machine learning research*, vol. 21, no. 140, pp. 1–67, 2020.
- [120] J. Wei *et al.*, “Finetuned language models are zero-shot learners,” *arXiv preprint arXiv:2109.01652*, 2021.
- [121] V. Sanh *et al.*, “Multitask prompted training enables zero-shot task generalization,” *arXiv preprint arXiv:2110.08207*, 2021.

- [122] S. Mishra, D. Khashabi, C. Baral, and H. Hajishirzi, “Cross-task generalization via natural language crowdsourcing instructions,” *arXiv preprint arXiv:2104.08773*, 2021.
- [123] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” *Advances in neural information processing systems*, vol. 36, pp. 10088–10115, 2023.
- [124] N. Rajani, L. Tunstall, E. Beeching, N. Lambert, A. M. Rush, and T. Wolf, “No robots,” *Hugging Face repository*. https://huggingface.co/datasets/HuggingFaceH4/no_robots; Hugging Face, 2023.
- [125] W. R. Gilks and P. Wild, “Adaptive rejection sampling for gibbs sampling,” *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, vol. 41, no. 2, pp. 337–348, 1992.
- [126] A. Ahmadian *et al.*, “Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms,” *arXiv preprint arXiv:2402.14740*, 2024.
- [127] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [128] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine learning*, vol. 8, pp. 229–256, 1992.
- [129] S. C. Huang, A. Ahmadian, and C. F. AI, “Putting RL back in RLHF,” https://huggingface.co/blog/putting_rl_back_in_rlhf_with_rloo, 2024.
- [130] W. Kool, H. van Hoof, and M. Welling, “Buy 4 reinforce samples, get a baseline for free!” 2019.
- [131] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [132] C. Berner *et al.*, “Dota 2 with large scale deep reinforcement learning,” *arXiv preprint arXiv:1912.06680*, 2019.
- [133] Z. Liu *et al.*, “Understanding R1-zero-like training: A critical perspective,” *arXiv preprint arXiv:2503.20783*, Mar. 2025, Available: <https://arxiv.org/abs/2503.20783>
- [134] Z. Shao *et al.*, “Deepseekmath: Pushing the limits of mathematical reasoning in open language models,” *arXiv preprint arXiv:2402.03300*, 2024.
- [135] A. Liu *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024.
- [136] D. Guo *et al.*, “Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning,” *arXiv preprint arXiv:2501.12948*, 2025.
- [137] H. Ivison *et al.*, “Unpacking DPO and PPO: Disentangling best practices for learning from preference feedback,” *arXiv preprint arXiv:2406.09279*, 2024.

- [138] S. Huang, M. Noukhovitch, A. Hosseini, K. Rasul, W. Wang, and L. Tunstall, “The n+ implementation details of RLHF with PPO: A case study on TL; DR summarization,” *arXiv preprint arXiv:2403.17031*, 2024.
- [139] L. Weng, “Policy gradient algorithms,” *lilianweng.github.io*, 2018, Available: <https://lilianweng.github.io/posts/2018-04-08-policy-gradient/>
- [140] D. Seita, “Notes on the generalized advantage estimation paper.” 2017. Available: <https://danieltakeshi.github.io/2017/04/02/notes-on-the-generalized-advantage-estimation-paper/>
- [141] T. Wu, B. Zhu, R. Zhang, Z. Wen, K. Ramchandran, and J. Jiao, “Pairwise proximal policy optimization: Harnessing relative feedback for llm alignment,” *arXiv preprint arXiv:2310.00212*, 2023.
- [142] Z. Li *et al.*, “Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models,” in *Forty-first international conference on machine learning*, 2023.
- [143] T. Gunter *et al.*, “Apple intelligence foundation language models,” *arXiv preprint arXiv:2407.21075*, 2024.
- [144] K. Team *et al.*, “Kimi k1. 5: Scaling reinforcement learning with llms,” *arXiv preprint arXiv:2501.12599*, 2025.
- [145] M. Tomar, L. Shani, Y. Efroni, and M. Ghavamzadeh, “Mirror descent policy optimization,” *arXiv preprint arXiv:2005.09814*, 2020.
- [146] Y. Zhang *et al.*, “Improving LLM general preference alignment via optimistic online mirror descent,” *arXiv preprint arXiv:2502.16852*, 2025.
- [147] Q. Yu *et al.*, “DAPO: An open-source LLM reinforcement learning system at scale.” 2025.
- [148] Y. Zhao, R. Joshi, T. Liu, M. Khalman, M. Saleh, and P. J. Liu, “Slic-hf: Sequence likelihood calibration with human feedback,” *arXiv preprint arXiv:2305.10425*, 2023.
- [149] M. G. Azar *et al.*, “A general theoretical paradigm to understand learning from human preferences,” in *International conference on artificial intelligence and statistics*, PMLR, 2024, pp. 4447–4455.
- [150] A. Amini, T. Vieira, and R. Cotterell, “Direct preference optimization with an offset,” *arXiv preprint arXiv:2402.10571*, 2024.
- [151] J. Hong, N. Lee, and J. Thorne, “Reference-free monolithic preference optimization with odds ratio,” *arXiv e-prints*, pp. arXiv–2403, 2024.
- [152] Y. Meng, M. Xia, and D. Chen, “Simpo: Simple preference optimization with a reference-free reward,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 124198–124235, 2025.
- [153] N. Razin, S. Malladi, A. Bhaskar, D. Chen, S. Arora, and B. Hanin, “Unintentional unalignment: Likelihood displacement in direct preference optimization,” *arXiv preprint arXiv:2410.08847*, 2024.
- [154] Y. Ren and D. J. Sutherland, “Learning dynamics of llm finetuning,” *arXiv preprint arXiv:2407.10490*, 2024.

- [155] T. Xiao, Y. Yuan, H. Zhu, M. Li, and V. G. Honavar, “Cal-dpo: Calibrated direct preference optimization for language model alignment,” *arXiv preprint arXiv:2412.14516*, 2024.
- [156] S. Guo *et al.*, “Direct language model alignment from online ai feedback,” *arXiv preprint arXiv:2402.04792*, 2024.
- [157] P. Singhal, N. Lambert, S. Niekum, T. Goyal, and G. Durrett, “D2po: Discriminator-guided dpo with response evaluation models,” *arXiv preprint arXiv:2405.01511*, 2024.
- [158] C. Rosset, C.-A. Cheng, A. Mitra, M. Santacroce, A. Awadallah, and T. Xie, “Direct nash optimization: Teaching language models to self-improve with general preferences,” *arXiv preprint arXiv:2404.03715*, 2024.
- [159] S. Jung, G. Han, D. W. Nam, and K.-W. On, “Binary classifier optimization for large language model alignment,” *arXiv preprint arXiv:2404.04656*, 2024.
- [160] H. Zhao *et al.*, “Rainbowpo: A unified framework for combining improvements in preference optimization,” *arXiv preprint arXiv:2410.04203*, 2024.
- [161] A. Gorbatoevski, B. Shaposhnikov, V. Sinii, A. Malakhov, and D. Gavrilov, “The differences between direct alignment algorithms are a blur,” *arXiv preprint arXiv:2502.01237*, 2025.
- [162] S. Xu *et al.*, “Is dpo superior to ppo for llm alignment? A comprehensive study,” *arXiv preprint arXiv:2404.10719*, 2024.
- [163] F. Tajwar *et al.*, “Preference fine-tuning of llms should leverage suboptimal, on-policy data,” *arXiv preprint arXiv:2404.14367*, 2024.
- [164] H. Lee *et al.*, “Rlaif: Scaling reinforcement learning from human feedback with ai feedback,” 2023.
- [165] A. Sharma, S. Keh, E. Mitchell, C. Finn, K. Arora, and T. Kollar, “A critical evaluation of AI feedback for aligning large language models.” 2024. Available: <https://arxiv.org/abs/2402.12366>
- [166] L. Castricato, N. Lile, S. Anand, H. Schoelkopf, S. Verma, and S. Biderman, “Suppressing pink elephants with direct principle feedback.” 2024. Available: <https://arxiv.org/abs/2402.07896>
- [167] L. J. V. Miranda *et al.*, “Hybrid preferences: Learning to route instances for human vs. AI feedback,” *arXiv preprint arXiv:2410.19133*, 2024.
- [168] T. Wang *et al.*, “Shepherd: A critic for language model generation,” *arXiv preprint arXiv:2308.04592*, 2023.
- [169] P. Ke *et al.*, “CritiqueLLM: Towards an informative critique generation model for evaluation of large language model generation,” *arXiv preprint arXiv:2311.18702*, 2023.
- [170] J. Li, S. Sun, W. Yuan, R.-Z. Fan, H. Zhao, and P. Liu, “Generative judge for evaluating alignment,” *arXiv preprint arXiv:2310.05470*, 2023.

- [171] S. Kim *et al.*, “Prometheus 2: An open source language model specialized in evaluating other language models,” *arXiv preprint arXiv:2405.01535*, 2024.
- [172] S. Lee, S. Kim, S. Park, G. Kim, and M. Seo, “Prometheus-vision: Vision-language model as a judge for fine-grained evaluation,” in *Findings of the association for computational linguistics ACL 2024*, 2024, pp. 11286–11315.
- [173] M. Y. Guan *et al.*, “Deliberative alignment: Reasoning enables safer language models,” *arXiv preprint arXiv:2412.16339*, 2024.
- [174] Anthropic, “Claude’s constitution.” Accessed: Feb. 07, 2024. [Online]. Available: <https://www.anthropic.com/news/claudes-constitution>
- [175] D. Ganguli *et al.*, “Collective constitutional AI: Aligning a language model with public input.” Anthropic, 2023.
- [176] S. Huang *et al.*, “Constitutional AI recipe,” *Hugging Face Blog*, 2024.
- [177] N. Lambert, H. Schoelkopf, A. Gokaslan, L. Soldaini, V. Pyatkin, and L. Castricato, “Self-directed synthetic dialogues and revisions technical report,” *arXiv preprint arXiv:2407.18421*, 2024.
- [178] Z. Sun *et al.*, “Principle-driven self-alignment of language models from scratch with minimal human supervision,” in *Thirty-seventh conference on neural information processing systems*, 2023. Available: <https://openreview.net/forum?id=p40XRfBX96>
- [179] Z. Sun *et al.*, “SALMON: Self-alignment with principle-following reward models,” in *The twelfth international conference on learning representations*, 2024. Available: <https://openreview.net/forum?id=xJbsmB8UMx>
- [180] I. Shumailov, Z. Shumaylov, Y. Zhao, N. Papernot, R. Anderson, and Y. Gal, “AI models collapse when trained on recursively generated data,” *Nature*, vol. 631, no. 8022, pp. 755–759, 2024.
- [181] M. Gerstgrasser *et al.*, “Is model collapse inevitable? Breaking the curse of recursion by accumulating real and synthetic data,” *arXiv preprint arXiv:2404.01413*, 2024.
- [182] Y. Feng, E. Dohmatob, P. Yang, F. Charton, and J. Kempe, “Beyond model collapse: Scaling up with synthesized data requires reinforcement,” in *ICML 2024 workshop on theoretical foundations of foundation models*, 2024.
- [183] Y. Wang *et al.*, “Self-instruct: Aligning language models with self-generated instructions,” *arXiv preprint arXiv:2212.10560*, 2022.
- [184] E. Beeching *et al.*, “NuminaMath 7B TIR,” *Hugging Face repository*. <https://huggingface.co/AI-MO/NuminaMath-7B-TIR>; Numina & Hugging Face, 2024.
- [185] M. Li *et al.*, “Superfiltering: Weak-to-strong data filtering for fast instruction-tuning,” *arXiv preprint arXiv:2402.00530*, 2024.
- [186] N. Lambert and R. Calandra, “The alignment ceiling: Objective mismatch in reinforcement learning from human feedback,” *arXiv preprint arXiv:2311.00168*, 2023.

- [187] N. Lambert, B. Amos, O. Yadan, and R. Calandra, “Objective mismatch in model-based reinforcement learning,” *arXiv preprint arXiv:2002.04523*, 2020.
- [188] J. Schulman, “Proxy objectives in reinforcement learning from human feedback.” Invited talk at the International Conference on Machine Learning (ICML), 2023. Available: <https://icml.cc/virtual/2023/invited-talk/21549>
- [189] C. A. Goodhart and C. Goodhart, *Problems of monetary management: The UK experience*. Springer, 1984.
- [190] M. Sharma *et al.*, “Towards understanding sycophancy in language models,” *arXiv preprint arXiv:2310.13548*, 2023.
- [191] T. Lu and C. Boutilier, “Learning mallows models with pairwise preferences,” in *Proceedings of the 28th international conference on machine learning (icml-11)*, 2011, pp. 145–152.
- [192] S. Zhuang and D. Hadfield-Menell, “Consequences of misaligned AI,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 15763–15773, 2020.